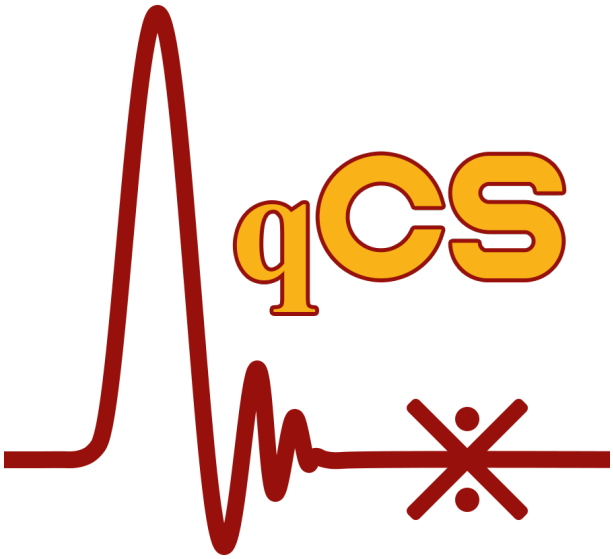


User Guide

Version 3.2

Sponsor: Intelligence Advanced Research Projects Activity (IARPA) | Coldflux Project

Developers: Mustafa Altay Karamuftuoglu, Changxu Song, Haolin Cong, Massoud Pedram
SPORT Lab

©University of Southern California
740 McClintock Ave • EEB 344
Los Angeles, CA, 90089-2562

May 15, 2026

License

©Copyright (C) 2021–2026 Mustafa Altay Karamuftuoglu, Changxu Song, Haolin Cong, and Massoud Pedram

SPORT lab, University of Southern California, Los Angeles, CA 90089 All rights reserved.

Permission is hereby granted, without written agreement and without license or royalty fees, to use, copy, and modify, this software and its documentation for any non-commercial purpose, provided that the above copyright notice and the following paragraphs appear in all copies of this software, whether in binary form or not. Commercial use of this software requires written agreement which must be obtained from Massoud Pedram <pedram@usc.edu>.

All trademarks are the property of their respective owners.

- Linux is a registered trademark of Linus Torvalds.
- Mac OS is a registered trademark of Apple Inc.
- Windows is a registered trademark of Microsoft Corporation.
- MATLAB is a registered trademark of The MathWorks, Inc.
- JSIM is originally written by Emerson S. Fang as part of his work in Ted Van Duzer's lab at the University of California, Berkeley.
- JoSIM is originally written by Johannes Delport at Stellenbosch University, Stellenbosch.
- PJSIM is a modified version of the JSIM circuit simulator, and it is developed by Yamanashi Group, Yokohama National University, Yokohama.

IN NO EVENT SHALL THE AUTHORS OR THE UNIVERSITY OF SOUTHERN CALIFORNIA BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THEY HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND THE UNIVERSITY OF SOUTHERN CALIFORNIA SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE AUTHORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

The development of this tool was supported by the Office of the Director of National Intelligence (ODNI), Intelligence Advanced Research Projects Activity (IARPA) SuperTools program, via the U.S. Army Research Office grant W911NF-17-1-0120.

The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the ODNI, IARPA, or the U.S. Government. The U.S. Government is authorized to reproduce and distribute copies for Governmental purposes notwithstanding any copyright notation herein.

Contents

1 Introduction **5**
- 1.1 Overview . . . 5
- 1.2 Audience . . . 6
- 1.3 System Requirements . . . 6
- 1.4 Installation . . . 6
  - 1.4.1 Windows OS . . . 6
  - 1.4.2 CentOS 7 / Ubuntu 22 . . . 7
  - 1.4.3 macOS . . . 7
- 1.5 Key Features . . . 7
- 1.6 Tool Flow . . . 8
- 1.7 Interface Introduction . . . 8
  - 1.7.1 Netlist . . . 9
  - 1.7.2 Simulation . . . 10
  - 1.7.3 Variables . . . 10
  - 1.7.4 Margin . . . 13
  - 1.7.5 Yield . . . 14
  - 1.7.6 Optimization . . . 15
  - 1.7.7 Manual . . . 18
  - 1.7.8 Information . . . 18

2 Netlist File Selection and Editing **20**
- 2.0.1 Current Controlled Current Source . . . 24
- 2.0.2 Current Controlled Voltage Source . . . 24
- 2.0.3 Voltage Controlled Current Source . . . 24
- 2.0.4 Voltage Controlled Voltage Source . . . 24
- 2.0.5 Voltage Source . . . 24
- 2.0.6 Current Source . . . 25
- 2.0.7 Phase Source . . . 25
- 2.0.8 Source Types . . . 25
  - 2.0.8.1 Piecewise Linear (PWL) . . . 25
  - 2.0.8.2 Pulse . . . 26
  - 2.0.8.3 Sinusoidal (SIN) . . . 26
  - 2.0.8.4 Custom Waveform (CUS) . . . 26
  - 2.0.8.5 DC . . . 26
  - 2.0.8.6 Noise . . . 26
  - 2.0.8.7 Exponential (EXP) . . . 27

3 Simulation **31**

4 Variable Parameter Specification **32**
- 4.1 Int Group and Ext Group columns . . . 32
- 4.2 Group Config dialog and Serial JJ chain rewrite . . . 33
  - 4.2.0.1 What happens when the user clicks OK. . . . 35
- 4.3 Load Group Preferences . . . 36
  - 4.3.0.1 Internal-L / JJ consistency check. . . . 36
  - 4.3.0.2 Failure preserves user edits. . . . 36

4.4 Min / Initial / Max value rules . . . 37

5 Margin Calculation **38**
5.1 Pulse Settings sub-tab . . . 38
5.2 Phase Settings sub-tab . . . 39
5.3 Sub-tab choice and analysis-mode validation . . . 40
5.4 Pattern syntax and examples . . . 41
5.5 How a pattern is matched . . . 41
5.6 Fill, Check, and Calculate . . . 43
5.7 Per-component margin scans . . . 43
5.8 Margin overrun gating against pre-extension TSTOP . . . 44
5.9 Visualize and Export . . . 44

6 Yield Analysis **45**
6.1 Group behaviour during yield . . . 45
6.2 Synthetic inter-L rows are skipped . . . 46

7 Optimization **47**
7.1 Ext Group secondaries in optimization cost . . . 47
7.1.0.1 Final margin output is always full-coverage. . . . 47
7.1.0.2 Runtime cost. . . . 48
7.2 TSTOP auto-extension during optimization . . . 48
7.3 Parallel workers and pulse mode . . . 48
7.4 Phase-mode optimization . . . 48

8 Examples **49**
8.1 RSFQ . . . 49
8.1.1 Loading Netlist . . . 49
8.1.2 Simulation . . . 49
8.1.3 Selecting Components . . . 49
8.1.4 Critical Margin Calculation . . . 52
8.1.5 Yield Analysis . . . 52
8.1.6 Critical Margin Optimization . . . 52
8.2 AQFP . . . 57
8.2.1 Loading Netlist . . . 57
8.2.2 Simulation . . . 57
8.2.3 Selecting Components . . . 57
8.2.4 Critical Margin Calculation . . . 57
8.2.5 Yield Analysis . . . 59
8.2.6 Critical Margin Optimization . . . 60
8.3 RQL . . . 60
8.3.1 Loading Netlist . . . 60
8.3.2 Simulation . . . 61
8.3.3 Selecting Components . . . 61
8.3.4 Critical Margin Calculation . . . 61
8.3.5 Yield Analysis . . . 62
8.3.6 Critical Margin Optimization . . . 62
8.4 Fast Phase Logic (FPL) . . . 64

8.4.1 Loading Netlist 64
8.4.2 Simulation 64
8.4.3 Selecting Components and Configuring the Chain 65
8.4.4 Critical Margin Calculation 67
8.4.5 Yield Analysis 68
8.4.6 Critical Margin Optimization 69

9 Notes **74**

1 Introduction

1.1 Overview

qCS (Cell Synthesis) is a tool for analyzing and optimizing small superconductor circuits, such as logic cells. qCS is a *general* superconductor analysis tool: it supports five logic families that differ only in how their output waveforms are interpreted. Every family supports both pulse-height and phase-jump analysis modes — the user selects the analysis mode in the GUI (via the *Pulse Settings* / *Phase Settings* sub-tabs) rather than declaring it in the netlist file. The bullets below describe each family's output behavior; the choice of analysis mode is independent of family and is entirely up to the user.

- **RSFQ / SFQ pulse logic** (default). Output node carries narrow voltage spikes; each spike represents one Single Flux Quantum (SFQ). Both pulse-height detection (counting the spikes) and phase-jump detection (tracking the certain amount of phase jump each SFQ produces) are supported.
- **AQFP** (Adiabatic Quantum-Flux-Parametron). Bipolar output driven by an excitation clock: each clock phase produces a positive pulse for logic-1 or a negative pulse for logic-0. Both pulse-height detection (with explicit negative-pulse support) and phase-jump detection are supported.
- **RQL** (Reciprocal Quantum Logic). Logic 0 produces no output pulses; logic 1 is encoded as a reciprocal pulse pair — one positive pulse immediately followed by one negative pulse. In phase view, logic 1 appears as a positive 2π jump followed by a negative 2π jump that returns the phase to its baseline; logic 0 leaves the phase flat. Both pulse-height detection (catching the $+/-$ pulse pair) and phase-jump detection (catching the $+2\pi/-2\pi$ return-to-baseline pair) are supported.
- **Fast Phase Logic (FPL)**. Circuit topology where multiple self-shunted Josephson junctions are intentionally chained together as a serial JJ structure for relaxation and phase propagation. Both pulse-height detection (observing SFQ pulses on the chain) and phase-jump detection (tracking the 2π jumps as phase propagates along the chain) are supported.
- **PCL** (Pulse Conserving Logic). Dual-rail family where each logic transition emits one bipolar SFQ pulse — positive for the transition to logical 1, negative for the transition to logical 0; SFQ count is conserved through every gate and inversion is free by swapping the two rails. Both pulse-height detection (catching the $+/-$ events on each rail) and phase-jump detection (tracking the $\pm 2\pi$ steps) are supported.

qCS also helps the user generate critical margin / yield-optimized variants of designed cells through three built-in capabilities: critical margin calculation, parametric yield analysis, and critical margin range optimization.

The purpose of this document is to provide an insight into the available features of qCS, and the following sections discuss the details with examples. Images and

screenshots may appear differently on each operating system. Every session can be exported and imported at any time. The generated files are independent of the operating system. Thus, files initially exported on a system with a Windows OS can be imported into a system with CentOS 7 and vice versa.

1.2 Audience

This document is prepared for the superconducting circuit designers. They are expected to be familiar with testbench preparation, circuit simulation, and design verification.

1.3 System Requirements

- *Certificates (non-Windows OS)*: If the installer doesn't start and the error is about certificates, use the following command:

  $ sudo ln -s /etc/ssl/certs/ca-bundle.trust.crt /etc/ssl/certs/ca-certificates.crt

- *JoSIM (non-Windows OS)*: It is the user's responsibility to satisfy JoSIM requirements (setting up the environment and installing packages). For more information, please visit the link below.

  https://joeydelp.github.io/JoSIM/#linux

- *MATLAB Compiler Runtime (MCR) 9.13*: The qCS installer will automatically download MCR from the Mathworks website. If it is already installed, the system will detect the installed path (Windows OS only).

  A minimum of 2 GB of disk capacity is required to accommodate the MCR installation. Any installation directory is permitted for MCR.

- *Directory*: Create a parent folder for the qCS installation directory since qCS deletes its parent folder upon uninstallation. The recommended directory is an appropriate location where the user has read/write permissions.

1.4 Installation

1.4.1 Windows OS

- Installer: "qCS_3.0_installer_win64.exe"

  1. L-click twice on the installer; click "Yes" on the permission prompt if it appears
  2. For MATLAB Runtime, select "Use existing MATLAB Runtime" and point at a MATLAB R2023b or MCR R2023b root, or pick the download option (∼4–5 GB)
  3. Choose an install directory that does not require administrator permissions (example: C:\Users\userName\qCS_3.0)
  4. Click Install

1.4.2 CentOS 7 / Ubuntu 22

- Installer: "qCS_3.0_installer_glnxa64.install"

  1. $ chmod +x qCS_3.0_installer_glnxa64.install
  2. $ ./qCS_3.0_installer_glnxa64.install
  3. For MATLAB Runtime, select "Use existing MATLAB Runtime" and point at a MATLAB R2023b or MCR R2023b root, or pick the download option (∼4–5 GB)
  4. Choose an install directory (example: /home/userName/qCS_3.0)
  5. Click Install

1.4.3 macOS

- Installer: "qCS_3.0_installer_maca64.app" (Apple silicon) or "qCS_3.0_installer_maci64.app" (Intel Macs)

  1. $ xattr -dr com.apple.quarantine qCS_3.0_installer_*.app
  2. $ open qCS_3.0_installer_*.app
  3. For MATLAB Runtime, select "Use existing MATLAB Runtime" and point at a MATLAB R2023b or MCR R2023b root, or pick the download option (∼4–5 GB)
  4. Choose an install directory (example: ~/Applications/qCS_3.0)
  5. Click Install

- On macOS, qCS ships only the JoSIM simulator. After launch, change the *Simulator Choice* dropdown on the Netlist tab from the default *JSIM* to *JoSIM*.

1.5 Key Features

The following chapters are organized to describe the main steps of qCS, and they do not correspond to a specific design flow.

- *Circuit specification*: File selection as an input and netlist adjustments on the interface.
- *Simulation*: Waveform analysis and desired node selection for confirming the correct behavior using JSIM, JSIM_n, and JoSIM.
- *Variable parameter specification*: Flexible parameter selection from a single cell or a multi-cell structure with various fanin and fanout counts. This tab additionally exposes *Int Group* (internal grouping for serial JJ chains) and *Ext Group* (external grouping for non-JJ siblings) so the user can describe complex chain and mirror structures without editing the netlist by hand.
- *Margin calculation*: Parametric sweeps around a design value for all selected cell components. Every physical JJ inside a chain is scanned independently, and Ext-Group secondaries are scanned right after their primaries.
- *Yield analysis*: Sampling and evaluating the randomized netlists with Monte Carlo simulations.
- *Circuit optimization*: Flexible adjustments on the predefined values of the algorithm for efficient optimizations and observing the trend during the process. Group secondaries can be optionally rolled into the optimization cost.

1.6 Tool Flow

qCS requires a netlist as an input. Upon browsing for the file, it converts the circuit into an object while making a copy of the selected netlist. Any adjustments made to the interface will not be reflected in the original file, except for the qCS copy. The simulations are performed using the circuit object by converting it back to a netlist after some adjustments. If there is an issue with the file, a pop-up error will appear. After confirming and selecting the desired waveforms, the next step is to select the cell components and optionally configure their Int / Ext groupings. The search space for the optimization can be changed in this step as well. Even if the output behavior is not as expected, the user can manually provide pattern texts. Otherwise, qCS is capable of automatically creating the patterns for the waveforms, and the pulse or phase information is automatically assigned based on the selected waveforms and the chosen analysis mode (Pulse Settings or Phase Settings sub-tab). At this point, the built-in features with default settings are ready to use. The related flow graph is shown in Fig. 1.

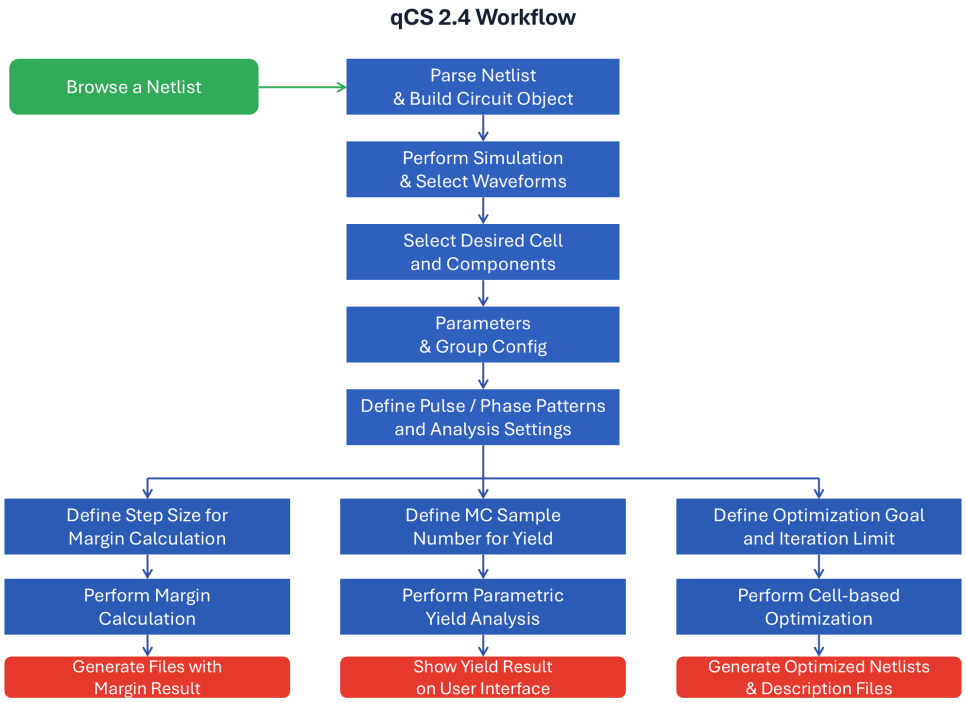


Figure 1: qCS workflow

1.7 Interface Introduction

After installing qCS, locate the installation directory/application. Click on qCS.exe to run the tool. Initially, a loading screen will appear, followed by the main interface of the tool. Each qCS tab is explained in the following sections.

qCS does not encode the logic family in the netlist. The analysis mode is determined by what the netlist prints: if the `.PRINT` statements report phase quantities (e.g. `PHASE`), qCS selects the *Phase Settings* sub-tab on the Margin, Yield, and Optimization tabs; if they report voltages or currents (e.g. `DEVV` / `DEVI`), qCS selects the *Pulse Settings* sub-tab. The two are not interchangeable on the same netlist.

1.7.1 Netlist

Netlist tab contains the testbench environment for the cells to be analyzed. In this tab, the user can import a previously saved qCS session or browse a new circuit netlist. To successfully load a netlist, the user must follow the netlist rules for qCS, as outlined in Section 2. The interface is illustrated in Fig. 2, and each interface component is listed below.

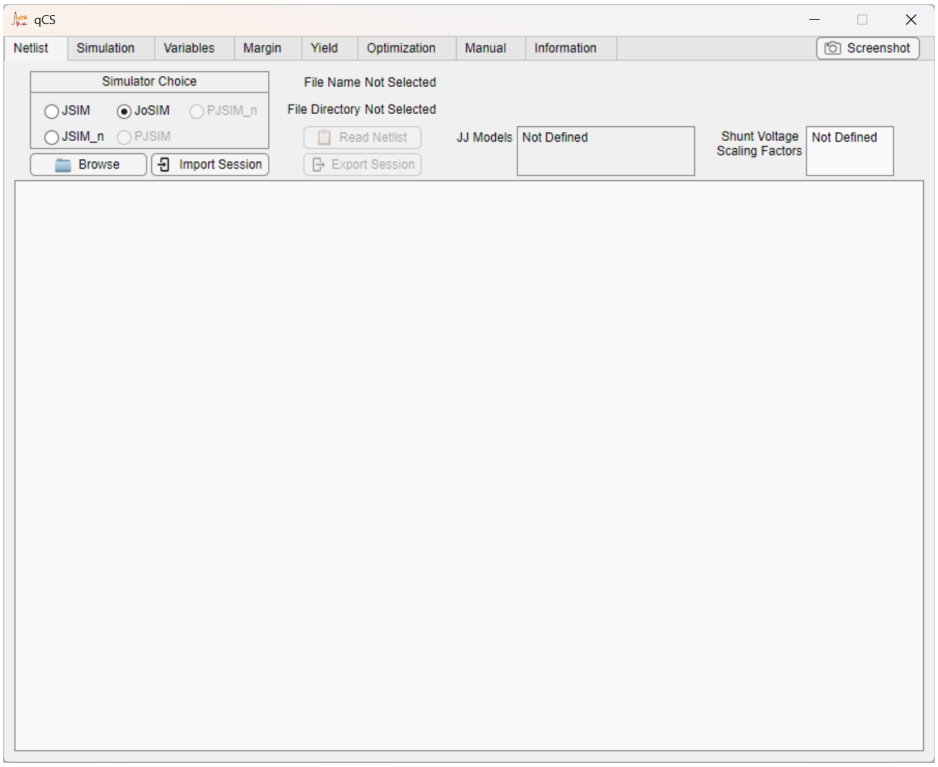


Figure 2: qCS: Netlist tab

- *Simulator Choice*: The options for netlist simulator. The default selection is assigned as JSIM. PJSIM and PJSIM_n options are currently disabled.
- *Browse*: Loads a netlist file.
- *Import Session*: Imports previously saved qCS session.
- *Export Session*: Exports current qCS session.
- *File Information*: Contains file name and its directory information.
- *Netlist Panel*: Shows the content in the netlist file and allows the user to change values on the interface.
- *JJ Models*: Josephson Junction model names defined in the selected netlist.
- *Shunt Voltage Scaling Factors*: The scaling factor is calculated by multiplying two values (junction area and its shunt resistor). For unshunted junctions, create a separate model and assign 0 to its scaling factor.
- *Read Netlist*: Every line of the netlist will be assigned to an object. To activate qCS features, the netlist must be read.

1.7.2 Simulation

Simulations are completed in this tab. qCS reads the output data file and plots the waveforms within the available panel space. The interface is shown in Fig. 3, and each interface component is listed below.

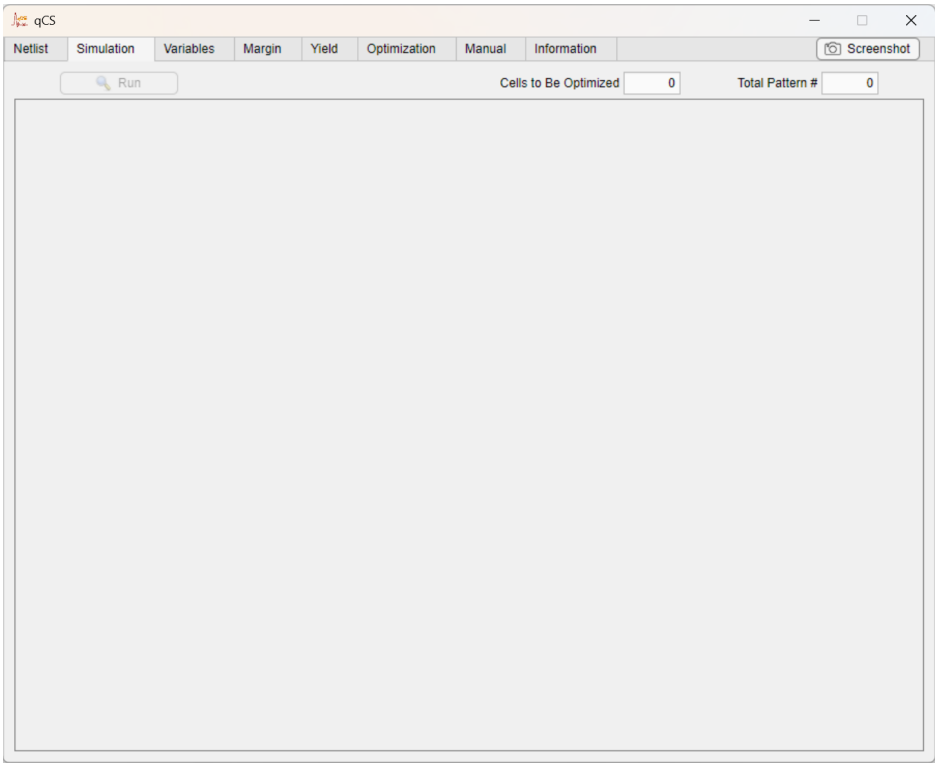


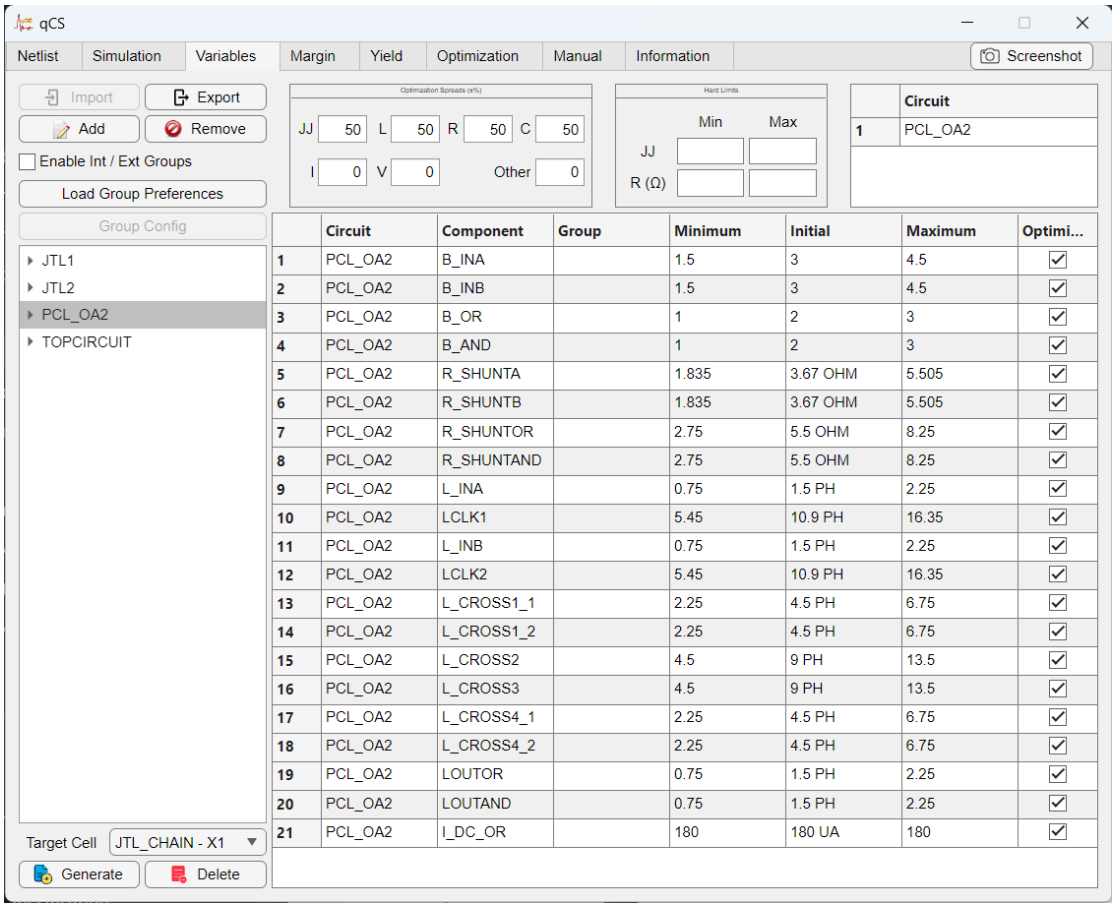
Figure 3: qCS: Simulation tab

- *Run*: Starts the simulation of selected netlist. To verify the compatibility of the selected netlist with the chosen simulator, this step must be performed.
- *Cells to Be Optimized*: This number is determined by the number of ".file" commands in the netlist. Each file command corresponds to a possible cell analysis. The number of independent cell analyses depends on this value.
- *Total Pattern #*: This number is determined by the number of ".print" commands in the netlist.
- *Waveform Panel*: Shows the waveform data obtained from the output file(s).

1.7.3 Variables

After reading the netlist file, it is converted into an object. The object can be edited in this tab, and cell component values are assigned here. This tab supports two layouts, toggled by the *Enable Int / Ext Groups* checkbox: a 7-column legacy layout where a single *Group* column behaves as a mirror-only Ext Group (Fig. 4), and an 8-column layout that exposes separate *Int Group* (Internal Group, for serial JJ chains) and *Ext Group* (External Group, for mirror grouping of everything else) columns (Fig. 5). The two-column rules, the *Group Config* dialog, and the *Load Group Preferences* naming convention are described in detail in §4.1–4.3.

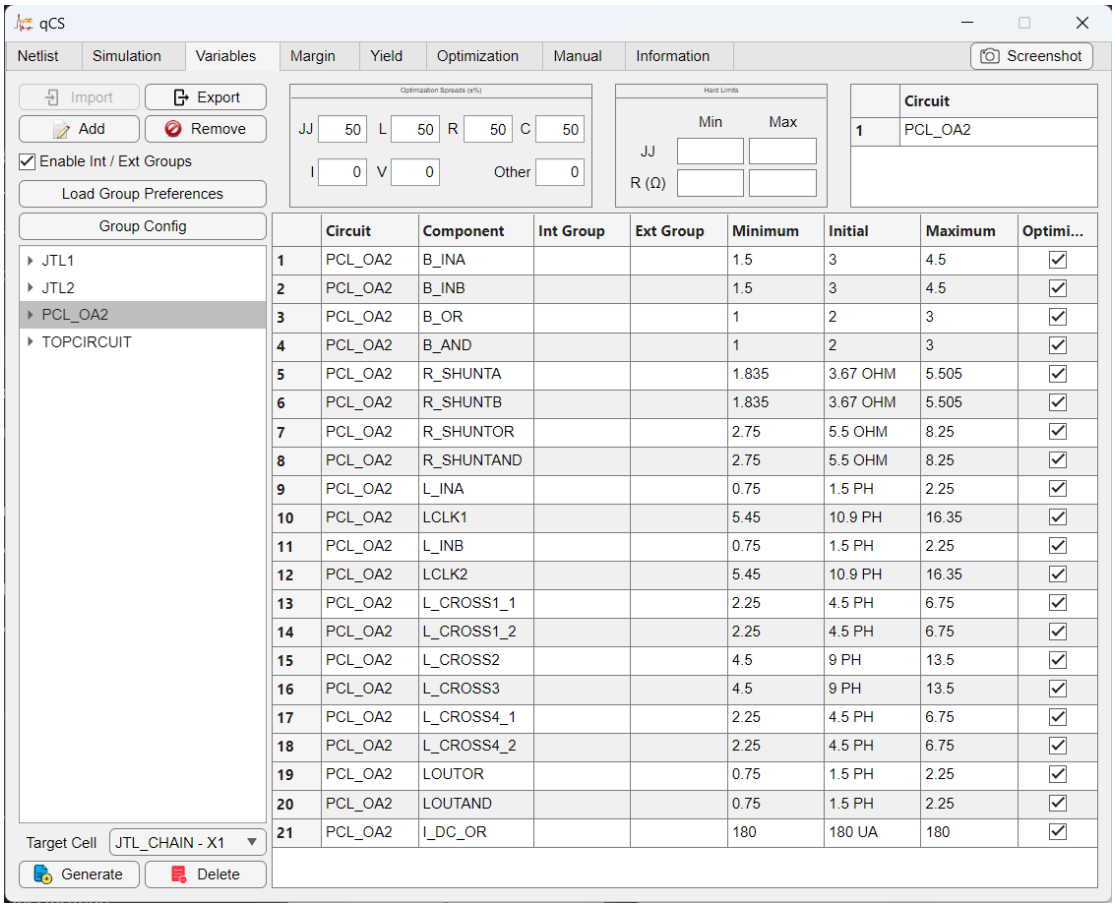
Each component of the interface is listed below.



	Circuit	Component	Group	Minimum	Initial	Maximum	Optimi...
1	PCL_OA2	B_INA		1.5	3	4.5	☑
2	PCL_OA2	B_INB		1.5	3	4.5	☑
3	PCL_OA2	B_OR		1	2	3	☑
4	PCL_OA2	B_AND		1	2	3	☑
5	PCL_OA2	R_SHUNTA		1.835	3.67 OHM	5.505	☑
6	PCL_OA2	R_SHUNTB		1.835	3.67 OHM	5.505	☑
7	PCL_OA2	R_SHUNTOR		2.75	5.5 OHM	8.25	☑
8	PCL_OA2	R_SHUNTAND		2.75	5.5 OHM	8.25	☑
9	PCL_OA2	L_INA		0.75	1.5 PH	2.25	☑
10	PCL_OA2	LCLK1		5.45	10.9 PH	16.35	☑
11	PCL_OA2	L_INB		0.75	1.5 PH	2.25	☑
12	PCL_OA2	LCLK2		5.45	10.9 PH	16.35	☑
13	PCL_OA2	L_CROSS1_1		2.25	4.5 PH	6.75	☑
14	PCL_OA2	L_CROSS1_2		2.25	4.5 PH	6.75	☑
15	PCL_OA2	L_CROSS2		4.5	9 PH	13.5	☑
16	PCL_OA2	L_CROSS3		4.5	9 PH	13.5	☑
17	PCL_OA2	L_CROSS4_1		2.25	4.5 PH	6.75	☑
18	PCL_OA2	L_CROSS4_2		2.25	4.5 PH	6.75	☑
19	PCL_OA2	LOUTOR		0.75	1.5 PH	2.25	☑
20	PCL_OA2	LOUTAND		0.75	1.5 PH	2.25	☑
21	PCL_OA2	I_DC_OR		180	180 UA	180	☑

Figure 4: Variables tab with the checkbox off (7 columns; the single *Group* column behaves as an Ext Group).

- *Import*: Imports previously saved variable setting.
- *Export*: Exports current variable setting.
- *Add*: Adds any selected element into the value table. Selecting a circuit branch will add all its elements.
- *Remove*: Removes any selected element. L-click the mouse and go over the other elements to select multiple elements.
- *Enable Int / Ext Groups*: Toggles between the 7-column legacy layout (a single *Group* column that behaves as an Ext Group — mirror-only, no serial JJ chain semantics) and the 8-column layout with separate *Int Group* (Internal Group, for serial JJ chains) and *Ext Group* (External Group, for mirror grouping of everything else) columns. In legacy mode, same-Group rows are folded into one optimization dimension automatically; no Group Config click is required.
- *Group Config*: Opens a modal dialog for configuring serial JJ chains. Enabled only when *Enable Int / Ext Groups* is on. See Section 4 for details.
- *Load Group Preferences*: Reads the `_int<N>_ext<M>` suffixes already on each component name in the parameter table and auto-fills the Int Group and Ext Group columns from them. Lets the user encode grouping intent directly in the netlist file rather than entering it cell-by-cell. The button switches between 7-column and 8-column layout



	Circuit	Component	Int Group	Ext Group	Minimum	Initial	Maximum	Optimi...
1	PCL_OA2	B_INA			1.5	3	4.5	☑
2	PCL_OA2	B_INB			1.5	3	4.5	☑
3	PCL_OA2	B_OR			1	2	3	☑
4	PCL_OA2	B_AND			1	2	3	☑
5	PCL_OA2	R_SHUNTA			1.835	3.67 OHM	5.505	☑
6	PCL_OA2	R_SHUNTB			1.835	3.67 OHM	5.505	☑
7	PCL_OA2	R_SHUNTOR			2.75	5.5 OHM	8.25	☑
8	PCL_OA2	R_SHUNTAND			2.75	5.5 OHM	8.25	☑
9	PCL_OA2	L_INA			0.75	1.5 PH	2.25	☑
10	PCL_OA2	LCLK1			5.45	10.9 PH	16.35	☑
11	PCL_OA2	L_INB			0.75	1.5 PH	2.25	☑
12	PCL_OA2	LCLK2			5.45	10.9 PH	16.35	☑
13	PCL_OA2	L_CROSS1_1			2.25	4.5 PH	6.75	☑
14	PCL_OA2	L_CROSS1_2			2.25	4.5 PH	6.75	☑
15	PCL_OA2	L_CROSS2			4.5	9 PH	13.5	☑
16	PCL_OA2	L_CROSS3			4.5	9 PH	13.5	☑
17	PCL_OA2	L_CROSS4_1			2.25	4.5 PH	6.75	☑
18	PCL_OA2	L_CROSS4_2			2.25	4.5 PH	6.75	☑
19	PCL_OA2	LOUTOR			0.75	1.5 PH	2.25	☑
20	PCL_OA2	LOUTAND			0.75	1.5 PH	2.25	☑
21	PCL_OA2	I_DC_OR			180	180 UA	180	☑

Figure 5: Variables tab in 8-column mode (*Enable Int / Ext Groups* on). Int Group and Ext Group are separate columns; the *Group Config* button commits chain configurations and the *Load Group Preferences* button auto-fills both group columns from `_int<N>_ext<M>` suffixes already present in the netlist's component names.

automatically based on whether any `_int` suffix is present. Any consistency error (non-JJ component carrying `_int`, an ext label shared across JJ and non-JJ rows, or an internal-L referencing JJs whose suffixes do not match) is reported and the table is rolled back to the exact pre-click state, including any group cells the user had typed manually. See Section 4 for the full suffix specification.

- *Netlist Cell Panel*: A netlist tree is a converted object of the netlist file. The parent nodes represent the cells in the netlist, and child nodes represent the components within the cells. All adjustable elements will be listed in this panel.
- *Optimization Spread Panel*: These spread values define the minimum and maximum limits for the optimization process. They don't affect margin calculation or yield analysis. Ranges for margin analysis are internally defined as -50% and +50% of initial values. Spreads for yield analysis are internally defined as 3σ, which depends on relative STD values.
- *Hard Limit Panel*: Hard limits define the minimum and maximum values for designs in the chip. They create the boundary for the optimization search space by changing the minimum and maximum component values in the columns below. Leaving a column empty means that the optimization limits are defined by the spread percentages. Note that these limits are not considered for margin calculation or yield analysis.

- *Circuit Table*: Depending on the component selection, a corresponding cell number is assigned to each cell. For multiple-cell optimization, select the cells in the same order as ".file" commands in the netlist. Output patterns are internally matched to the cell selections in the same order.
- *Value Table*: Any added netlist element will be listed here. In 8-column mode this is the canonical *Circuit / Component / Int Group / Ext Group / Min / Initial / Max / Optimize* table; in 7-column mode the Ext Group column is hidden.

1.7.4 Margin

This tab contains the margin analysis feature of qCS. The analysis-mode controls are split into two sub-tabs — *Pulse Settings* (pulse-height detection) and *Phase Settings* (phase-jump detection). qCS auto-selects the sub-tab based on what the netlist's `.PRINT` statements report (phase → Phase Settings; voltages or currents → Pulse Settings); the two are not interchangeable on the same netlist. The user can either automatically or manually enter the output pattern and initiate the analysis to evaluate circuit performance by using the Calculate button. The interface is shown in Fig. 6.

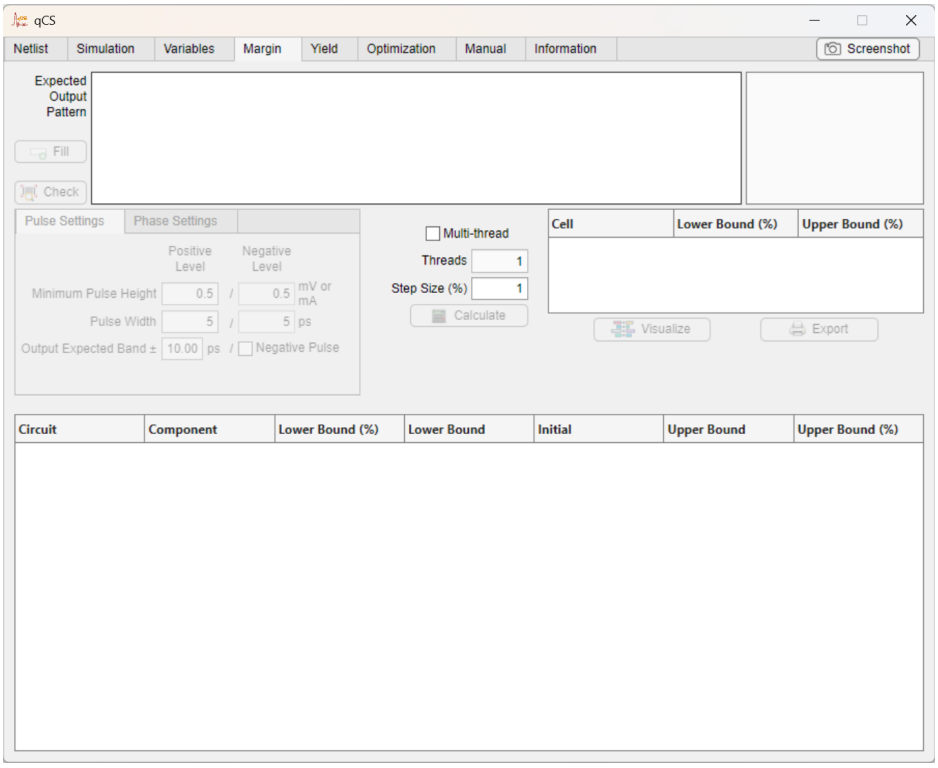


Figure 6: qCS: Margin tab

- *Pulse Settings sub-tab*: Contains *Minimum Pulse Height*, *Pulse Width*, *Output Expected Band* ±, and *Negative Pulses*. Selected automatically when the netlist's `.PRINT` statements report voltages or currents (e.g. `DEVV` / `DEVI`).
- *Phase Settings sub-tab*: Contains *Phase Jump Threshold* (default 5 rad), *Speed Threshold*, *Smoothing Window*, and *Momentum Factor*. Selected automatically when the netlist's `.PRINT` statements report phase quantities (e.g. `PHASE`).

- *Fill*: Automatically fills the expected output pattern area. It uses the parameters from the *currently selected* sub-tab (pulse height/width when Pulse Settings is active; phase jump threshold when Phase Settings is active).
- *Check*: Depending on the active sub-tab, displays the pulses (Pulse Settings) or phase jumps (Phase Settings) that will be considered as logic 1 or 0. This feature is added to enable users to verify the pattern and timings.
- *Expected Output Pattern Panel*: Defines the number of pulses (or phase jumps) that are expected to appear within a certain time interval.
- *Multi-threading*: Allows utilization of more resources.
- *Threads*: Number of assigned threads that define the parallelism.
- *Step Size (%)*: Each parameter value will be scanned by using the defined step size. Recommended: 1 (1%).
- *Calculate*: To enable, select cell components and fill the pattern textbox. This feature performs a margin calculation for all selected cell components independently. The upper and lower limits are internally defined as -50% and +50%.
- *Visualize*: Creates a bar graph of margin results for better visualization.
- *Export*: Saves the tables into an Excel file and other variables into a text file.
- *Cell Table*: Displays the critical margins for each selected cell.
- *Margin Table*: Displays the detailed margin scores for all selected elements. Each physical JJ inside a serial chain occupies its own row, and Ext-Group secondaries are appended directly after their primaries (not at the end of the table).

1.7.5 Yield

Yield analysis can be performed within this tab. The pulse / phase sub-tab structure introduced for the Margin tab also applies here; the settings inside *Pulse Settings* (Positive / Negative Level, Minimum Pulse Height, Pulse Width, Output Expected Band $\pm$, Negative Pulse) and *Phase Settings* mirror their Margin-tab counterparts and are not re-listed below. Randomization is applied only to the components selected on the Variables tab. The interface is shown in Fig. 7, and each interface component is listed below.

- *Expected Output Pattern*: Multi-line text field for the expected pulse / phase-jump pattern. Same time-window-then-relations syntax as on the Margin tab.
- *Fill*: Auto-populates the Expected Output Pattern from the simulation waveforms using the parameters of the active sub-tab.
- *Check*: Overlays the pulses (Pulse Settings) or phase jumps (Phase Settings) that meet the active thresholds on the waveform so the user can verify the pattern and timings.
- *Relative STD (%)* (panel with four separate fields: *JJ*, *L*, *R*, *C*): Per-class relative standard deviation for the Monte-Carlo randomization, expressed as a percentage of each component's initial value. Each selected component is randomized within its 3σ range. Defaults: JJ = 2%, L = R = C = 10%. Every physical component is randomized independently inside the Monte-Carlo loop, regardless of Int Group or Ext Group membership.

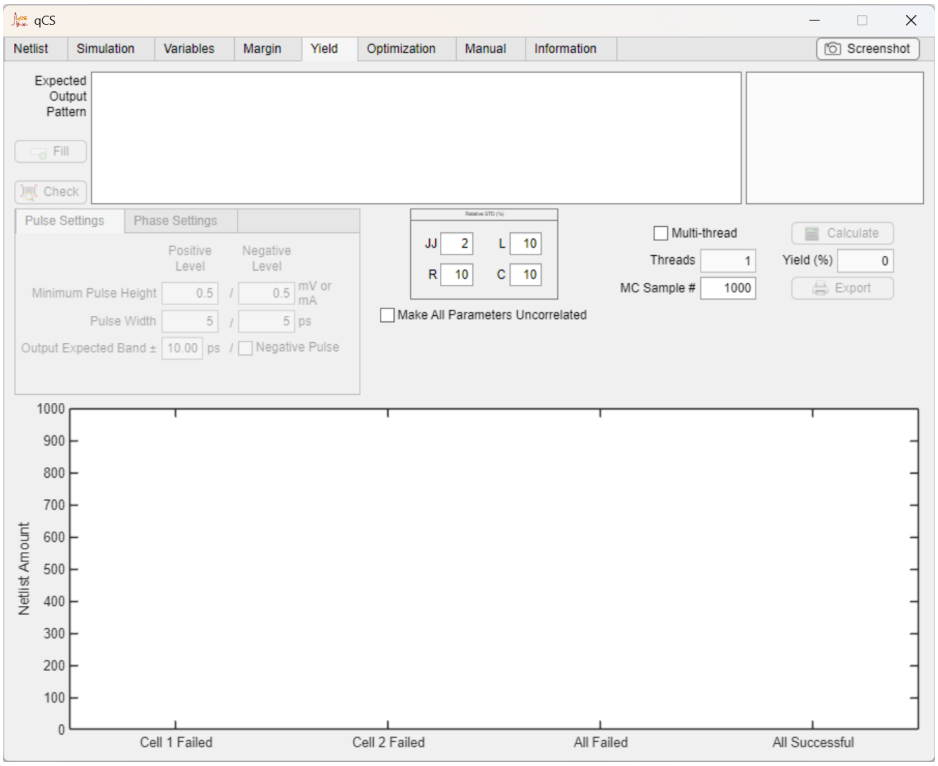


Figure 7: qCS: Yield tab

- *Make All Parameters Uncorrelated*: No-op checkbox; retained for layout compatibility.
- *Multi-thread / Threads*: Enables parallel evaluation of the Monte-Carlo netlists. *Threads* sets the number of workers (active only when *Multi-thread* is ticked).
- *MC Sample #*: Number of randomized netlists generated and compared against the expected output pattern. Default 1000.
- *Calculate*: To enable, select cell components and fill the pattern textbox. Runs the parametric yield analysis.
- *Yield (%)*: Number of working netlists divided by the total number of netlists.
- *Export*: Saves the yield figure and other variables into a text file.
- *Yield bar graph*: Bar chart at the bottom of the tab showing *Cell i Failed* bars (one per selected cell), *All Failed* (samples in which every cell failed simultaneously), and *All Successful*. Because failures across cells can overlap, the All-Failed bar is not the sum of the individual Cell-Failed bars.

1.7.6 Optimization

This tab contains the optimization feature and its settings. The pulse / phase sub-tab structure also applies here. Blue colored parameters don't affect the algorithm, and the others are provided for experienced users. The interface is shown in Fig. 8, and each interface component is listed below.

- *Optimization Rounds*: Number of re-optimization process. In each round, the best value is selected and assigned to a single particle. The other particles are randomized.

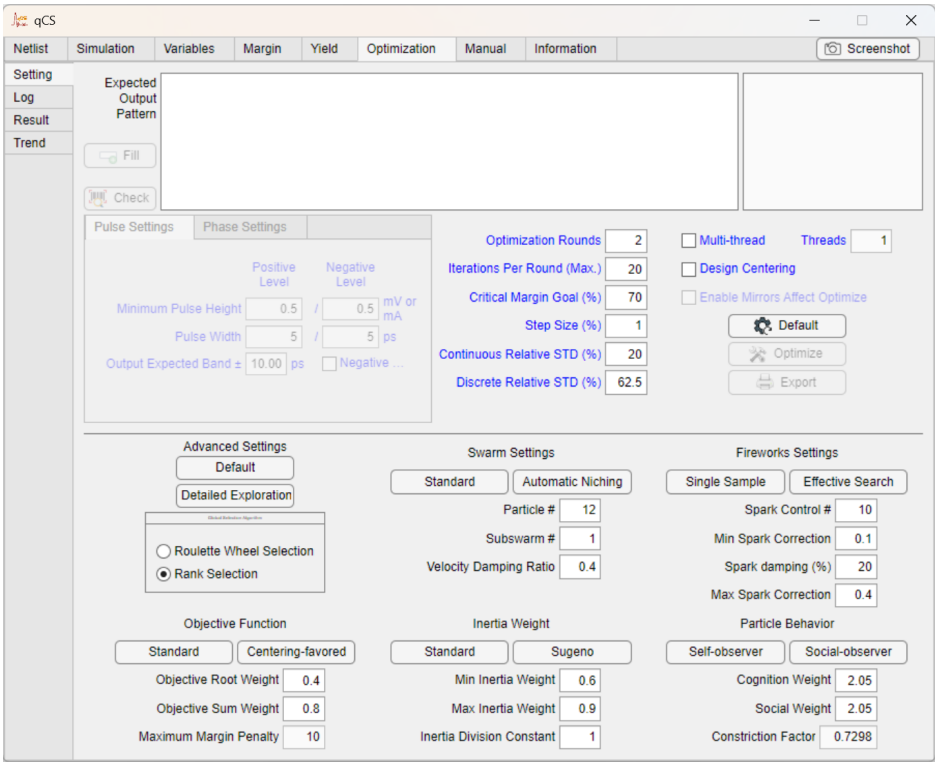


Figure 8: qCS: Optimization tab

- *Iterations Per Round (Max.)*: Terminates the optimization when search iteration limit is reached.
- *Critical Margin Goal (%)*: Stops the optimization after reaching the goal. The goal is to determine the minimum critical margin range for all selected cells. This condition is only compared to the scores of global particles.
- *Step Size (%)*: Each parameter value will be scanned by using the defined step size. Recommended: 1 (1%).
- *Continuous Relative STD (%)*: Sets the relative standard deviation (as a percent of each value) for Gaussian randomization of *continuous* optimization variables — junction area AREA, inductances, resistors, capacitors, and any other real-valued parameter. It shapes the normal distribution used both for probabilistic mutations during optimization and for the PSO initialization of non-first particles. Small values narrow exploration; larger values widen it. 20% works well in typical cases.
- *Discrete Relative STD (%)*: Sets the relative standard deviation for the *discrete* Gaussian distribution used to sample integer-valued parameters — specifically, serial JJ chain length N. The effective sigma is $\max(0.5,\ (\text{Discrete Relative STD}/100) \cdot (N_{\max} - N_{\min}))$, so the spread scales with the chain-length range set on the Variables tab. Default 62.5%.
- *Spark damping (%)*: Scales the magnitude of each Fireworks spark step by $(1 - \text{damping}/100)$ relative to the current parameter value. Higher damping shrinks the per-spark step; lower damping lets sparks make larger moves. Default 20%.
- *Design Centering*: At the end of each round, the best result values are centered relative to their critical margin. qCS generates additional files for the netlist with centered

values at the end of the optimization.

- *Enable Mirrors Affect Optimize*: When ticked, every mirror secondary tied to a primary by Ext Group membership — non-JJ siblings (R / L / C / K rows mirroring a non-JJ primary) and secondary JJ chains that share an Ext Group with another chain — gets its own independent margin scan inside the optimization inner loop, and the resulting margins contribute to the optimization cost. Each secondary is re-scanned every PSO iteration, so the per-iteration cost grows with the number of secondaries and the overall optimization runtime increases accordingly. Untick to scan only primaries during optimization (faster, but the optimizer no longer sees secondary margins). Auto-enabled when any IntGroup or ExtGroup cell is non-empty. Note that the final margin output (Margin tab Calculate, post-optimization summary) always scans every secondary regardless of this checkbox; the gate only takes effect inside the PSO cost loop.
- *Default*: Resets the values of optimization parameters.
- *Optimize*: To enable, select cell components and fill the pattern textbox. This feature performs critical margin optimization.
- *Export*: Saves the tables into an Excel file and other variables into a text file.
- *Global Selection Algorithm Panel*: Roulette Wheel Selection will do a probabilistic approach, but Rank Selection will do a greedy approach and pick the particles with the highest scores. For a small number of particles and sparks, Rank Selection is recommended.
- *Particle #*: Defines the number of particles searching through space. The search space is defined by using the minimum and maximum columns on the Variables Tab.
- *Subswarm #*: Number of global particles that will lead the subswarms.
- *Velocity Damping Ratio*: Affects the lower and upper bounds of the velocity range. Defines the maximum allowable change in a circuit parameter per iteration.
- *Spark Control #*: Controls the number for spark amount of each firework. It will be used with spark correction limits.
- *Min Spark Correction*: Lower Bound for spark amount of each firework. It will be used with a spark control number.
- *Max Spark Correction*: Upper Bound for spark amount of each firework. It will be used with a spark control number.
- *Objective Root Weight*: Defines the weight for the square root of the product of upper and lower bounds on a given margin range.
- *Objective Sum Weight*: Defines the weight for the summation of upper and lower bounds on a given margin range.
- *Maximum Margin Penalty*: Determines the range of difference between the centering-favored and standard critical margin scores for the corner cases. Example: The upper and lower bounds of a netlist are 50% and 0%, respectively. The standard margin score is 50%, but the new score will be (50 - Penalty).
- *Min Inertia Weight*: Defines the effects of the previous velocity on the next iteration and creates a lower bound.

- *Max Inertia Weight*: Defines the effects of the previous velocity on the next iteration and creates an upper bound.
- *Inertia Division Constant*: Affects the velocity by using Sugeno inertia law.
- *Cognition Weight*: Defines how much a particle listens to its own best location.
- *Social Weight*: Defines how much a particle listens to the closest global particle.
- *Constriction Factor*: Downscales the total velocity of a particle. Calculated by using Cognition and Social Weights.

In addition to the optimization parameters, preset values are defined using the buttons listed below.

- *Default*: Assigns recommended values to the optimization algorithm parameters.
- *Detailed Exploration*: Assigns convenient values to the parameters that allow better local and global search with increased runtime.
- *Standard (Swarm Settings)*: A single point convergence setting.
- *Automatic Niching (Swarm Settings)*: Multiple points convergence setting.
- *Single Sample (Fireworks Settings)*: Every particle is randomized and sampled once.
- *Effective Search (Fireworks Settings)*: Utilizes Fireworks Algorithm by assigning more sparks. Increases runtime, but local search becomes better.
- *Standard (Objective Function)*: Eliminates the margin penalty and focuses on increasing the overall margin.
- *Centering-favored (Objective Function)*: Increases the margin penalty to prioritize well-centered results.
- *Standard (Inertia Weight)*: Inertia weight remains the same.
- *Sugeno (Inertia Weight)*: Inertia weight decreases over time to carefully search while converging.
- *Self-observer (Particle Behavior)*: Prioritizes local results.
- *Social-observer (Particle Behavior)*: Prioritizes global results.

1.7.7 Manual

This tab provides the documentation for the simulators (JSIM and JoSIM) and qCS. The interface is shown in Fig. 9.

1.7.8 Information

License and all release information are given in this tab. All notable changes to qCS will also be documented here. The interface is shown in Fig. 10.

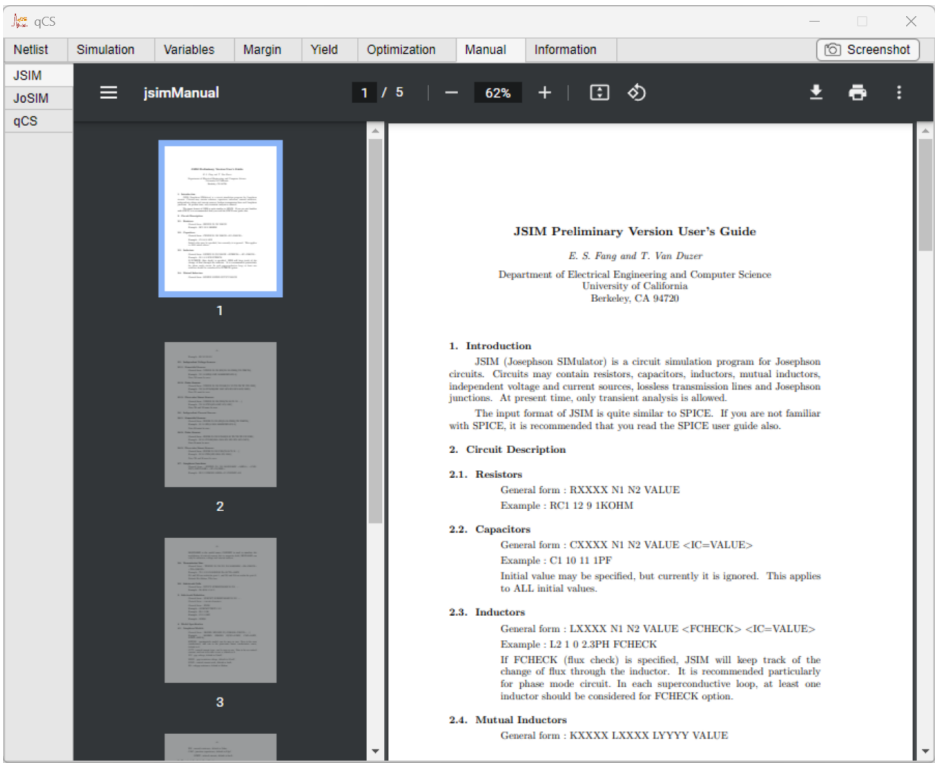


Figure 9: qCS: Manual tab

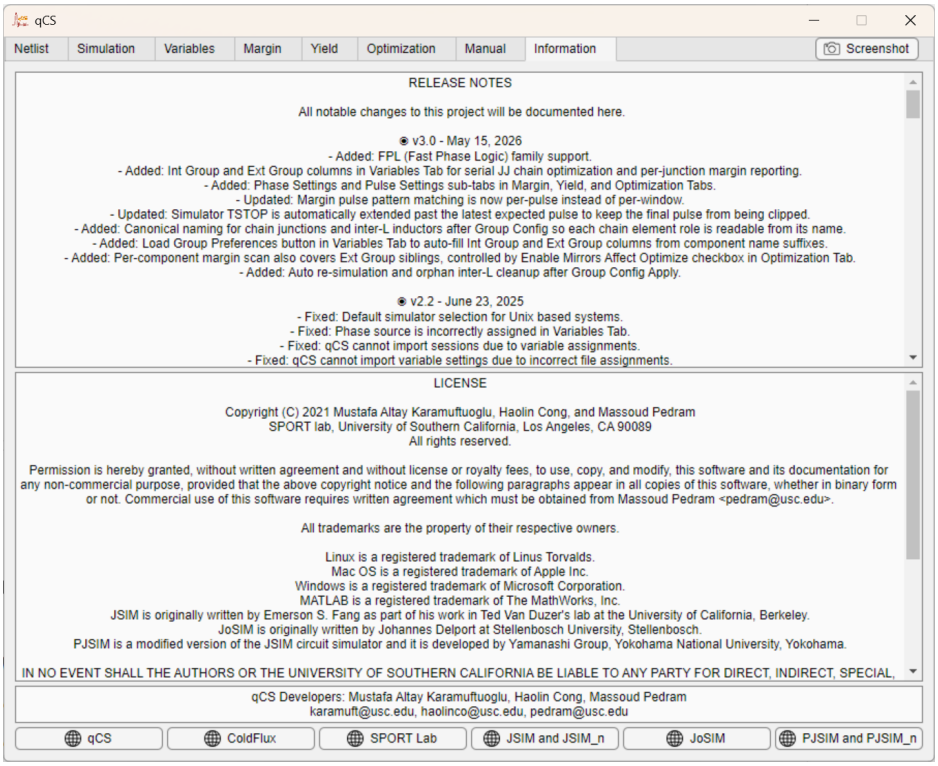


Figure 10: qCS: Information tab

2 Netlist File Selection and Editing

The tool allows only a single file selection. **The analysis mode is dictated by what the netlist prints**: a netlist whose `.PRINT` statements report phase quantities is analyzed in Phase mode (qCS selects the Phase Settings sub-tab on the Margin / Yield / Optimization tabs automatically); a netlist that prints voltages or currents is analyzed in Pulse mode (Pulse Settings sub-tab selected automatically). The two modes are not interchangeable on the same netlist (Sections 5–7). Before loading a netlist, ensure that you select the desired simulator from among JSIM, JSIM_n, and JoSIM. The default simulator selection for qCS is defined as JSIM.

Note: *The logic family (RSFQ / AQFP / RQL / FPL / PCL) is not declared in the netlist; the analysis mode (Pulse vs. Phase) is determined by what the netlist's `.PRINT` statements report*. A netlist that prints phase quantities is run in Phase mode; a netlist that prints voltages or currents is run in Pulse mode. The same physical cell can be analyzed in either mode by writing a netlist that prints the corresponding observables, but a single netlist binds qCS to one mode — the Pulse / Phase sub-tabs on the Margin / Yield / Optimization pages cannot be switched arbitrarily.

The difference between JSIM and JSIM_n is only the noise component, which allows for the addition of local noise. The JSIM manual provided with qCS covers both versions. The selected netlist file will be presented in the text field. JJ model types are acquired from the netlist file, and they will be presented on the same tab. Additionally, shunt voltage scaling factors are automatically assigned for each model. This scaling factor is defined as the multiplication of a junction area and its shunt resistor value.

If there is no shunt resistor, the value will be assigned a value of zero. For different scaling factors, different junction models should be created. The tool internally uses the first found junction and its shunt resistor. Afterwards, it applies the calculated scaling factors to the junction models. After obtaining the scaling factors, the user can make changes to the netlist on the interface, and the Read Netlist button will be enabled to proceed with further configurations. The user must follow the parameter definitions given in the manual of the selected simulator.

Although the simulators have a way of defining the parameters, there are restrictions on the parameter assignments in the netlist. "RB" naming is restricted to use only on shunt resistors. The shunt resistor is placed parallel to its corresponding Josephson junction. If there is a series inductor, its name must start with "LRB". Each shunt resistor and inductor must contain its corresponding junction's name. The supported Josephson junction models are shown in Fig. 11. For example, if a junction is labeled as B01, its shunt resistor and inductor must be RB01 and LRB01, respectively.

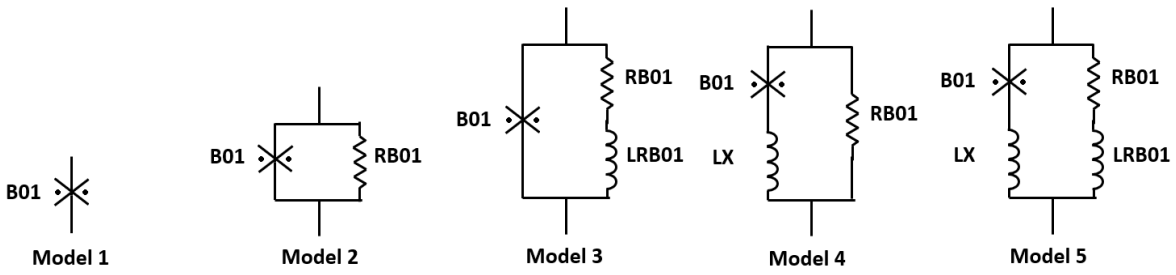


Figure 11: Supported Josephson junction models

Since there are different rules for different simulators, component declarations are important, and they should be adjusted as stated in the following subsections. These declarations

are provided for the qCS input file, and the user must adhere to the restrictions to avoid errors. Otherwise, qCS might fail while reading the netlist, or it might ignore some components even if they are suitable for the selected simulator. While using JoSIM, qCS changes the declarations with the syntax of WRspice mode into JSIM mode syntax. The bold **VALUE**s will be selected by qCS to put into a component table on the interface for the adjustments, and the red colored texts in *Not Allowed* parts are the reason for the errors. One of the common reasons is the direct use of exponential values on component values.

Values in the declarations may use the suffixes given in Table 1. Note that the component values in the general forms must include prefix symbols, and for JoSIM, .PARAM netlist commands must use e-notation. This limitation is due to some MATLAB toolboxes that are not included in MCR; however, this may be addressed in future releases. The assignments in qCS are case-insensitive; thus, it treats uppercase and lowercase letters as the same. Once the netlist file is loaded into qCS, all elements will be internally assigned uppercase letters.

Table 1: Available suffixes in qCS

Prefix Symbol	Prefix Name	E-Notation
F	Femto	1E-15
P	Pico	1E-12
N	Nano	1E-9
U	Micro	1E-6
M	Milli	1E-3
K	Kilo	1E+3
MEG	Mega	1E+6
G	Giga	1E+9

Resistor

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* R* NODE1 NODE2 **VALUE**
- *Example 1:* R01 11 12 1KOHM
- *Example 2:* R02x4 13 14 1000
- *Not Allowed:* R3 15 16 1E+3

qCS also supports [temp=] [neb=] optional parameters for JoSIM. Resistors' names starting with "RB" will be considered as possible shunt resistors. Shunt resistor names must start with "R" followed by the name of a Josephson junction. For example, RB01 will match with a junction named B01, but not with a junction named B01RX. Components identified as shunt resistors will not appear in the component list as they are internally assigned.

Inductor

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* L* NODE1 NODE2 **VALUE**
- *Example 1:* L11 11 12 1PH

- *Example 2:* L2 13 14 0.001U
- *Not Allowed:* L03rX 15 16 1E-12

qCS also supports [] [IC=] optional parameters for JSIM and JSIM_n. The tool will automatically put FCHECK in the declaration when using JSIM and JSIM_n. Inductor names starting with "LRB" will be considered as a possible inductor in series with a shunt resistor. Inductor names must start with "L" followed by the name of a shunt resistor. For example, LRB01 will match with a junction named B01 and a shunt resistor named RB01, but not with a junction named B01TX or a shunt resistor named RB01TX. Components identified as inductors, which are series to shunt resistors, will not appear in the component list as they are internally assigned.

Capacitor

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* C* NODE1 NODE2 **VALUE**
- *Example 1:* C1 11 12 1PF
- *Example 2:* C2L4 13 14 0.001U
- *Not Allowed:* C03 15 16 1E-12

qCS also supports [IC=] optional parameter for JSIM and JSIM_n. Since there is no default assignment for this parameter, the internal assignment will be empty if it is not provided. This applies to all initial values stated as [IC=].

Josephson Junction

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* B* NODE1 NODE2 MODELNAME AREA=**VALUE**
- *Example 1:* B01rxTX 11 12 JJMITLL
- *Example 2:* B2 13 14 JJMITLL AREA=0.9
- *Not Allowed:* B3 15 16 JJMITLL AREA=900M
- *Not Allowed:* B04 17 18 JJMITLL AREA=9E+2M

If there is a third node that is defined for a JJ, qCS will erase the third node of the JJ, and it will keep NODE1 and NODE2. The tool will automatically assign [AREA=1] to JJs with no scaling factor [AREA=]. qCS also supports [IC=] optional parameter for JSIM, JSIM_n, and JoSIM. Additionally, there is support for the [] optional parameter for JSIM and JSIM_n.

Serial JJ chains. Multiple Josephson junctions inside the same subcircuit can be marked as members of a single *Int Group* on the Variables tab; qCS then rewrites those junctions as a canonical serial chain. After the rewrite, the user-visible netlist will contain a single chain primary `_SJJ_A_Gg` (shared area), an integer chain-length parameter `_SJJ_N_Gg` (number of physical junctions in the chain, ranged in $[N_{\min}, N_{\max}]$), and optionally a series of `_SJJ_L_Gg_*` bridging inductors between adjacent junctions. *Example: an FPL cell subcircuit with three originally-named junctions B1, B2, B3 becomes one chain with two optimization variables — shared area A and chain length N.* The chain rewrite is described in detail in Section 4; the netlist syntax for individual junctions is unchanged.

Transmission Line

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* T* NODE1 NODE2 NODE3 NODE4 TD=VALUE Z0=**VALUE**
- *Example 1:* T1 11 12 111 112 TD=100PS Z0=50OHM
- *Example 2:* T022 13 14 113 114 LOSSLESS Z0=50 TD=100P
- *Example 3:* Tx3 15 16 115 116 LOSSLESS TD=100P
- *Not Allowed:* TX4 17 18 117 118 TD=100E-12 Z0=0.05E+3

If there is no assignment for TD and Z0, qCS will assign 50OHM and 1S, respectively. The tool will also automatically put LOSSLESS in the declaration.

Mutual Inductance

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* K* LXX1 LXX2 **VALUE**
- *Example 1:* K011 L01 L02 0.5
- *Example 2:* K2B L03 L04 -0.5
- *Not Allowed:* K3 L05 L06 500E-3

Constraint: when a K row is part of an Ext Group, its Min, Initial, and Max values must all be strictly positive or all strictly negative (no zero, no sign-crossing). This is enforced at edit time by qCS, which auto-reverts the table to the last valid snapshot if the rule is violated. The reason is that an Ext Group secondary K row inherits a `signRatio` of ± 1 from its primary so that the secondary's adjusted value can keep its original sign relative to the primary — this only works if both rows are unambiguously single-signed.

Subcircuits

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* X* SUBCKTNAME NODE1 NODE2 ...
- *Example 1:* Xjtl1 JTL 11 12
- *Example 2:* XSPL2 JTL 13

If SUBCKTNAME is placed at the end of the declaration, which is acceptable for JoSIM, qCS will automatically adjust it in the way shown in the general form above. Moreover, the user must follow the wrapping control syntax (.SUBCKT and .ENDS) stated in the simulator manuals for subcircuit designs. The subcircuit components (X* in the general form) will not be listed in the component table, as there is no value to adjust.

Dependent Sources

There are four different dependent sources.

2.0.1 Current Controlled Current Source

- *Applies to:* JoSIM
- *General Form:* F* NODE1 NODE2 NODE3 NODE4 **VALUE**
- *Example 1:* F01x 11 12 13 14 1
- *Not Allowed:* F2 15 16 17 18 10E-1

2.0.2 Current Controlled Voltage Source

- *Applies to:* JoSIM
- *General Form:* H* NODE1 NODE2 NODE3 NODE4 **VALUE**
- *Example 1:* H1X 11 12 13 14 1
- *Not Allowed:* H022 15 16 17 18 10E-1

2.0.3 Voltage Controlled Current Source

- *Applies to:* JoSIM
- *General Form:* G* NODE1 NODE2 NODE3 NODE4 **VALUE**
- *Example 1:* G1y 11 12 13 14 1
- *Not Allowed:* G22 15 16 17 18 10E-1

2.0.4 Voltage Controlled Voltage Source

- *Applies to:* JoSIM
- *General Form:* E* NODE1 NODE2 NODE3 NODE4 **VALUE**
- *Example 1:* Etx1 11 12 13 14 1
- *Not Allowed:* ETX2 15 16 17 18 10E-1

Independent Sources

There are three different independent sources: voltage, current, and (in JoSIM only) phase. Each source carries one of several source types: piecewise-linear (PWL), pulse, sinusoidal (SIN), custom waveform (CUS), DC, noise, or exponential (EXP). The same source-type definitions can be used for any of the three independent source kinds, with the unit determined by the source kind.

2.0.5 Voltage Source

The first type of source is a Voltage source. Inside the source type definition, the units should be written as Voltage (V).

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* V* NODE1 NODE2 **SOURCE TYPE**

2.0.6 Current Source

The second type of source is the Current source. Inside the source type definition, the units should be written as Ampere (A).

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* I* NODE1 NODE2 **SOURCE TYPE**

2.0.7 Phase Source

The third type of source is the Phase source. Phase source uses radians; inside the source type definition, the units should be written as multiples of π (`*pi`). **Phase sources are particularly relevant for driving RQL and FPL circuits**, where the input signal is conveniently expressed as a phase trajectory rather than a voltage or current pulse train. *Example: an RQL gate input is typically driven by `PPULSE` delivering a phase pulse train, while an FPL chain test bench may use `PPWL` for a piecewise-linear phase ramp.*

- *Applies to:* JoSIM
- *General Form:* P* NODE1 NODE2 **SOURCE TYPE**
- *Example 1 (DC):* P01 11 0 dc 1*pi
- *Example 2 (Pulse):* P02 11 0 pulse(0 2*pi 0p 1p 1p 50p 100p)
- *Example 3 (PWL):* P03 11 0 pwl(0 0 100p 1*pi)

The seven source-type variants of the Phase source — `PDC`, `PPULSE`, `PPWL`, `PEXP`, `PSIN`, `PCUS`, `PNOISE` — mirror the corresponding voltage and current source types described below; substitute the prefix `P` for `V` or `I` and write amplitude tokens as multiples of π.

2.0.8 Source Types

The declarations should be placed in the source type part of the source component, which includes a label and two nodes. Where “Applies to: JoSIM” is shown, the same syntax may be used with phase sources (`P*`) by substituting amplitudes in radians.

2.0.8.1 Piecewise Linear (PWL)

Only the first time point can be selected for the analysis.

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* PWL(0 0 T1 **A1** [T2 A2 ...])
- *Example 1:* pwl(0 0 10p 2.5m)
- *Not Allowed:* PWL(0 0 10ps 2.5E-3)

2.0.8.2 Pulse

The second amplitude can be selected for the analysis. Although some parameters are optional for different simulators, qCS expects the components to be in a general form.

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* PULSE(V1 **V2** TD TR TF PW PER)
- *Example 1:* puLSe(0 1M 0PS 2PS 2PS 10PS 50PS)
- *Not Allowed:* PULSE(0 1E-3 0 2E-12 2E-12 10E-12 50E-12)

2.0.8.3 Sinusoidal (SIN)

The second amplitude can be selected for the analysis.

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* SIN(A0 **A** FREQ TD THETA)
- *Example 1:* SiN(0 1MV 100MEGHZ 0US 0)
- *Not Allowed:* sIN(0 1E-3 100E+6 0U 0)

2.0.8.4 Custom Waveform (CUS)

The scaling factor can be selected for the analysis.

- *Applies to:* JoSIM
- *General Form:* CUS(WAVEFILE TS **SF** IM TD PER)
- *Example 1:* cus(*/home/waveform_file* 1P 2 2 10PS 0)
- *Not Allowed:* CUS(*/home/waveform_file* 1E-12 2 2 10E-12S 0)

2.0.8.5 DC

The amplitude can be selected for the analysis.

- *Applies to:* JoSIM
- *General Form:* DC **A**
- *Example 1:* dc 2.5m
- *Not Allowed:* dC 25E-4

2.0.8.6 Noise

The amplitude can be selected for the analysis.

- *Applies to:* JSIM_n and JoSIM
- *General Form (JSIM_n):* NOISE(0 **A** TSTEP TD)
- *Example 1:* Noise(0 10U 1P 1P)
- *Not Allowed:* NOISE(0 10U 1E-12 1E-12)
- *General Form (JoSIM):* NOISE(**A** TD TSTEP)
- *Example 1:* Noise(10U 1PS 1P)
- *Not Allowed:* NOISE(10E-6 1E-12S 1E-12)

2.0.8.7 Exponential (EXP)

The second amplitude can be selected for the analysis.

- *Applies to:* JoSIM
- *General Form:* EXP(A1 **A2** TD1 TAO1 TD2 TAO2)
- *Example 1:* EXP(1 5 0S 2PS 1PS 1PS)
- *Not Allowed:* exp(1 5 0S 2E-12S 1E-12S 1E-12S)

Model

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* .MODEL MODELNAME JJ([PARAMETER=VALUE])
- *Example 1:* .model JJMODEL1 JJ(RTYPE=0, ICRIT=250uA, CAP=1.32PF)
- *Example 2:* .modEL jjmod1 JJ(RTYPE=1, VG=2.6mV, ICRIT=0.1mA)

qCS assigns a scaling factor for each defined model for JJs. Therefore, the user must ensure that multiple models are defined if they have both shunted and unshunted JJs, and assign them to the corresponding JJs. For other optional parameters of the model command, the user must check the simulator manuals.

Transient Analysis

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form:* .TRAN TSTEP TSTOP PSTART PSTEP
- *Example 1:* .TRAN 1PS 100PS 20PS 1PS
- *Example 2:* .tran 0.2P 2.2N 0 0.2P
- *Not Allowed:* .TRAN 0.2E-12S 2.2E-9S 0 0.2E-12

It should be noted that PSTART and PSTEP parameters are optional for all simulators. However, qCS uses the default values of JSIM for these parameters. The default values for PSTART and PSTEP are 0S and 1PS, respectively. Therefore, when assigning a value to these parameters using JoSIM, the user must ensure that PSTEP is equal to or larger than TSTEP, as stated in the JoSIM manual. Moreover, it is the user's responsibility to determine the transient values required for correct operation. Simulators might give an integration error, and qCS will consider this case as an incorrect operation. Note that if any error is detected by qCS, it might interrupt the process and terminate the analysis.

During margin / yield / optimize, qCS may automatically extend TSTOP so the simulation always covers the latest expected pattern window plus half that window's width on each signal. The user's original (pre-extension) TSTOP is preserved internally and used by the overrun check, so a late filler pulse appearing inside the extension will not be counted as an overrun violation. Simulators see the extended window; the user's intended simulation window remains the source of truth for correctness.

Option Specifications

- *Applies to:* JSIM and JSIM_n
- *General Form:* .OPTIONS [PARAMETER=VALUE]
- *Example 1:* .options RELTOL=0.01 MAXPHISTEP=1.5
- *Example 2:* .OPTIONS NUMDGT=3

If there is no option command in the netlist, qCS automatically assigns default values to the option values while using JSIM and JSIM_n. See the simulator document for the details of this command.

Parameters

- *Applies to:* JoSIM
- *General Form:* .PARAM VARNAME=EXPRESSION
- *Example 1:* .PARAM SCALING=1.0
- *Example 2:* .param L01=1e-3*(1/SCALING-(1-SCALING))
- *Example 3:* .PARAM L02=2u*L01

As mentioned at the beginning of this section, even though exponential expressions are not allowed for direct component value assignments in qCS while using JoSIM, the user can adjust or assign values by using the ".PARAM" command. qCS will calculate these parameters and assign them to the related component values. After assigning the values, ".PARAM" commands will be removed from the imported netlist. To make sure that there is no error during the calculation, do not put any whitespace. Whitespaces are allowed, but not every corner case is tested.

Include

- *Applies to:* JoSIM
- *General Form:* .INCLUDE RELATIVE_PATH_TO_FILE
- *Example 1:* .include /home/ABC/example_CENTOS.cir
- *Example 2:* .INCLUDE C:\Users\ABC\Downloads\example_WINDOWS.cir

The examples 1 and 2 given above are for CentOS 7 and Windows 10 operating systems, respectively. Ensure that the file path is correct and avoid using whitespaces in the directory name. Upon accessing the file, qCS will merge the data with the imported netlist, as the tool only allows access to one file at a time.

Output

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form 1:* .PRINT PRINTTYPE DEVICE
- *General Form 2:* .PRINT PRINTTYPE NODE
- *Example 1:* .print DEVI R01
- *Example 2:* .PRINT NODEV 11 22

JoSIM offers three distinct commands for generating output data. Among ".PRINT", ".PLOT", and ".SAVE", use the ".PRINT" command to keep the consistency. qCS still converts every ".PLOT" and ".SAVE" commands into ".PRINT". Furthermore, the tool will divide the multiple devices stated after ".PRINT". It should be noted that the separator between the subcircuit name and its components for the ".PRINT" command is a period or a vertical bar for JoSIM, while JSIM and JSIM_n use an underscore for the same purpose. By following this idea, the same operation can be performed with the following examples 3 and 4 for JSIM/JSIM_n and JoSIM, respectively.

- *Example 3 (JSIM/JSIM_n):* .PRINT DEVI X01_L01
- *Example 4 (JoSIM):* .print DEVI L01|X01

These printing commands can be placed into a control block, which has a control syntax of ".CONTROL" and ".ENDC". The devices or nodes within the control block do not require any printing command. However, qCS gets the control block and puts all devices and nodes into the individual ".PRINT" command. Each of these printing commands should be placed after the ".FILE" command, which stores the data in the corresponding file. The ".FILE" command defines the number of cells to be optimized. For multiple-cell analysis, the user must provide multiple ".FILE" commands, each with a unique file name. Each selected cell should be in the same order as ".FILE" commands, and the data within the files will be assigned to the corresponding selected cell.

- *Applies to:* JSIM, JSIM_n and JoSIM
- *General Form 3:* .FILE FILENAME
- *Example 5:* .file FILENAME1.DAT
- *Ex. 5 Data:* .PRINT DEVI L01|OR2
- *Example 6:* .FILE FILENAME2.DAT
- *Ex. 6 Data:* .PRINT DEVI L01|XOR2

For instance, if we use examples 5 and 6 within the same netlist for the optimization of OR and XOR logic gates, data provided in example 5 will be considered for the analysis of the OR gate, and example 6 will be considered for the analysis of the XOR gate. Therefore, the user must match the file order when performing multiple-cell optimization. On the other hand, if there is no file command in the netlist, qCS will automatically create one for the user.

Unused Netlist Commands

Global thermal noise temperature and bandwidth commands are disabled to keep the analysis accurate. Therefore, additional ".TEMP" and ".NEB" commands in JoSIM are not supported on qCS. Moreover, since parameter spread is applied on the selected parameters for the analysis, the global spread command ".SPREAD" in JoSIM is not supported on qCS. The purpose of the IV curve for a specified JJ model does not fit the concept of qCS. Thus, it is also not supported on qCS.

3 Simulation

To optimize the circuit parameters, the netlist simulation must be completed without an error. The user must run the selected netlist on this tab to enable margin calculation, yield calculation, and optimization features. Depending on the number of parameters to be plotted, the interface will adjust its result windows. Additionally, this Simulation tab helps the user to identify the pattern and use the correct desired timing.

Upon successfully running the simulation, there will be checkboxes for observing the data in the analysis. This feature enables the user to run the simulation for comparing multiple nodes, and it is possible to exclude plotting data for analysis by setting the checkbox value to zero. The selected plotted data should contain the signal that the analysis mode will interpret: *pulses* (voltage spikes) for Pulse-mode analysis, or *phase trajectories* for Phase-mode analysis. Selecting the wrong type of node for the chosen mode will produce incorrect margin or yield results.

For JSIM and JSIM_n, only one node at a time is allowed while using the ".PRINT" command. However, JoSIM allows multiple observabilities on the same command. When loading the netlist, if JoSIM is selected as the simulator, the ".SAVE" and ".PLOT" commands will be changed to ".PRINT" commands. Additionally, multiple nodes on a single ".PRINT" command will be divided into unique lines, providing qCS with more flexibility.

For writing the output data into a file, the ".FILE" command is used. Each ".FILE" command corresponds to a different cell optimization. It means that if the user wants to independently analyze two cells at a time, they need to define two ".FILE" commands in the netlist. After this command, the related data should be written into the output file by using the corresponding ".PRINT" command. So, each ".PRINT" command is assigned to a parent ".FILE" command that occurs first on top of the ".PRINT" command. The number of cells to be optimized will be shown on the interface together with the number of observation points. The user can zoom to the plotted data, and qCS allows them to save the plotted data.

JSIM has an undocumented limit for printing data to output files. This limit is observed as 19. Therefore, the simulation window limit is set to 19 plots for the simulators. Additionally, since the resize function is not yet activated, it is recommended that the number of ".PRINT" commands be limited to a maximum of 10 for the current window size.

4 Variable Parameter Specification

The Variables tab is where the user selects the netlist components that will be subject to margin, yield, and optimization analyses, and where the search space and grouping for the optimizer is described. A single component selection or a single complete sub-circuit selection is allowed on this panel. After selecting the netlist component tree, related components will be added to the parameter table. The parameter table can be displayed in either a 7-column legacy layout or an 8-column layout that exposes separate Int Group and Ext Group columns. The Int Group column expresses internal serial-JJ-chain membership, and the Ext Group column expresses external mirror grouping among same-type siblings (B / L / R / C / K). Each row also has minimum, initial, and maximum value columns plus an Optimize toggle.

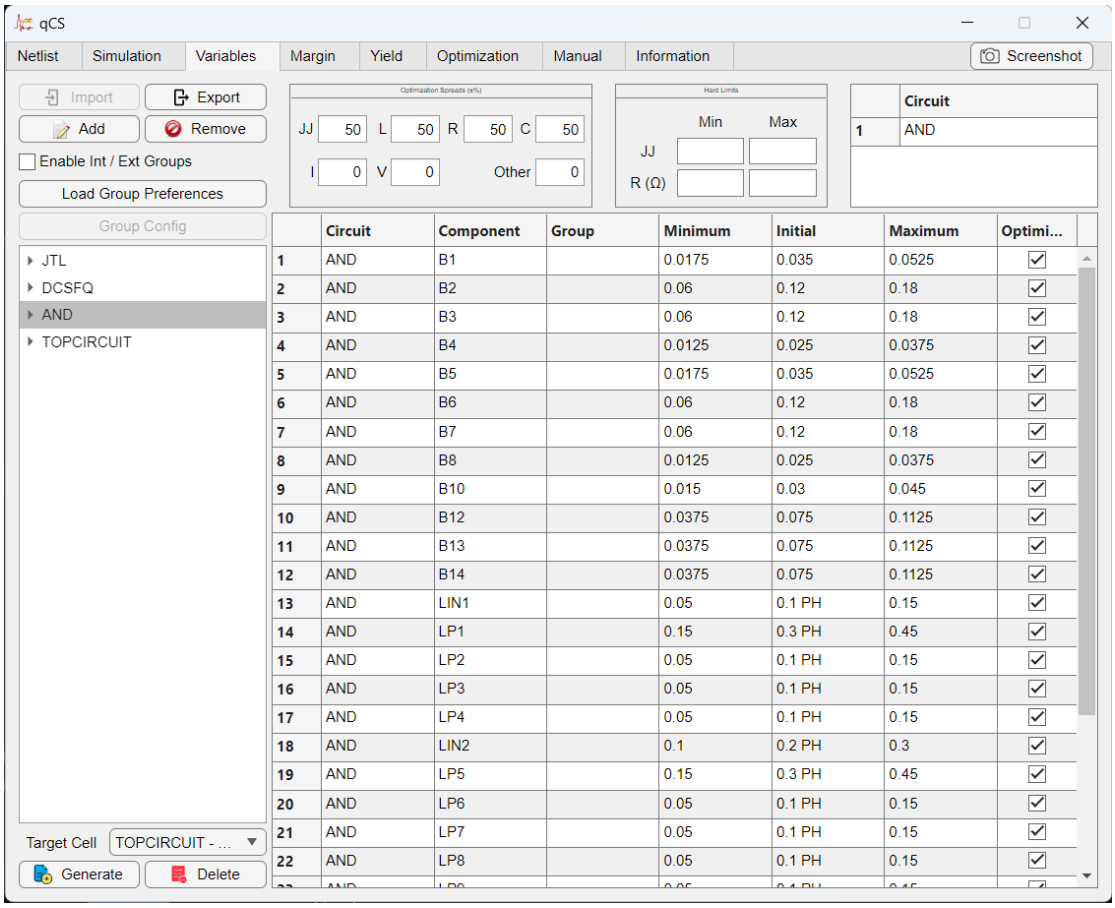
The minimum and maximum limits are used in the optimization process and are initially defined as −50% and +50% of the default value. These columns do not affect the margin calculation or yield analysis, but they define the search space for the optimization. The maximum number of cells to be optimized will be set as the number of ".FILE" lines in the netlist. Each circuit that requires individual optimization should have its own ".FILE" line and ".PRINT" lines for its components. If the user exceeds the assigned number of cells, the additional cells will be designated as global cells, and their components will be treated as global components. It means that these global components will affect both cells simultaneously, and the cells are no longer independent. The components assigned as global can be grouped with other cell components, even though this is not typically allowed. This feature gives more flexibility to the tool user.

4.1 Int Group and Ext Group columns

The *Enable Int / Ext Groups* checkbox toggles between two layouts of the parameter table:

- **7-column legacy layout (Int/Ext disabled)**: The visible *Group* column behaves as an Ext Group (mirror grouping only); the Int Group is implicitly empty, so no serial JJ chain can be formed in this mode. Same-Group rows are folded into a single optimization dimension at run time, and the *Group Config* button is disabled. This compact layout is well-suited for single-junction RSFQ designs (Fig. 12).
- **8-column Int/Ext layout (Int/Ext enabled)**: Both *Int Group* and *Ext Group* columns are editable. Int Group is restricted to JJ rows (any non-JJ entry is rejected at edit time); Ext Group accepts siblings of any single type (B / L / R / C / K) that should mirror a primary value, so a B-only Ext Group bucket is legal alongside the more common R / L / C / K mirror groups. The *Group Config* button becomes enabled in this mode. Each time the user changes any Int Group or Ext Group cell, *Group Config* must be re-opened and confirmed before margin, yield, or optimize can run; the tool blocks the run otherwise and prompts the user to apply the updated grouping first (Fig. 13).

Worked example (Fast Phase Logic cell): consider an FPL subcircuit with three Josephson junctions B1, B2, B3 that should be rewritten as a serial chain whose length the optimizer can vary, plus two symmetric shunt resistors RB1, RB2 that must take identical values during optimization. After ticking *Enable Int / Ext Groups*, the user assigns Int Group = 1 to B1, B2, and B3, and Ext Group = 2 to RB1 and RB2. qCS now understands that the three junctions form one chain primary (a candidate for serial-chain rewrite), and that the two resistors form one mirror group (RB1 is the primary, RB2 follows RB1's value during optimization, yield, and margin). The same row can carry both an Int Group (for chain membership) and an Ext Group



	Circuit	Component	Group	Minimum	Initial	Maximum	Optimi...
1	AND	B1		0.0175	0.035	0.0525	☑
2	AND	B2		0.06	0.12	0.18	☑
3	AND	B3		0.06	0.12	0.18	☑
4	AND	B4		0.0125	0.025	0.0375	☑
5	AND	B5		0.0175	0.035	0.0525	☑
6	AND	B6		0.06	0.12	0.18	☑
7	AND	B7		0.06	0.12	0.18	☑
8	AND	B8		0.0125	0.025	0.0375	☑
9	AND	B10		0.015	0.03	0.045	☑
10	AND	B12		0.0375	0.075	0.1125	☑
11	AND	B13		0.0375	0.075	0.1125	☑
12	AND	B14		0.0375	0.075	0.1125	☑
13	AND	LIN1		0.05	0.1 PH	0.15	☑
14	AND	LP1		0.15	0.3 PH	0.45	☑
15	AND	LP2		0.05	0.1 PH	0.15	☑
16	AND	LP3		0.05	0.1 PH	0.15	☑
17	AND	LP4		0.05	0.1 PH	0.15	☑
18	AND	LIN2		0.1	0.2 PH	0.3	☑
19	AND	LP5		0.15	0.3 PH	0.45	☑
20	AND	LP6		0.05	0.1 PH	0.15	☑
21	AND	LP7		0.05	0.1 PH	0.15	☑
22	AND	LP8		0.05	0.1 PH	0.15	☑

Figure 12: Parameter table in the 7-column legacy layout (*Enable Int / Ext Groups* off). The single *Group* column behaves as an Ext Group; no Int Group is exposed, so no serial JJ chain can be configured in this mode.

(for ext-mirror membership), but the rules above on which rows are eligible for each column still apply.

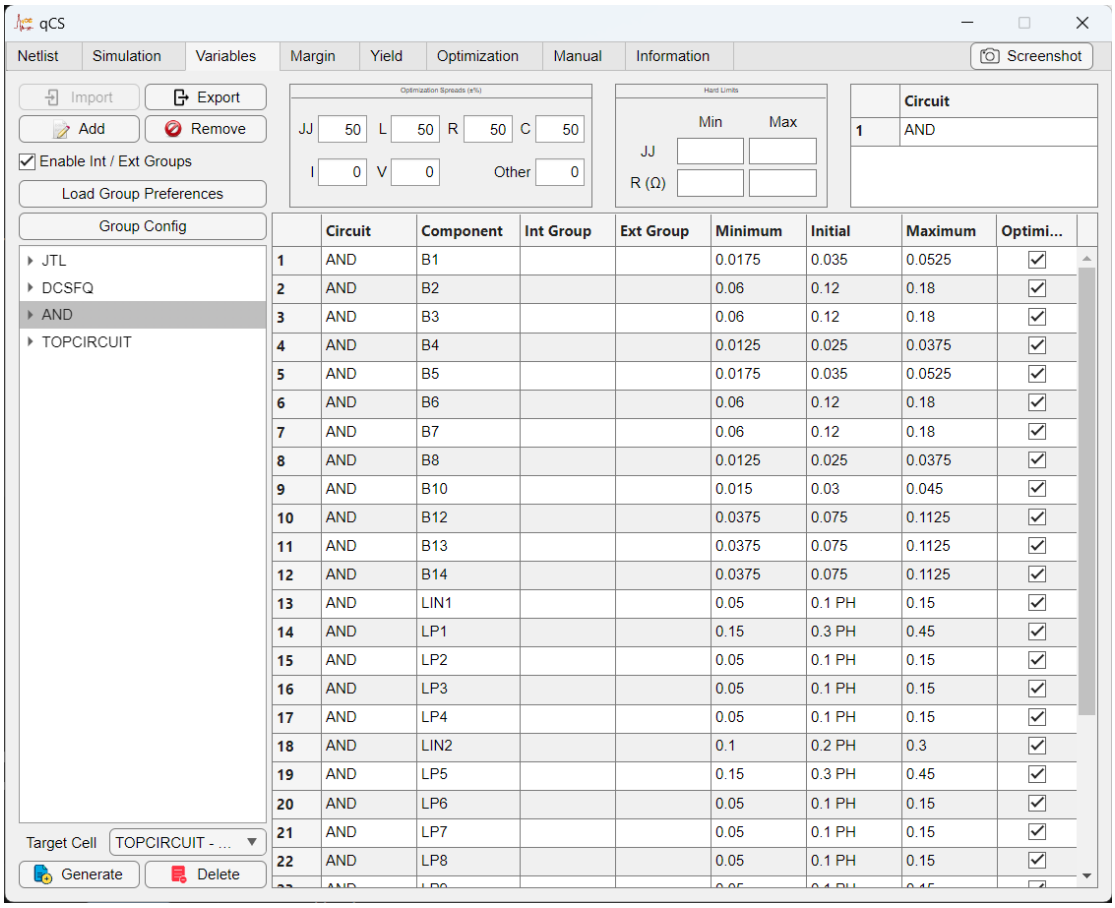
4.2 Group Config dialog and Serial JJ chain rewrite

Once Int Groups have been assigned, click the Group Config button to open the Group Config dialog. The dialog enumerates every Int Group bucket (chains) and every Ext-only bucket (rows whose Int Group is blank but whose Ext Group is set). For each chain bucket, the user provides:

- N_{min} and N_{max}: the minimum and maximum number of physical junctions allowed in the chain. The optimizer will sample chain length N in this discrete integer range.
- *Insert inter-L*: when ticked, qCS automatically adds bridging inductors between every pair of adjacent junctions in the chain. The bridging inductors' values scale with N.

Ext-only buckets (those rows where Int Group is blank but Ext Group is set) appear as greyed gear-less rows in the dialog — they have no chain length to configure; the dialog merely registers them as a confirmed mirror group.

Worked example continued (FPL cell): for the Int Group 1 bucket holding B1, B2, B3, the user sets $N_{min} = 2$, $N_{max} = 6$, and ticks *Insert inter-L*. The bucket for Ext Group 2 (RB1, RB2) is



	Circuit	Component	Int Group	Ext Group	Minimum	Initial	Maximum	Optimi...
1	AND	B1			0.0175	0.035	0.0525	☑
2	AND	B2			0.06	0.12	0.18	☑
3	AND	B3			0.06	0.12	0.18	☑
4	AND	B4			0.0125	0.025	0.0375	☑
5	AND	B5			0.0175	0.035	0.0525	☑
6	AND	B6			0.06	0.12	0.18	☑
7	AND	B7			0.06	0.12	0.18	☑
8	AND	B8			0.0125	0.025	0.0375	☑
9	AND	B10			0.015	0.03	0.045	☑
10	AND	B12			0.0375	0.075	0.1125	☑
11	AND	B13			0.0375	0.075	0.1125	☑
12	AND	B14			0.0375	0.075	0.1125	☑
13	AND	LIN1			0.05	0.1 PH	0.15	☑
14	AND	LP1			0.15	0.3 PH	0.45	☑
15	AND	LP2			0.05	0.1 PH	0.15	☑
16	AND	LP3			0.05	0.1 PH	0.15	☑
17	AND	LP4			0.05	0.1 PH	0.15	☑
18	AND	LIN2			0.1	0.2 PH	0.3	☑
19	AND	LP5			0.15	0.3 PH	0.45	☑
20	AND	LP6			0.05	0.1 PH	0.15	☑
21	AND	LP7			0.05	0.1 PH	0.15	☑
22	AND	LP8			0.05	0.1 PH	0.15	☑

Figure 13: Parameter table in the 8-column Int / Ext layout (*Enable Int / Ext Groups* on). Int Group and Ext Group are separate columns, and the *Group Config* button is enabled so chain configurations can be committed.

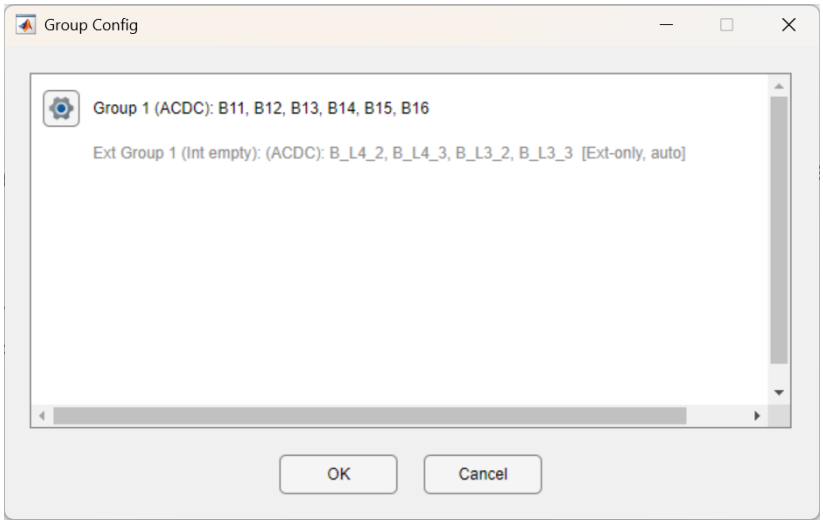


Figure 14: Group Config dialog: chain bucket plus ext-only mirror bucket.

shown as greyed and gear-less. After clicking OK, qCS performs the chain rewrite (described next). The user does not need to edit the netlist text by hand to switch the cell between 2 junctions and 6 junctions — the optimizer explores the integer range during the optimization

run.

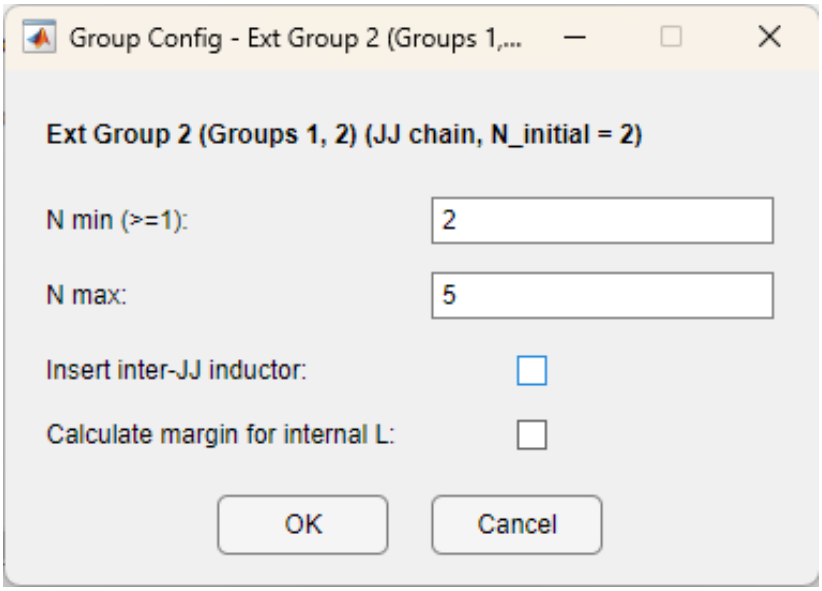


Figure 15: Expanded JJ chain bucket inside the Group Config dialog: N_{min} / N_{max} fields and the *Insert inter-L* checkbox populated for the worked example.

4.2.0.1 What happens when the user clicks OK.

The *Group Config* commit performs an authoritative **netlist mutation**: for every JJ-only Int Group, qCS deletes the user's original junctions from the netlist and re-inserts them with canonical, self-describing names. The new convention is:

- *Junctions:* `B<k>_int<I>_ext<E>` where `<k>` is the 1-based position in the chain, `<I>` is the Int Group label, and `<E>` is the Ext Group label (or `B<k>_int<I>` when no Ext Group is set, or `B<k>_ext<g>` in the 7-column legacy mode).
- *Inter-L bridging inductors* (only when *Insert inter-L* is ticked): `L<k>_B<n>_B<n+1>_int<I>_ext<E>`. The embedded `_B<n>_B<n+1>` part identifies the two junctions the inductor sits between, so an inter-L's role is readable directly from its name. Inter-L values are automatically scaled to keep electrical behaviour consistent across chain lengths.

When the user runs Margin / Yield / Optimize on a netlist that contains a Serial JJ chain, qCS treats the whole chain as just two tunable quantities: a single shared junction area that is applied identically to every junction in the chain, and an integer chain length that is allowed to vary in the $[N_{min}, N_{max}]$ range specified earlier in Group Config. Per-junction areas are not exposed — that would defeat the point of declaring a serial chain, in which all junctions are assumed to be physically identical copies. The optimizer accordingly explores just one area and one length per chain rather than one independent area per junction. *What this means for the result tables.* The Margin table still lists every physical junction individually (`B1_int<I>_ext<E>`, `B2_int<I>_ext<E>`, ...) so the user can see which junction in the chain is the weakest. The shared-area constraint means every row in that group shows the same Initial / Final area value, but each row's per-junction margin is computed and reported separately. The netlist that qCS writes out for simulation always contains the full

set of B<k>_int<I>_ext<E> junctions; the chain length value chosen by the optimizer only determines how many junctions get instantiated.

After the rewrite, the parameter table is refreshed: rows for the original junctions disappear, replaced by rows carrying the new canonical names. Existing Min / Initial / Max values are preserved by cloning a row template before mutation. qCS also runs a lightweight re-simulation immediately after the rewrite so that downstream tabs see fresh waveform data, and as part of the same Apply step it cleans up any inter-L inductors left behind by a previous Group Config snapshot (so toggling *Insert inter-L* between configurations never leaves orphan bridging inductors in the netlist). If the rewrite is byte-identical to the previous netlist (the user opened the dialog and clicked OK without making changes), the simulation is skipped.

4.3 Load Group Preferences

The Load Group Preferences button on the Variables tab is the inverse of *Group Config*: instead of writing canonical names back into the netlist, it reads the names that are already there and reverse-derives the Int Group / Ext Group columns from them. This lets the user encode grouping intent directly in the netlist file (e.g. when hand-authoring a cell or when importing a design that was prepared outside qCS) and have qCS pick it up automatically.

The button parses the trailing portion of each Component name against three patterns (case-insensitive):

- <stem>_int<N>_ext<M> — both Int Group and Ext Group are filled.
- <stem>_int<N> — only Int Group is filled. Legal only on JJ rows (component name starts with B); other types trigger an error.
- <stem>_ext<M> — only Ext Group is filled.
- *No suffix:* both Int Group and Ext Group are left blank.

Names that match the canonical inter-L pattern L<k>_B<n>_B<n+1>_... are detected up front and skipped from suffix parsing. This keeps the rule "L cannot belong to an Int Group" intact even though an inter-L's full canonical name does carry an _int<I> fragment.

4.3.0.1 Internal-L / JJ consistency check.

When an inter-L name with a _int<I>_ext<E> (or _int<I>, or _ext<E>) suffix is found and the corresponding B<n> / B<n+1> junction rows are also present in the parameter table, those JJ rows must carry the same suffix the inter-L claims. Mismatches (e.g. an inter-L L1_B1_B2_int1_ext1 sitting next to plain B1 / B2 JJ rows that were never given the matching suffix) are reported as an error and the Load operation is aborted.

4.3.0.2 Failure preserves user edits.

The button snapshots the table before any changes are written. If parsing succeeds and validation passes, the parsed Int / Ext Group numbers are committed and the layout switches between 7-column and 8-column mode automatically (any _int present selects 8-column; _ext-only stays in 7-column). If anything fails along the way — a non-B component carries _int, an Ext label is shared between JJ and non-JJ rows, an Ext-only bucket mixes types, or the inter-L / JJ consistency check trips — the table is rolled back to the exact pre-click state, including any Int / Ext Group cells the user had typed manually before clicking. After the table is restored, the user is shown the specific error and the netlist may need to be edited (or the offending row removed from the table) before re-clicking the button.

4.4 Min / Initial / Max value rules

Min, Initial, and Max columns describe the search range for each row. The general rule is Min < Max with Initial somewhere inside that interval. Two refinements apply:

- **Group lock**: when a row is locked to a primary by Ext Group membership, its Min / Initial / Max cells are greyed out — the secondary always inherits the primary's values. A K-typed secondary additionally inherits a *signRatio* of ± 1 derived from the relative sign of its Initial vs the primary's Initial; this ratio is multiplied into the primary's adjusted value before the secondary is written so that K secondaries can have an opposite sign to the primary while still tracking magnitude.
- **K-row sign rule**: every K row's Min, Initial, and Max must be *strictly* same-sign (all positive or all negative). Zero is not allowed, and sign-crossing is not allowed. If the user makes an edit that breaks this rule, the edit is rejected and the table is auto-restored from the last valid snapshot. Reason: the signRatio mechanism above only works if both the primary and the secondary are unambiguously single-signed.

Worked example (AQFP transformer K pair): an AQFP transformer has two coupling coefficients tied by Ext Group: the primary K1 with Initial +0.5 and the secondary K2 with Initial -0.5. The user sets K1's range to $[0.3, 0.7]$. K2 inherits the signRatio -1 and its locked range becomes $[-0.7, -0.3]$ (the bounds are mirrored and swapped to preserve Min < Max). If the user accidentally edits K1's Min to -0.1, that crosses zero — the change is rejected, an alert is displayed, and the table reverts.

5 Margin Calculation

To calculate the working margin range of the given netlist, qCS sweeps each selected parameter around its initial value and measures, for each step, whether the simulated waveform still matches the user-specified expected output pattern. The margin tab exposes two analysis sub-tabs — *Pulse Settings* and *Phase Settings* — and qCS selects one of them automatically from the signal types declared in the netlist's `.PRINT` statements (see §5.3). The same Step Size, Multi-threading, Threads, Calculate, and Visualize controls apply to both sub-tabs.

The output pattern number should be equal to the number of nodes that are printed to the simulator output file. Multiple patterns can be written if the number of ".PRINT" lines increased on the netlist for a related ".FILE" line.

Each pattern entry has three fields: start time, end time, and a signed pulse-count token. The same triple format is used uniformly across SFQ pulses, AQFP ± pulse pairs, and RQL / FPL phase jumps; bipolar outputs are expressed as two adjacent windows of opposite sign. A positive token (e.g. +1) asserts that one positive-going pulse or phase jump is expected inside the window; a negative token (e.g. −1) asserts one negative-going pulse or phase jump. The Fill button emits the bare +1 / −1 form (one window per detected event), and a window written by hand can carry one of two prefixes. == (e.g. ==+1) keeps the ordinary window that expects one event, the same as the bare form; >= sets a minimum count: >=*N* (e.g. >=1, >=3, >=−2) asserts *N* or more pulses or phase jumps of that sign inside the window. The sign can be written on either side of the prefix (>=−2 and −>=2 are equivalent).

The tool checks the number of pulses (or phase jumps, in Phase Settings) that meet the threshold criteria set on the active sub-tab and compares this number to the expected pattern. The initial values of pulse height, pulse width, expected output band, and phase jump threshold are internally calculated from the simulation data — the user must verify them before proceeding to the analysis.

5.1 Pulse Settings sub-tab

The Pulse Settings sub-tab is the analysis path for pulse-height detection on voltage/current waveforms. qCS auto-selects it when every `.PRINT` line on the netlist reports a voltage or current quantity. Its controls are:

- *Minimum Pulse Height* (*Positive Level* / *Negative Level*): Any pulse that crosses this voltage / current threshold counts as a logic pulse. When *Negative Pulse* is unchecked only the Positive Level is editable; when checked, both Positive Level and Negative Level fields are enabled. Recommended setting: at least 70% of the expected pulse height observed in simulation.
- *Pulse Width* (*Positive Level* / *Negative Level*): Expected distance between rising and falling edges of a logic pulse, per polarity.
- *Output Expected Band* ± (ps): Symmetric tolerance band used by the *Fill* button to lay out pattern windows around detected pulses.
- *Negative Pulse*: Checkbox that enables the Negative Level column so Fill detects downward pulses as well as upward ones. With it ticked, Fill emits one window per detected pulse using the appropriate sign (+1 for an upward spike, −1 for a downward spike).

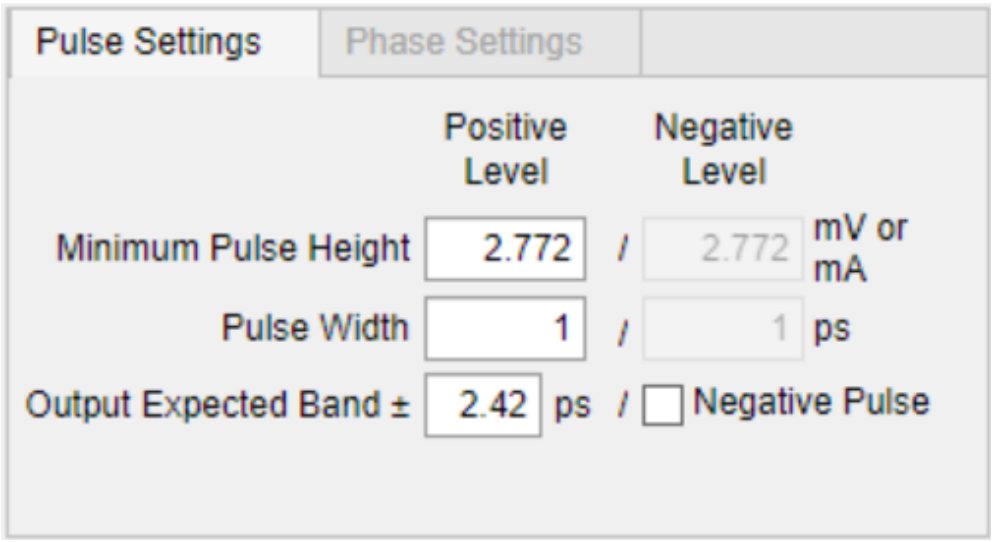


Figure 16: Margin tab, Pulse Settings sub-tab.

Worked example. On the Margin tab, the Pulse Settings sub-tab is selected by default. After Run Simulate, the Fill button populates the form with *Minimum Pulse Height* = 2.772 mV, *Pulse Width* = 1 ps, and *Output Expected Band* $\pm$ 2.42 ps (Fig. 16); the *Negative Pulse* checkbox stays off for a single-polarity SFQ output, so the mirror-side fields are pre-filled to the same values but are not used. A typical Fill-generated entry is `1000ps 1200ps +1`, meaning "one positive pulse expected between 1 ns and 1.2 ns"; additional pulses become additional adjacent windows. The Check button then overlays the detected pulses on the waveform so the user can confirm.

5.2 Phase Settings sub-tab

The Phase Settings sub-tab is the analysis path for phase-jump detection on continuous phase trajectories. qCS auto-selects it when every `.PRINT` line on the netlist reports a PHASE quantity. Its controls are:

- *Phase Jump Threshold* (UI: *Phase Threshold*): Minimum signed phase change, in radians, accumulated over one detection window for the window to be counted as a valid jump. Default is 5 rad (about 1.6π); a window whose total phase change $\Delta\phi$ spans several thresholds is split into $\lfloor|\Delta\phi|/\text{threshold}\rfloor$ jumps, each located where the phase crosses the middle of its step, one threshold wide. With the default 5 rad, a window holding up to three consecutive 2π slips is counted exactly, while four or more are overcounted (four slips are reported as five); the located times follow the threshold spacing, not the 2π spacing of the slips, so in a long burst a located time can land on the tail of the preceding slip. A positive accumulation counts as one (or several) positive "phase pulse"; a negative accumulation counts as one (or several) negative phase pulse.
- *Speed Threshold* (UI: *Phase Slew Rate*): Minimum smoothed velocity (rad/ps) that the candidate window must reach at some point before it is accepted — if the peak velocity inside a window never gets this high, the change is treated as too small to count, so a shallow wiggle or a tiny phase excursion is filtered out even when its direction is consistent.

- *Smoothing Window* (UI: *Moving Average Window*): Window length (in ps) of the moving-average filter applied to the raw phase trajectory before the velocity is computed. Used to suppress high-frequency simulator noise on the phase signal itself, so the subsequent velocity is not dominated by sample-to-sample ripple.
- *Momentum Factor*: EMA smoothing factor α (0 to 1) applied to the instantaneous phase velocity. The detector identifies a jump by opening a window when the smoothed velocity is positive (or negative) and closing it when the smoothed velocity returns to zero, then totalling the phase change inside the window. A real junction phase rarely climbs monotonically through a 2π transition — it often dips back, plateaus, then continues up, and the Smoothing Window alone cannot remove these mid-jump dips because they are not high-frequency noise. The Momentum Factor carries inertia from the prior velocity into the current sample, so a brief dip or plateau does not drag the smoothed velocity back to zero and prematurely close the window; the whole stepwise climb is therefore captured as a single sustained transition and its full multi-2π extent is counted by the Phase Jump Threshold rule above. Higher α gives more inertia (more robust to dips/plateaus) but slower reaction to genuine direction reversals; set to 0 to disable.

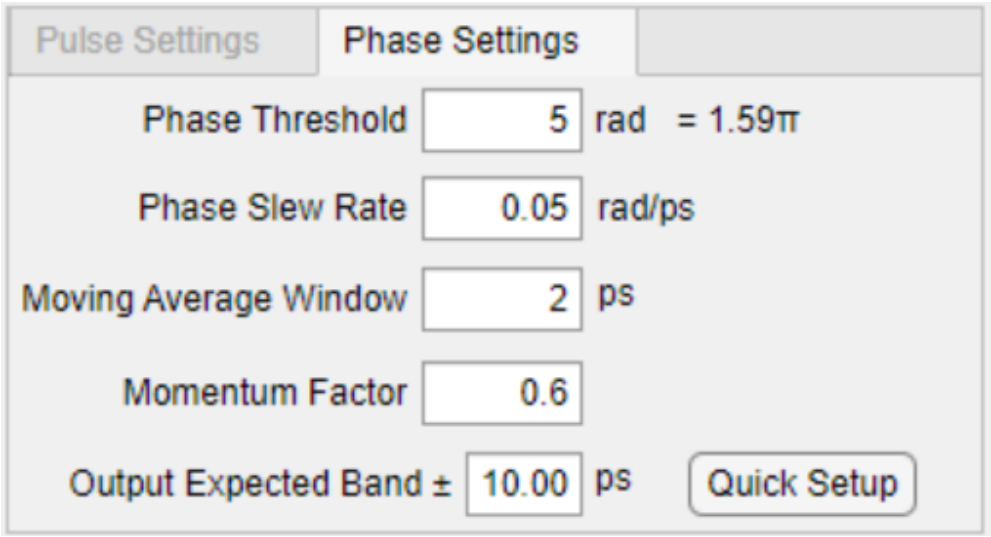


Figure 17: Margin tab, Phase Settings sub-tab.

Worked example. When the netlist's `.PRINT` output is phase data, the Phase Settings sub-tab is auto-selected; this circuit's phase-detection parameters are then auto-populated from the simulation as shown in Fig. 17 (Phase Threshold = 5 rad $\approx 1.59\pi$, Phase Slew Rate = 0.05 rad/ps, Moving Average Window = 2 ps, Momentum Factor = 0.6, Output Expected Band $\pm$10 ps). The Fill button then reads those parameters and identifies one positive jump per positive-direction phase transition and one negative jump per negative-direction phase transition. A typical Fill-generated entry is `200ps 400ps +1  400ps 600ps -1`, meaning "one positive jump expected in 200–400 ps and one negative jump expected in 400–600 ps".

5.3 Sub-tab choice and analysis-mode validation

qCS chooses the active sub-tab automatically from the signal types declared in the netlist's `.PRINT` statements (the `PRTYPE` field on each PRINT). When every PRINT reports

a PHASE quantity, the Phase Settings sub-tab is selected and the Pulse Settings sub-tab is greyed out; when no PRINT reports PHASE, the Pulse Settings sub-tab is selected and the Phase Settings sub-tab is greyed out; when PHASE and voltage/current PRINTs are mixed, both sub-tabs are greyed and Fill / Check are disabled until the user resolves the mix on the Simulation tab. The selection is locked: clicking a greyed sub-tab snaps the selection straight back to the active one, so the user never operates on a sub-tab whose detector does not match the simulated signal.

The Fill, Check, and Calculate buttons read parameters from whichever sub-tab is currently active. Internally, the per-particle cost evaluation chooses the phase-detection or pulse-detection code path from the active output pattern labels — patterns containing PHASE, P(, or NODEP go through the phase path; everything else goes through the pulse path. Mixing phase and voltage/current pattern labels in the same run is rejected: when the user enters the Margin / Yield / Optimization tab with such a mix, the tab is disabled and a "Tab Locked" alert names the offending labels so the user can deselect either the PHASE rows or the voltage/current rows on the Simulation tab before retrying.

5.4 Pattern syntax and examples

Every analysis mode (SFQ pulse, AQFP ± pulse pair, RQL / FPL phase jump) uses the same uniform triple format: *t1 t2 sign*. The sign token is +1 for one expected positive-going event and -1 for one expected negative-going event; bipolar outputs are described by adjacent windows of opposite sign.

- **Example 1 (RSFQ, Pulse Settings):** 1000ps 1200ps +1 — one positive SFQ pulse expected between 1 ns and 1.2 ns.
- **Example 2 (RSFQ, two consecutive pulses):** 1000ps 1200ps +1 1300ps 1500ps +1 — two positive SFQ pulses, one in each window.
- **Example 3 (AQFP, Pulse Settings + Negative Pulses):** 500ps 700ps +1 700ps 900ps −1 — one positive pulse followed by one negative pulse (the AQFP ± pair).
- **Example 4 (RQL / FPL, Phase Settings):** 200ps 400ps +1 400ps 600ps −1 — one positive-direction phase jump followed by one negative-direction phase jump.

Example 1 shows the single-event SFQ form. Example 2 illustrates that multiple events become multiple adjacent windows of the same sign — each window asserts exactly one event of its sign. Examples 3 and 4 show the bipolar form: each polarity gets its own window, regardless of whether the underlying physics is a voltage pulse pair (AQFP) or a phase-jump pair (RQL / FPL). A window written by hand can also carry the prefix ==, which keeps the ordinary window that expects one event, or >=, where >=*N* expects at least *N* events (§5.5). A window whose sign token is 0 is not counted, so a row such as `0ps 2000ps 0` expects no event on that signal before TSTOP. Times may also be written in exponent notation (for example 1e3ps); qCS reads them by value and rewrites them as plain numbers (1000PS).

5.5 How a pattern is matched

Each observed signal is matched on its own, and positive and negative events are matched separately, each against the windows of its own sign; the netlist passes when every signal passes. In Pulse Settings, negative pulses are detected only when *Negative Pulse* is ticked; in Phase Settings, both directions of phase jump are always detected.

Windows with the >= prefix are resolved first. A >=*N* window counts every event of its sign that lies inside it and must reach *N*; an event inside two >= windows counts for both. An event inside a >= window can also serve an ordinary window that overlaps it: an ordinary window keeps one such event when it has no event of its own outside every >= window. All other events inside >= windows are then set aside, and the remaining events are checked against the ordinary windows with three rules:

- *No surplus*: from the start of the output data up to the netlist's `.TRAN` TSTOP, the number of events must not exceed the number of ordinary windows. Events in the automatic extension after TSTOP are not counted by this rule.
- *No shortfall*: the total number of events, including those in the automatic extension, must be at least the number of ordinary windows.
- *In order*: sorted by time, the *k*-th event must fall inside the *k*-th ordinary window, in the order the windows are written; with *N* ordinary windows, events after the *N*-th are ignored.

An event that falls outside every window therefore counts against the *no surplus* rule, and so does an event of a sign for which the pattern has no window: a signal whose pattern holds only +1 windows fails when a negative phase jump appears before TSTOP, and a row holding only `0` windows fails on any event before TSTOP.

Overlapping windows. An event in the overlap of two >= windows counts for both. An event in the overlap of a >= window and an ordinary window counts for the >= window and can still serve the ordinary window, so the ordinary window has a surplus only when two or more events lie in it outside every >= window. For example, with `100ps 140ps >=1  120ps 160ps +1`, a single event at 130 ps passes both windows, events at 130 and 150 ps also pass, and events at 130, 145 and 155 ps fail. Two ordinary windows that overlap are paired by the *in order* rule, in the order they are written.

Example. An SFQDC output slips a varying number of times per input. Writing its pattern as `100ps 140ps >=1  200ps 240ps >=1` lets each window pass with one or more positive jumps inside it, however many there are, while a jump between the two windows fails the pattern.

Events close to TSTOP. The *no surplus* rule stops exactly at TSTOP. An output event can lie just after TSTOP, typically when the input is a continuous pulse train. A small timing shift at a perturbed parameter value then moves it before TSTOP, where it counts as a surplus, and the margin reflects where TSTOP falls rather than the circuit. Before a margin, yield or optimization run, qCS simulates the netlist once at its current values; if it finds such an event after TSTOP within half the width of that signal's latest window (the tolerance band, for a pattern made by Fill), it lists the signals and event times in a warning shown when the run finishes. A signal without a window of its own, such as a `0` row, is checked as far past TSTOP as the widest span of the other signals. The run itself is not stopped. To avoid the effect, lower the input frequency so the output is quiet around TSTOP, cover these events with a >= window (for example >=1), or move TSTOP into a gap between events.

Input checks. When a pattern is entered, every window must end after it starts, and the windows of a row must be written in time order (compared across both signs, after unit conversion); otherwise qCS reports an error. When two windows of the same sign overlap and at least one of them is a >= window, qCS shows a warning and accepts the pattern; windows that only touch at an end point do not count as overlapping. While a pattern has an error (format, row count or window order), the text is kept as typed, Calculate, Check and Optimize are disabled, and the other tabs are greyed out and cannot be selected until the pattern is corrected or cleared.

An event is counted in the window that contains its detected time, so a >= window should enclose the whole burst of events with some margin, and a window boundary should not cut through a burst. In Phase Settings, the number of consecutive slips that can be counted exactly is set by the *Phase Jump Threshold* (see its description above).

5.6 Fill, Check, and Calculate

Once the analysis sub-tab is selected and its parameters reviewed, click the Fill button to auto-generate the expected pattern from the current simulation waveform. The Fill button reads the parameters from the *currently selected* sub-tab (pulse height/width when Pulse Settings is active; phase jump threshold when Phase Settings is active). To verify the inferred pulses or phase jumps, click Check: a separate window plots the waveform with detected events overlaid, so the user can confirm that the thresholds picked up the right transitions before running the full margin calculation. If Fill detects no pulse or phase jump on a signal, it writes a single `0` window over the `.TRAN` range for that signal, meaning that no event is expected there, and shows a warning; the other signals are filled as usual.

To start the margin calculation, click on the Calculate button. Note that if the simulator generates an error during the calculation (for example, an integration error), the particular parameter set for the netlist will be considered incorrect operation, and the margin calculation will continue with the next parameter sample. There will be no pop-up warning about this case — it is the user's responsibility to assign satisfactory transient values in the netlist for the selected simulator.

5.7 Per-component margin scans

For serial-JJ-chain configurations, every physical junction inside a chain is scanned independently. After the scan, the Margin Table displays one row per physical junction using the canonical chain naming `B<n>`*suf*, where the suffix *suf* reflects the chain's Group classification (`_int<I>` for an Int-Group-only chain, `_int<I>_ext<E>` when an Ext Group is also bound, or `_ext<g>` for a 7-column implicit chain). For example, a chain bound to Int Group 1 yields rows `B1_int1`, `B2_int1`, ..., `B<N>_int1`, so the user can see whether one specific junction in the chain is the margin-limiting element.

In addition, when an Ext Group is in use, each Ext-Group secondary row gets its own independent margin scan, and the result row is inserted *directly after* its primary row in the Margin Table (not at the end). For K-typed secondaries, the displayed Initial and Final values include the signRatio so the secondary's sign matches the user's original convention.

Worked example. The FPL `AND` cell from the walkthrough in §8 holds three serial chains: Group 1 and Group 2 share Ext Group 1 (so their chain-length parameter is co-optimised), and Group 3 is standalone. At the initial chain lengths (N = 2 for the ext-bound pair, N = 3 for the standalone), the Margin Table contains seven chain-expansion rows: `B1_int1_ext1`, `B2_int1_ext1` (Group 1); `B1_int2_ext1`, `B2_int2_ext1` (Group 2); `B1_int3`, `B2_int3`, `B3_int3` (Group 3) — all under Cell column = `AND`. Each chain row carries its own lower / upper margin bound; when the chain is not the margin-limiting element the bounds saturate at the ±50% default scan range. The ext-only mirror pairs (`B1/B5` on Ext Group 2, `B4/B8` on Ext Group 3) keep their original names and each member contributes its own row to the table.

5.8 Margin overrun gating against pre-extension TSTOP

During margin / yield / optimize, qCS may automatically extend the netlist's `.TRAN TSTOP` so the simulation runs past each signal's latest expected pattern window by half that window's width, and moves TSTART earlier when needed, so that the output starts at least half of each signal's earliest window width before that window. TSTOP is never shortened and TSTART is never moved later. Only the copies used for judging are changed; the netlists exported after optimization keep the user's `.TRAN`. The user's pre-extension TSTOP is preserved internally (in picoseconds) on the transient object as `PRE_EXTEND_TSTOP_PS`. The pattern-overrun check ("did extra pulses appear after the expected window?") uses the *user's* TSTOP, not the extended one — so a late pulse landing inside the auto-extension is treated as a filler that may cover for late expected pulses, never as an overrun violation. This is what lets JTL-style cells with non-trivial intrinsic delay run correctly: the simulator sees enough time to produce the expected pulses, and overrun violations are only attributed to pulses outside the user's intended end-of-test point. An event that sits just after TSTOP can still move inside it at a perturbed parameter value; see *Events close to TSTOP* in §5.5.

5.9 Visualize and Export

Once the margin calculation completes, the Visualize button opens a horizontal bar graph titled *Margin Results* that draws each parameter row's lower margin (orange, on the left of zero) and upper margin (blue, on the right of zero) in percent. Each y-axis label is formed by concatenating the row's Cell and Component fields with a hyphen, and any underscore in the Component name is rendered as a hyphen too — so a chain-expanded JJ recorded in the Margin Table as Cell = `AND`, Component = `B1_int1_ext1` appears on the bar graph as `AND-B1-int1-ext1` (Fig. 18). The Export button creates an Excel file containing the Cell Table and Margin Table as well as a companion text file with the parameters used (pulse / phase thresholds, step size, etc.). Note that the complete margin analysis can only be achieved by adding all of the cell components to the parameter table; partial selections do not produce trustworthy results.

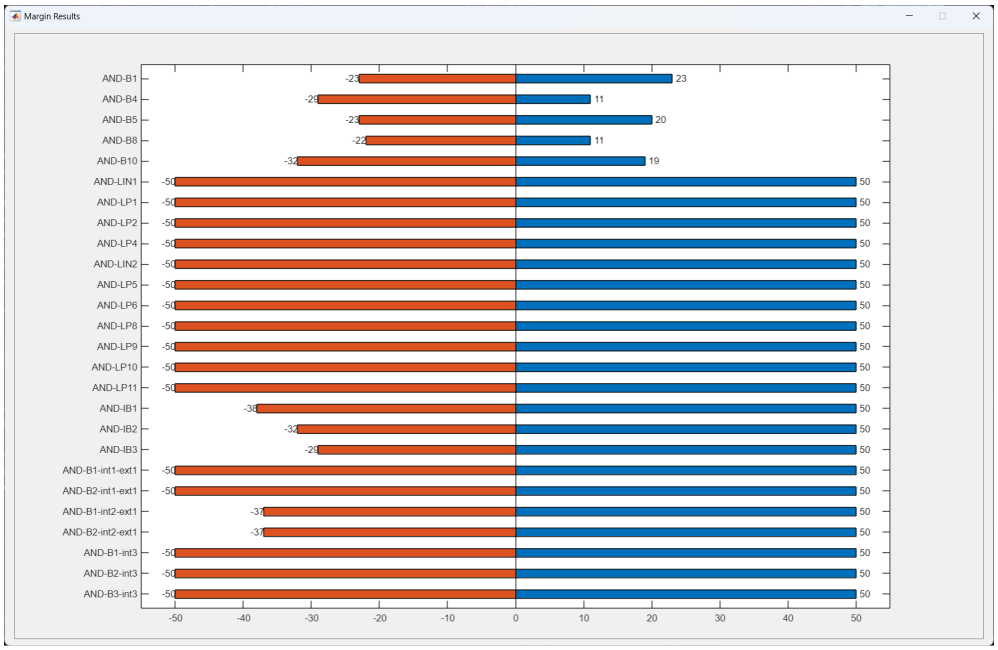


Figure 18: Margin Results bar graph from Visualize on the FPL `AND` cell: orange = lower margin %, blue = upper margin %. The chain expansions appear as `AND-B1-int1-ext1`, `AND-B2-int1-ext1` (Group 1), `AND-B1-int2-ext1`, `AND-B2-int2-ext1` (Group 2), `AND-B1-int3`, `AND-B2-int3`, `AND-B3-int3` (Group 3), interleaved with the standalone JJs (`AND-B1`, `AND-B4`, `AND-B5`, `AND-B8`, `AND-B10`), inductors, and bias resistors. Bars saturate at the $\pm 50\%$ default scan range when the parameter is not the margin-limiting element.

6 Yield Analysis

To calculate the yield, there are additional parameters that need to be defined. The MC sample parameter simply provides the information for the number of netlists. Parameter STD is given for the standard deviation of selected parameters. It should be noted that the adjustments will only affect the selected components, and their values will remain within 3σ control limits during the randomization process. The Pulse / Phase sub-tab structure introduced for the Margin tab also applies on this tab; the user picks the same sub-tab as on the Margin tab (Pulse Settings is the default; switch to Phase Settings for phase-mode output nodes). Mixing PHASE and voltage/current output labels in one run is rejected with the same Tab Locked alert as on the Margin tab.

After the yield analysis, the Yield percentage will be shown, and the bar graph will be updated. If there are multiple selected cells, qCS will create a unique netlist for each cell and check them individually. However, there may be cases where multiple cells fail simultaneously — the number of all-failed bars might therefore not equal the summation of the individual cell-failure bars. If there are global components, they will be put into the cell component listing and evaluated together. The user can save the data by hovering the mouse in the top right corner of the graph.

6.1 Group behaviour during yield

Two rules govern how grouped components are randomized:

- **Per-physical-component randomization**. The yield path hard-codes `parameterGrouping = 0`: every physical component is randomized independently inside Monte-Carlo, regardless of Int Group or Ext Group membership. This conservative behaviour avoids false-positive yields that would arise if correlated offsets masked weak components within a group. The *Make All Parameters Uncorrelated* checkbox is a no-op kept for layout stability.
- **Ext Group K-row signRatio is preserved during randomization**. When a K-row primary and secondary are tied by Ext Group, the secondary's sample equals the primary's sample multiplied by the secondary's signRatio of ± 1. This preserves the original sign relationship.

Worked example (AQFP transformer). Suppose the design has K1 with Initial +0.5 and K2 with Initial -0.5 tied by Ext Group. Across 1000 MC samples, if K1 happens to draw 0.48 in one trial, K2 takes -0.48 in that same trial (signRatio -1), *not* an independent sample around -0.5. This preserves the physical anti-correlation of the transformer windings. Within each MC trial, K1 itself is still randomized independently of every other parameter.

6.2 Synthetic inter-L rows are skipped

When a serial-JJ chain is configured with *Insert inter-L* on, the bridging inductors created by the chain rewrite are book-kept as synthetic rows (their values are derived from the chain length *N*, not chosen independently). Yield's MC sampling skips these synthetic rows by default, via a margin-skip mask (`serialJJInterLMask`), so the variance budget is not consumed by inductors that have no independent design choice. Users who explicitly want to randomize chain bridging inductors can flip the corresponding flag — see Section 7 for cross-reference.

7 Optimization

qCS uses a hybrid algorithm combining Automatic Niching Particle Swarm Optimization (ANPSO) with the Fireworks Algorithm (FA) — referred to throughout the manual as ANPS-FW — to optimize circuit parameters. The optimizer compares the expected control patterns with those obtained from each particle's simulation and uses the resulting margin score as the objective. The algorithm is independent of the analysis mode: the inner margin evaluation runs on whichever sub-tab (Pulse Settings or Phase Settings) is currently active. The user picks the sub-tab manually on the Margin tab (Section 5); the Optimization tab inherits that choice through the shared sub-tab state.

In the algorithm, a set of particles searches through the search space for the global optimum. On each iteration, every particle additionally acts as a firework, creating sparks to facilitate better local search. By combining the two algorithms, qCS can refine component values both locally and globally. If the user is satisfied with the current result and wants to terminate the optimization, it can be stopped by clicking the Stop button under the Log sub-tab. There are breakpoints between optimization steps, but some of the inner loops are uninterruptible; depending on the number of components selected and the number of particles, it may take some time to actually exit after pressing Stop.

If the result table is filled with data, the Export button will be enabled for creating related data files. The parameters in blue on the interface do not affect the algorithm itself; the others are intended for experienced users to adjust the optimization process. Users can also track improvements and progress in the Trend sub-tab.

7.1 Ext Group secondaries in optimization cost

The *Enable Mirrors Affect Optimize* checkbox controls whether Ext-Group secondaries contribute to the optimization cost during the PSO inner loop. Two flavors of secondary are gated together:

- **Non-JJ ext-only mirrors** (R / L / C / K rows whose Int Group is blank but whose Ext Group is set, sharing the primary's value).
- **Secondary JJ chains in an Ext fold** (a second serial-JJ chain that shares an Ext Group label with a primary chain — both chains then run with a shared area parameter and a shared chain length).

When ticked, each such secondary gets its own independent margin scan inside the inner cost loop and its margin folds into the overall objective; when unticked, only the primaries' margins enter the cost. The checkbox auto-enables whenever any IntGroup or ExtGroup cell on the Variables tab is non-empty.

Worked example (RSFQ buffer). A buffer cell has two symmetric shunt resistors RB1 (primary) and RB2 (secondary, Ext Group 1). Without *Enable Mirrors Affect Optimize*, the optimizer measures only RB1's margin during PSO; an optimum that happens to leave RB2 sitting near a margin cliff is invisible. With *Enable Mirrors Affect Optimize* on, RB2's margin is scanned independently every iteration, so a robust solution must balance both. The auto-enable behaviour means the user does not need to remember to tick the box manually whenever a Group is in use.

7.1.0.1 Final margin output is always full-coverage.

The gate above only applies inside the PSO cost loop. When the user clicks *Calculate* on the Margin tab or qCS writes

the post-optimization summary, every Ext-Group secondary is scanned regardless of the checkbox state, so the displayed margin table always shows the complete picture.

7.1.0.2 Runtime cost. Each secondary scanned by the cost loop is re-scanned every PSO iteration, so the per-particle cost grows roughly linearly with the number of mirror secondaries (counting both flavors above). On cells with many group siblings, untick the checkbox to fall back to primary-only scoring during PSO if turn-around time matters more than mirror-aware optimization.

7.2 TSTOP auto-extension during optimization

qCS auto-extends `.TRAN TSTOP` during PSO so the simulation always covers the latest expected pattern window plus half that window's width on each signal. The user's pre-extension TSTOP is preserved on the netlist's transient object as `PRE_EXTEND_TSTOP_PS`; the pattern overrun check uses the user's TSTOP, not the extended one. The user does not need to choose between "a TSTOP large enough for late expected pulses" and "a TSTOP tight enough to catch overrun bugs" — qCS uses different TSTOPs for different purposes.

Worked example. A user writes `.TRAN 0.25p 1n` in the netlist and writes a pattern reaching out to 1.5 ns. During optimization, qCS internally extends TSTOP to (at least) 1.5 ns plus half the width of that last window, so all expected pulses are inside the simulation window. The overrun check still uses 1 ns as the user's intended end-of-test point, so a stray late pulse appearing at 1.4 ns inside the extension is not counted as an overrun violation.

7.3 Parallel workers and pulse mode

On multi-threaded runs, every parallel worker queries `getNetlistPulseMode` on its assigned netlist clone and uses the result locally for its inner margin evaluation. This guarantees that a parallel pool can never have one worker doing pulse-mode detection while another worker on the same particle is doing phase-mode detection. The user does not need to take any action; the synchronization is internal.

7.4 Phase-mode optimization

qCS also supports optimization driven by phase data. When the active sub-tab is *Phase Settings*, the inner cost loop measures phase jumps rather than pulse heights. If the netlist has no detectable pulses or phase jumps in the selected output node, the corresponding boxes will be disabled. Otherwise, the tool will initially assign values obtained from the waveforms; it is expected that the user verifies them before proceeding.

8 Examples

This guide provides one example netlist for each of the four supported logic families. These files are adjusted in accordance with the restrictions stated in Section 2. The RSFQ DFF netlist comes from the JoSIM GitHub; the AQFP buffer chain was provided by Professor Christopher Ayala from Yokohama National University. All four example netlists ship under the `Netlists/` directory of the qCS distribution.

- *RSFQ JoSIM Example File:* `RSFQ_DFF_JOSIM.cir`
- *AQFP JoSIM Example File:* `AQFP_BUFFER_CHAIN_JOSIM.cir`
- *RQL JoSIM Example File:* `RQL_ANDOR_JOSIM.cir`
- *Fast Phase Logic JoSIM Example File:* `FPL_DFF_JOSIM.cir`

The user has the flexibility to create a unique testbench for a cell, but note that if there is a series of cells, pulse-timing or phase-jump-timing errors will accumulate. In these example files, appending "_TEST" to a subcircuit name identifies it as the one to be tested.

8.1 RSFQ

8.1.1 Loading Netlist

A single netlist file can be loaded by using the Browse button on the Netlist tab. Upon successful operation, a message will appear. If there is a problem with the input file, qCS will indicate the potential issue.

For this example, `RSFQ_DFF_JOSIM.cir` from the `Netlists/` folder is used as input. Related operations are shown in Fig. 19 and 20.

The shunt voltage scaling factor is automatically calculated after reading the netlist by using the first Josephson junction's scaling factor (AREA) and its shunt resistor value. This value is assigned to the defined Josephson junction (JJ) model and applied to all JJs sharing the same model definition. Before proceeding, the user is allowed to modify the component values or add new commands to the netlist.

The netlist data will be converted into an object by clicking the Read Netlist button. A confirmation message will appear (Fig. 21).

8.1.2 Simulation

Proceed to the Simulation tab to confirm correct operation of the netlist. The number of plots is automatically adjusted by the number of ".PRINT" commands. Each checkbox corresponds to a data set; the selected data should contain pulses (Fig. 22).

8.1.3 Selecting Components

On the Variables tab, click the Add button to populate the parameter table with the components of the DFF cell that should participate in the analysis: the four Josephson junctions `B1`–`B4`, the bias resistor `RIB1`, the four loop inductors `L1`–`L4`, the inter-loop inductor `LIB1`, and the bias supply `VBIAS` (11 rows total). For this RSFQ walkthrough we leave *Enable Int / Ext Groups* **off** — the 7-column legacy layout is the simplest path for single-cell RSFQ designs, and same-Cell rows fold automatically into one parameter dimension per component at margin / yield / optimize time, so the Group column can stay empty.

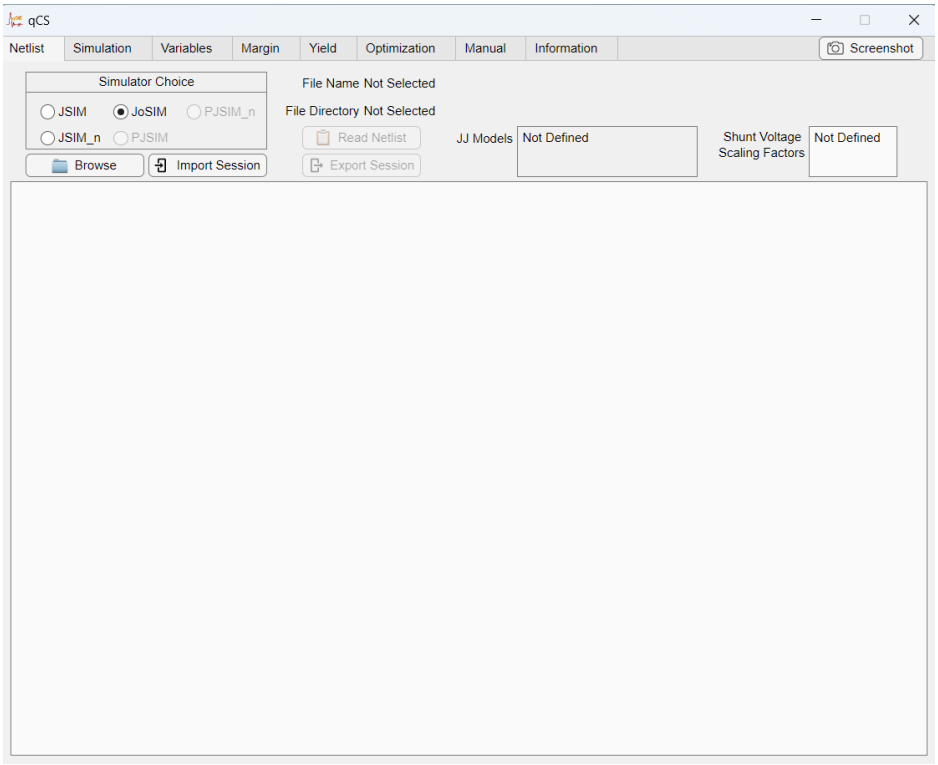


Figure 19: Netlist tab before opening a file (RSFQ).

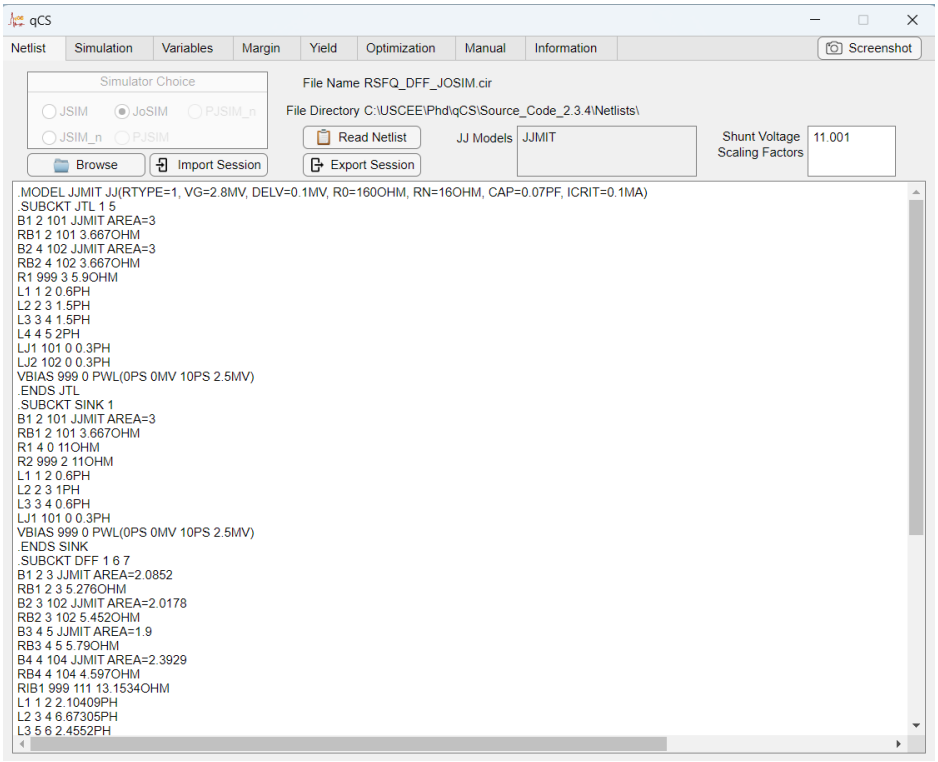


Figure 20: Netlist tab after opening a file (RSFQ).

The completed selection is shown in Fig. 23. The Min / Max columns default to ±50% around Initial; these are the bounds the optimizer will search.

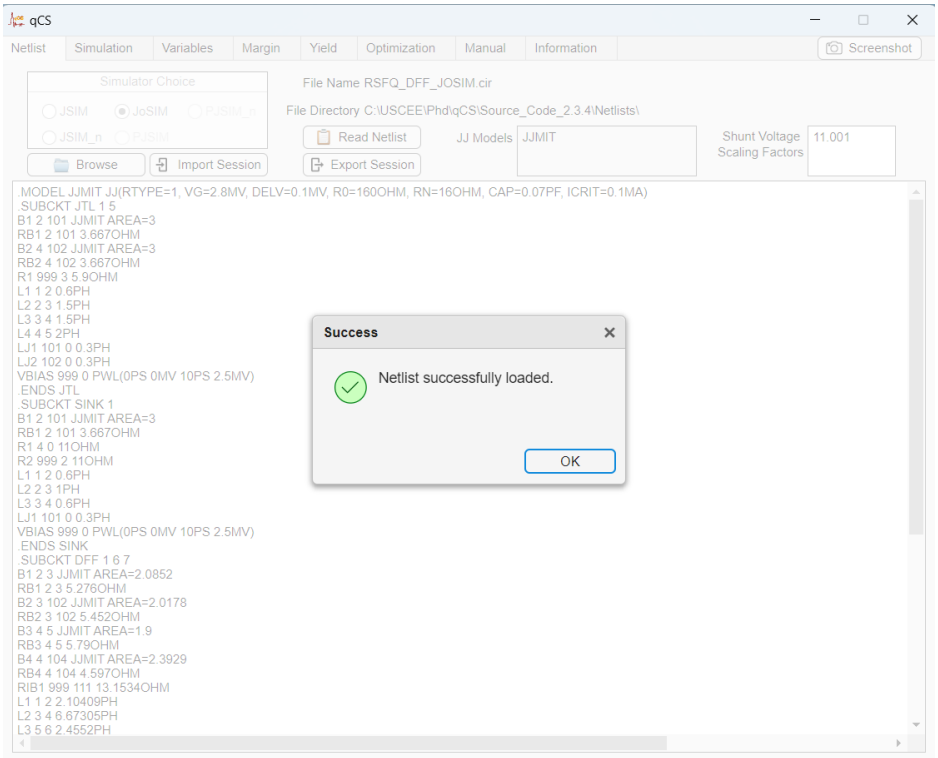


Figure 21: Netlist tab after reading the netlist (RSFQ).

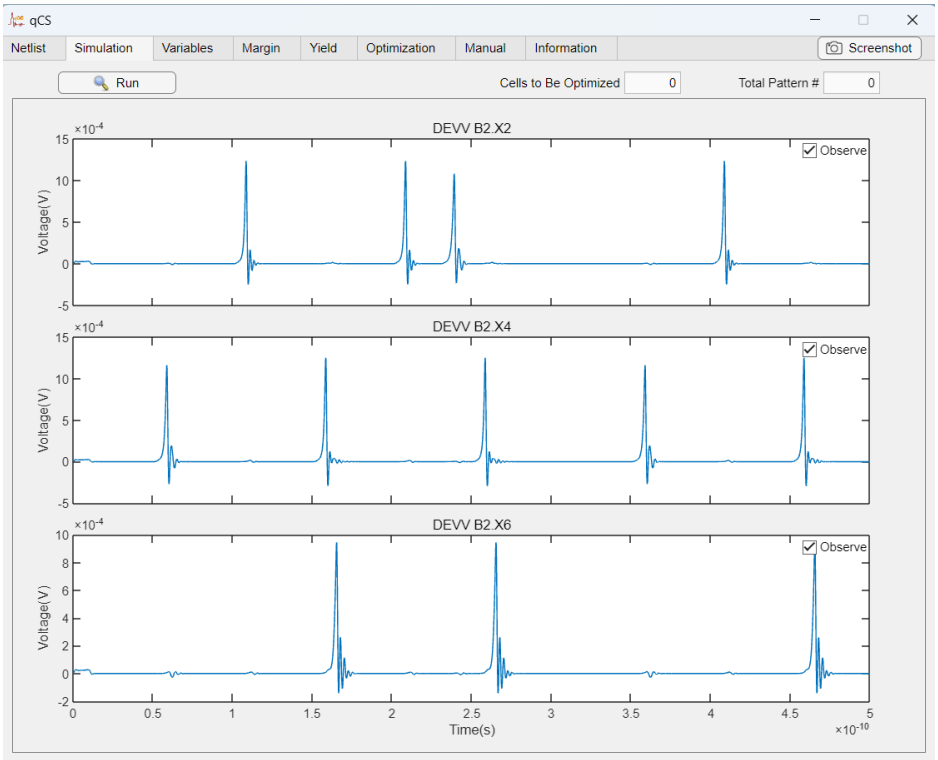
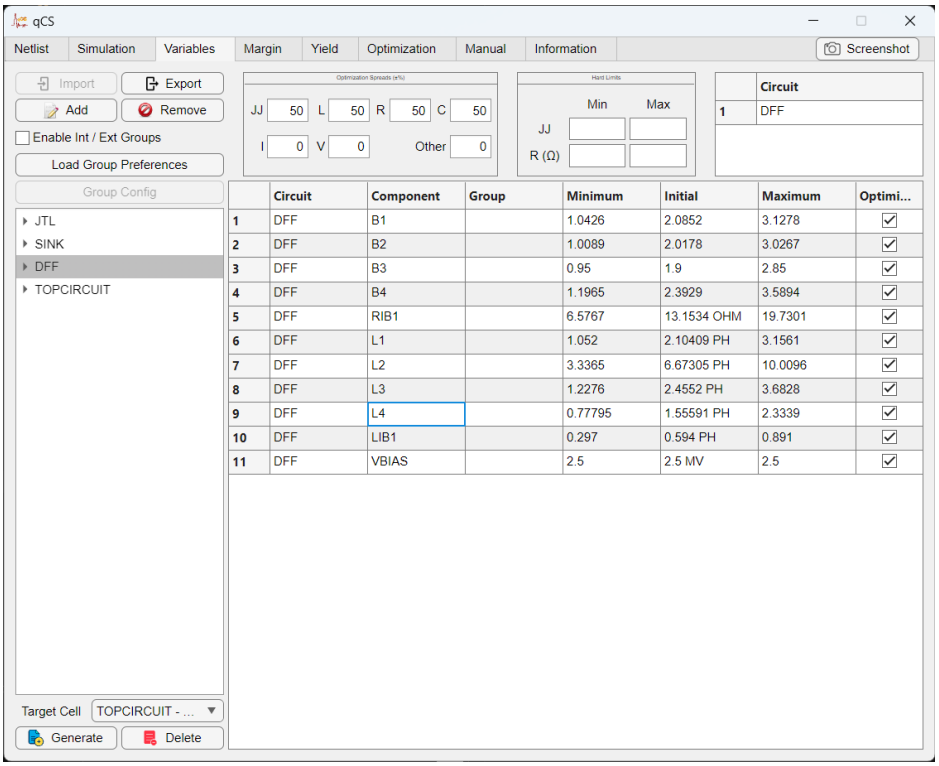


Figure 22: Simulate tab after completing the simulation (RSFQ).



	Circuit	Component	Group	Minimum	Initial	Maximum	Optimi...
1	DFF	B1		1.0426	2.0852	3.1278	☑
2	DFF	B2		1.0089	2.0178	3.0267	☑
3	DFF	B3		0.95	1.9	2.85	☑
4	DFF	B4		1.1965	2.3929	3.5894	☑
5	DFF	RIB1		6.5767	13.1534 OHM	19.7301	☑
6	DFF	L1		1.052	2.10409 PH	3.1561	☑
7	DFF	L2		3.3365	6.67305 PH	10.0096	☑
8	DFF	L3		1.2276	2.4552 PH	3.6828	☑
9	DFF	L4		0.77795	1.55591 PH	2.3339	☑
10	DFF	LIB1		0.297	0.594 PH	0.891	☑
11	DFF	VBIAS		2.5	2.5 MV	2.5	☑

Figure 23: Variables tab after adding the DFF cell's components (RSFQ): 7-column legacy mode with the Group column intentionally left blank.

8.1.4 Critical Margin Calculation

On the Margin tab, the Pulse Settings sub-tab is selected automatically (RSFQ default). Pulse height, pulse width, and output expected band are pre-filled from the simulation waveform; verify them, then click Fill to auto-generate the expected pattern. Check overlays the detected pulses on the waveform (Fig. 25).

Click Calculate to start the margin scan. Results appear in the Margin Table along with a per-cell summary in the top-right *Cell* panel (Fig. 26); the Visualize and Export buttons become active. Export produces an Excel file with the data plus a text file recording the pulse / phase parameters used.

8.1.5 Yield Analysis

Pulse Settings and the expected output pattern are also assigned automatically on the Yield tab. Set MC Sample # = 100 and click Calculate; the bar graph updates with the percentage of MC samples that match the expected pattern (Fig. 27).

8.1.6 Critical Margin Optimization

On the Optimization tab, click Optimize to run ANPS-FW. The Result, Log, and Trend sub-tabs are shown in Fig. 28–31. After convergence, qCS automatically generates optimized netlists and selects the best version (smallest Euclidean distance from the original where margins tie).

This concludes the qCS RSFQ JoSIM example.

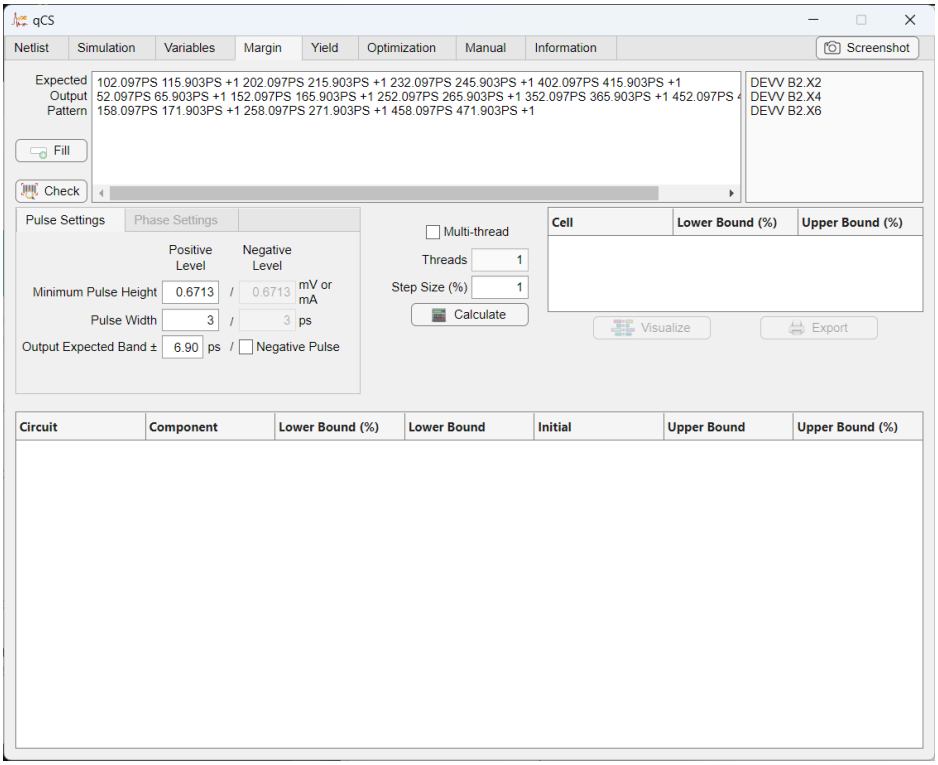


Figure 24: Margin tab after providing the specifications (RSFQ): Pulse Settings filled with *Min Pulse Height* = 0.6713 mV, *Pulse Width* = 3 ps, *Output Expected Band* = ±6.90 ps, and the expected output pattern populated for the three observed PRINT signals.

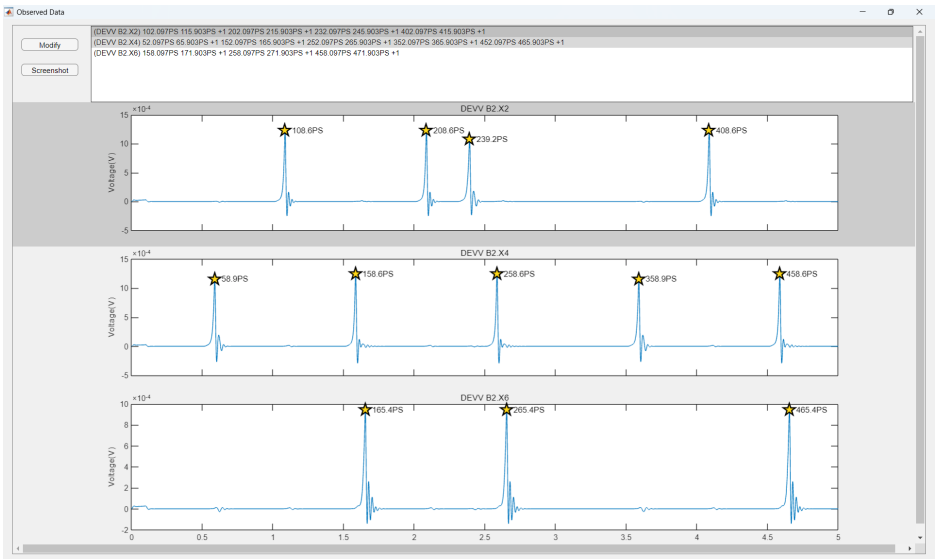


Figure 25: Waveform window opened by the Check button (RSFQ): one panel per observed PRINT signal (`DEVV B2.X2`, `B2.X4`, `B2.X6`); yellow star markers tag each detected SFQ peak with its timestamp.

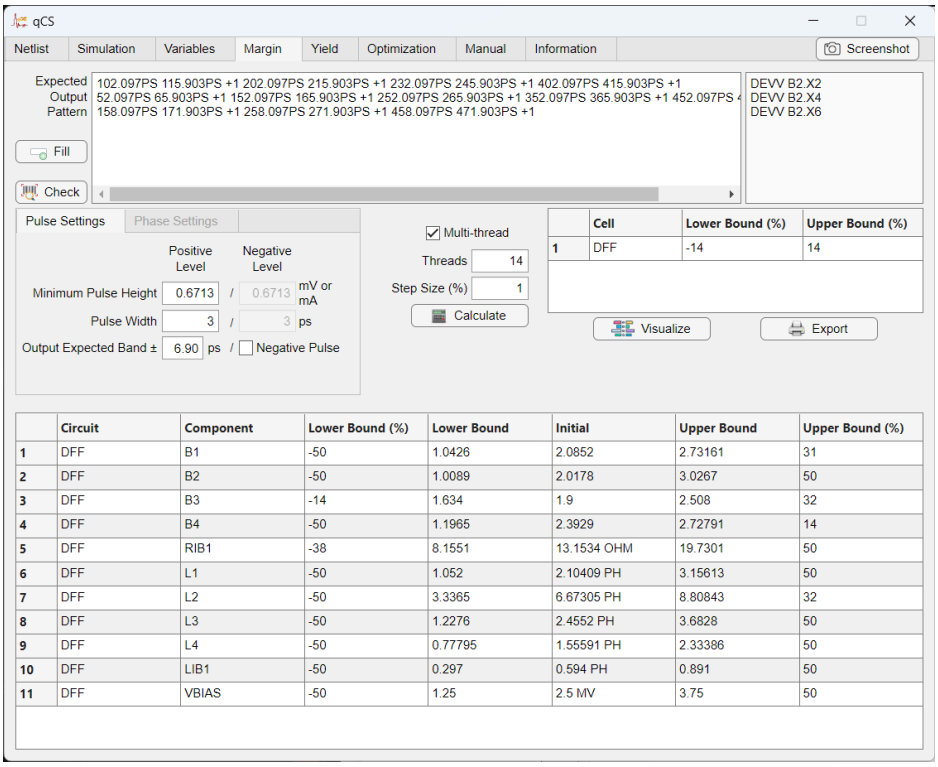


Figure 26: Margin results on Margin tab (RSFQ): top-right Cell panel summarises the DFF cell's overall margin window (−14% / +14%); the lower Margin Table lists per-component Lower Bound %, Lower Bound, Initial, Upper Bound, Upper Bound % for all 11 rows.

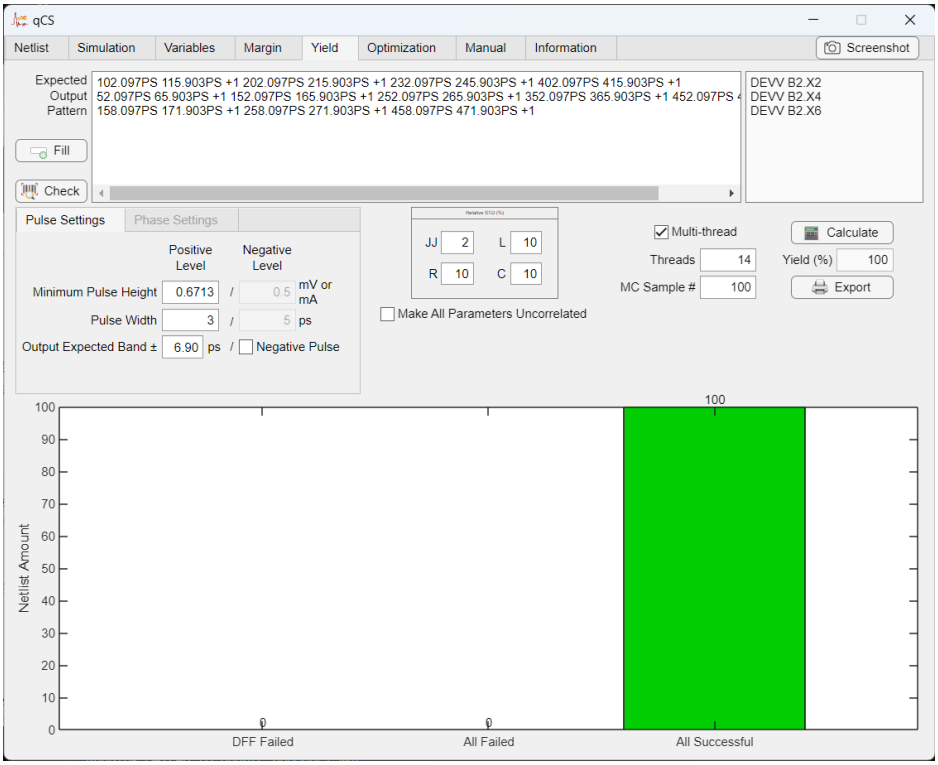


Figure 27: Yield analysis on the Yield tab (RSFQ).

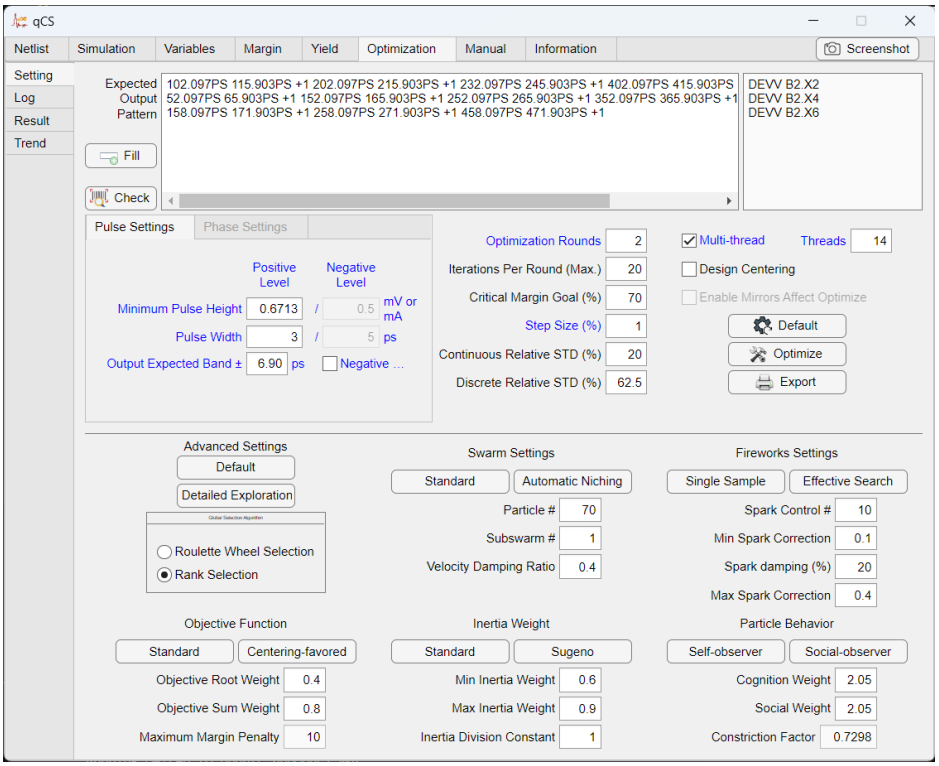


Figure 28: Optimization settings on Optimization tab (RSFQ).

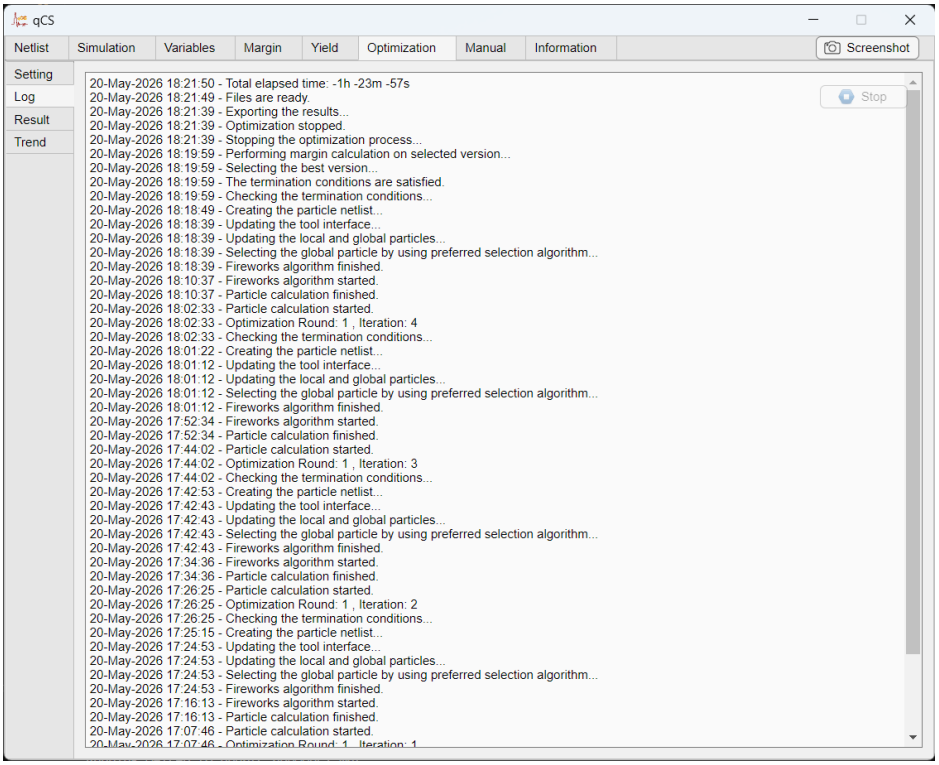
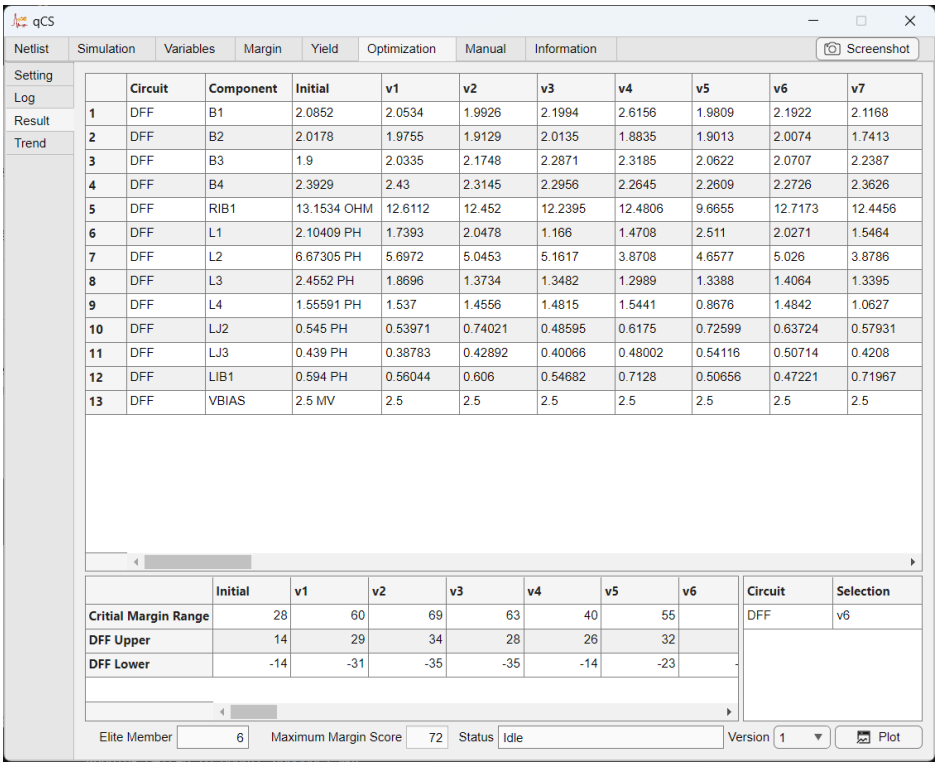


Figure 29: Optimization log on Optimization tab (RSFQ).



	Circuit	Component	Initial	v1	v2	v3	v4	v5	v6	v7
1	DFF	B1	2.0852	2.0534	1.9926	2.1994	2.6156	1.9809	2.1922	2.1168
2	DFF	B2	2.0178	1.9755	1.9129	2.0135	1.8835	1.9013	2.0074	1.7413
3	DFF	B3	1.9	2.0335	2.1748	2.2871	2.3185	2.0622	2.0707	2.2387
4	DFF	B4	2.3929	2.43	2.3145	2.2956	2.2645	2.2609	2.2726	2.3626
5	DFF	RIB1	13.1534 OHM	12.6112	12.452	12.2395	12.4806	9.6655	12.7173	12.4456
6	DFF	L1	2.10409 PH	1.7393	2.0478	1.166	1.4708	2.511	2.0271	1.5464
7	DFF	L2	6.67305 PH	5.6972	5.0453	5.1617	3.8708	4.6577	5.026	3.8786
8	DFF	L3	2.4552 PH	1.8696	1.3734	1.3482	1.2989	1.3388	1.4064	1.3395
9	DFF	L4	1.55591 PH	1.537	1.4556	1.4815	1.5441	0.8676	1.4842	1.0627
10	DFF	LJ2	0.545 PH	0.53971	0.74021	0.48595	0.6175	0.72599	0.63724	0.57931
11	DFF	LJ3	0.439 PH	0.38783	0.42892	0.40066	0.48002	0.54116	0.50714	0.4208
12	DFF	LIB1	0.594 PH	0.56044	0.606	0.54682	0.7128	0.50656	0.47221	0.71967
13	DFF	VBIAS	2.5 MV	2.5	2.5	2.5	2.5	2.5	2.5	2.5

	Initial	v1	v2	v3	v4	v5	v6
Critial Margin Range	28	60	69	63	40	55	
DFF Upper	14	29	34	28	26	32	
DFF Lower	-14	-31	-35	-35	-14	-23	

Circuit	Selection
DFF	v6

Figure 30: Optimization results on Optimization tab (RSFQ).

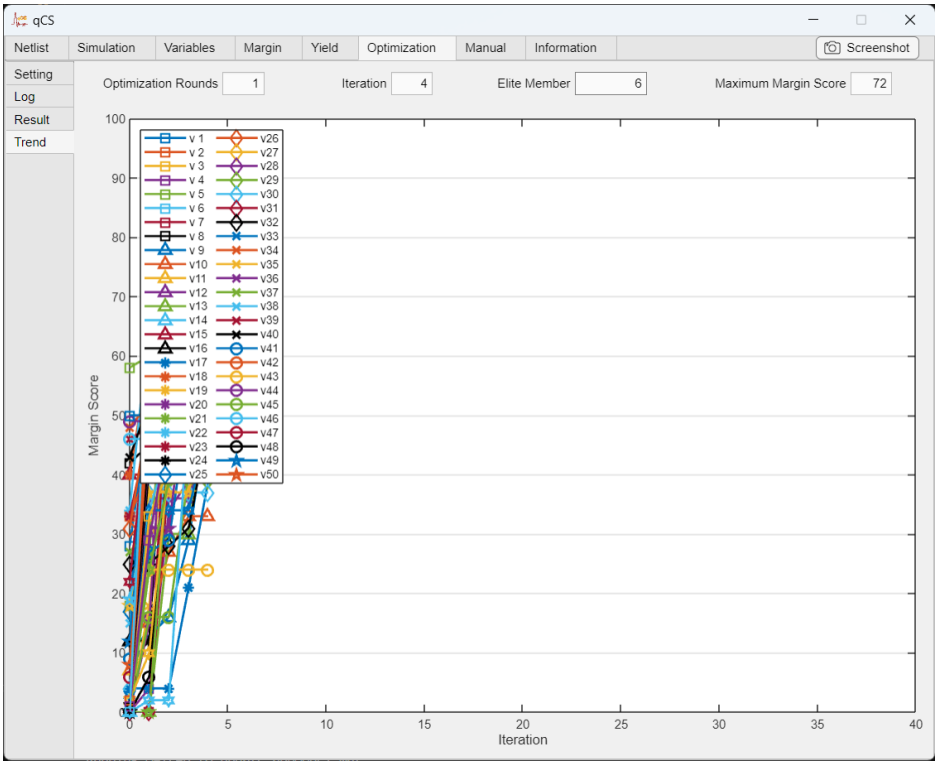


Figure 31: Optimization trend on Optimization tab (RSFQ).

8.2 AQFP

8.2.1 Loading Netlist

The AQFP flow is similar to RSFQ, with two differences: (1) the output waveform contains both positive and negative pulses (logic 1 and logic 0), and (2) the Pulse Settings sub-tab automatically enables *Negative Pulses* when AQFP is auto-detected from the netlist content. Use `AQFP_BUFFER_CHAIN_JOSIM.cir` from the `Netlists/` folder.

In this example the AQFP junctions are unshunted. Click Read Netlist (Fig. 32).

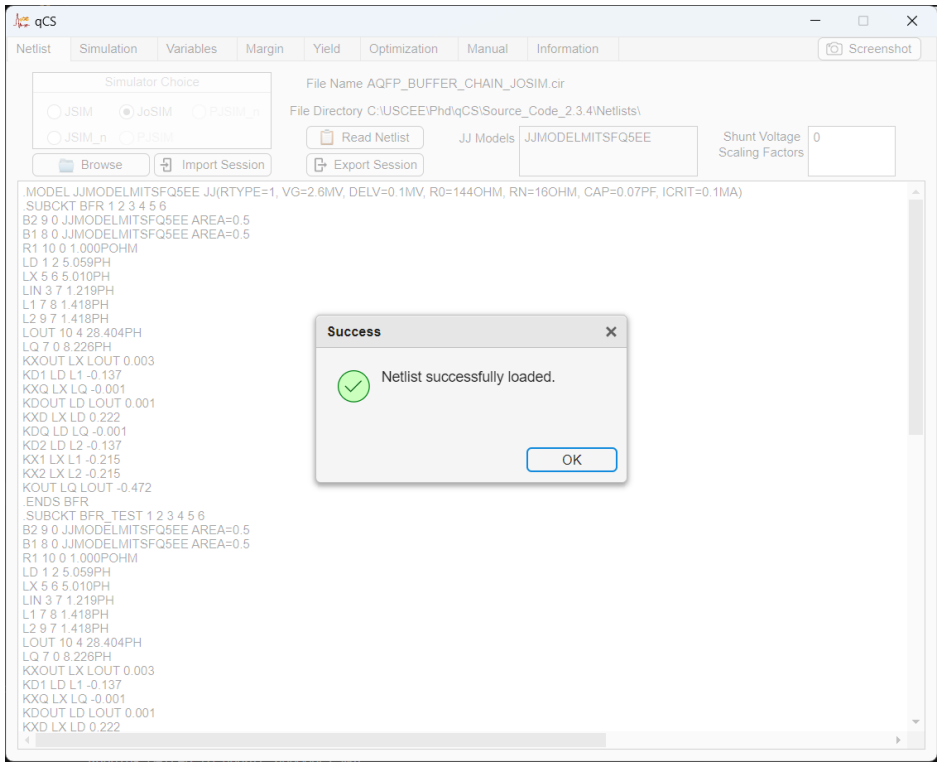


Figure 32: Netlist tab after reading the netlist (AQFP).

8.2.2 Simulation

Proceed to Simulation and run. Select an output node where both positive and negative pulses are visible (Fig. 33).

8.2.3 Selecting Components

On Variables, add two JJs from BFR_TEST. As in the RSFQ walkthrough we use the 7-column legacy layout with Group = 1 for both JJs (Fig. 34).

8.2.4 Critical Margin Calculation

On the Margin tab, the Pulse Settings sub-tab is selected with *Negative Pulses* enabled. Both positive and negative pulse-height / width fields are shown. Click Fill; the auto-generated pattern is a sequence of +1 (positive pulse) and −1 (negative pulse) triples — one window per detected pulse, with sign chosen by polarity. Check overlays the detected pulses (Fig. 36).

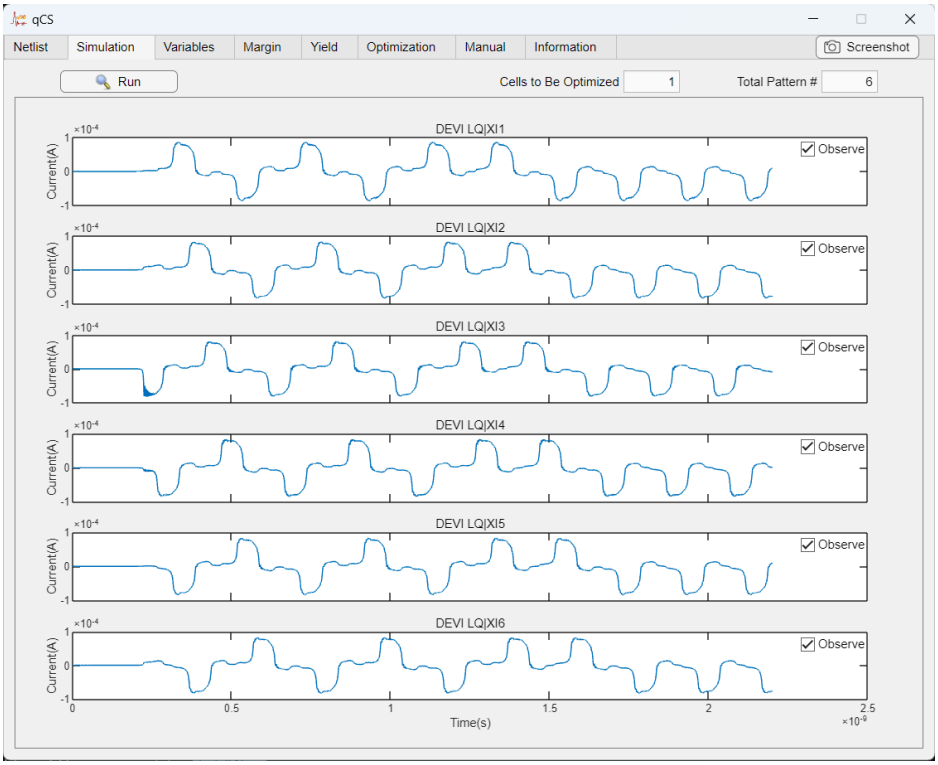
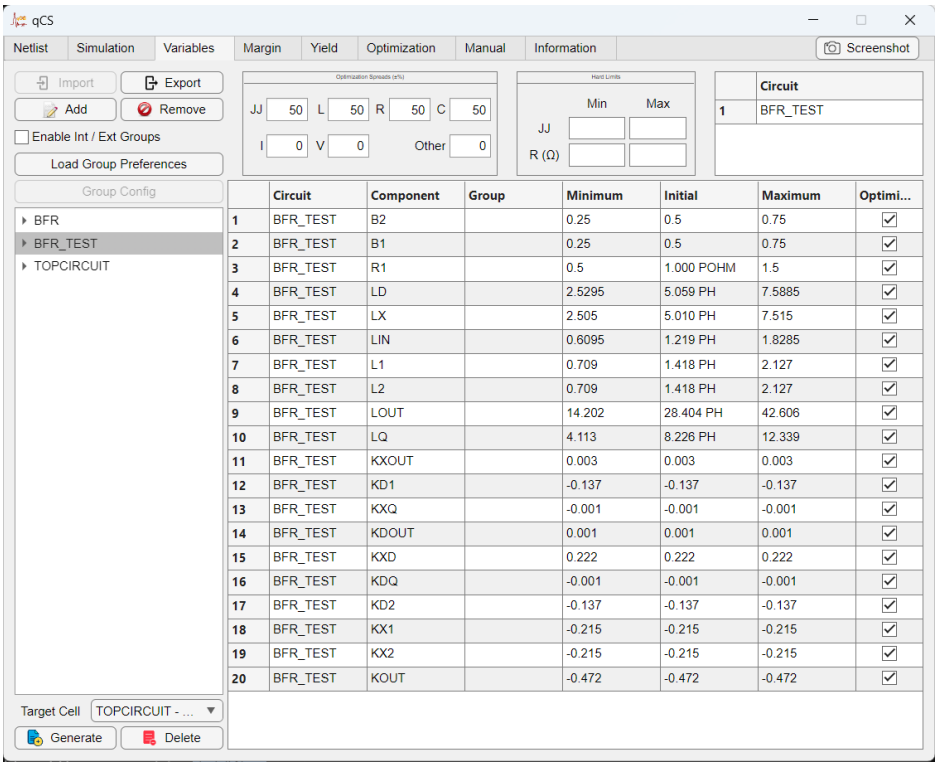


Figure 33: Simulate tab after completing the simulation (AQFP).



	Circuit	Component	Group	Minimum	Initial	Maximum	Optimi...
1	BFR_TEST	B2		0.25	0.5	0.75	☑
2	BFR_TEST	B1		0.25	0.5	0.75	☑
3	BFR_TEST	R1		0.5	1.000 POHM	1.5	☑
4	BFR_TEST	LD		2.5295	5.059 PH	7.5885	☑
5	BFR_TEST	LX		2.505	5.010 PH	7.515	☑
6	BFR_TEST	LIN		0.6095	1.219 PH	1.8285	☑
7	BFR_TEST	L1		0.709	1.418 PH	2.127	☑
8	BFR_TEST	L2		0.709	1.418 PH	2.127	☑
9	BFR_TEST	LOUT		14.202	28.404 PH	42.606	☑
10	BFR_TEST	LQ		4.113	8.226 PH	12.339	☑
11	BFR_TEST	KXOUT		0.003	0.003	0.003	☑
12	BFR_TEST	KD1		-0.137	-0.137	-0.137	☑
13	BFR_TEST	KXQ		-0.001	-0.001	-0.001	☑
14	BFR_TEST	KDOUT		0.001	0.001	0.001	☑
15	BFR_TEST	KXD		0.222	0.222	0.222	☑
16	BFR_TEST	KDQ		-0.001	-0.001	-0.001	☑
17	BFR_TEST	KD2		-0.137	-0.137	-0.137	☑
18	BFR_TEST	KX1		-0.215	-0.215	-0.215	☑
19	BFR_TEST	KX2		-0.215	-0.215	-0.215	☑
20	BFR_TEST	KOUT		-0.472	-0.472	-0.472	☑

Figure 34: Variables tab after adding components (AQFP).

Click Calculate. The Margin Table populates with lower / upper margin bounds per parameter row, in the same format as RSFQ §8.1.4; see Fig. 26 for a representative Margin

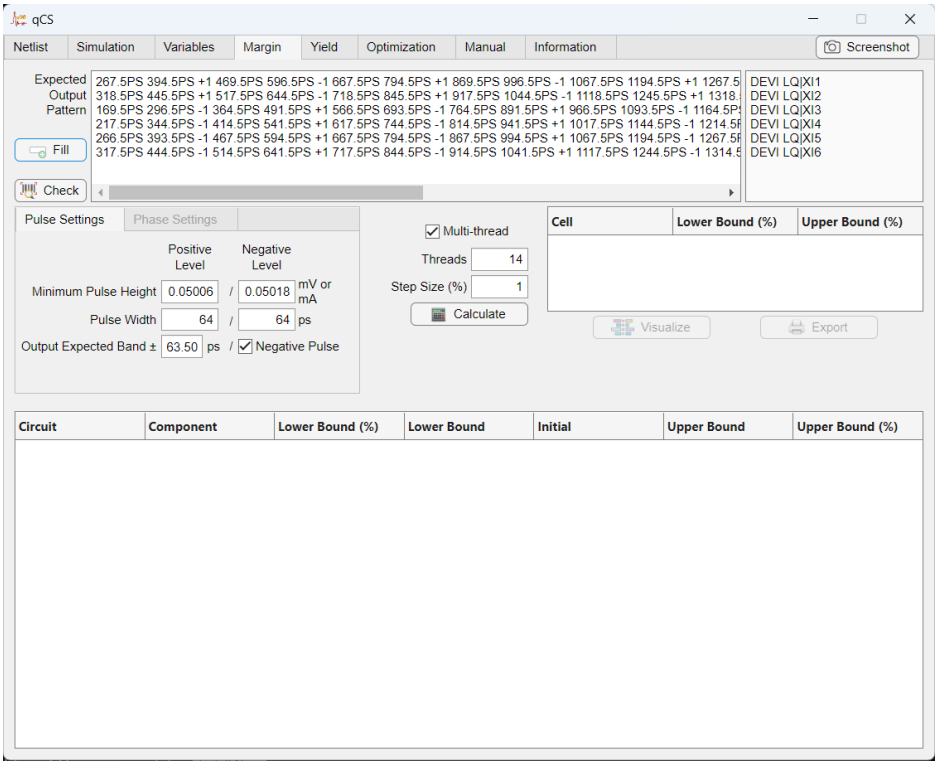


Figure 35: Margin tab after providing the specifications (AQFP).

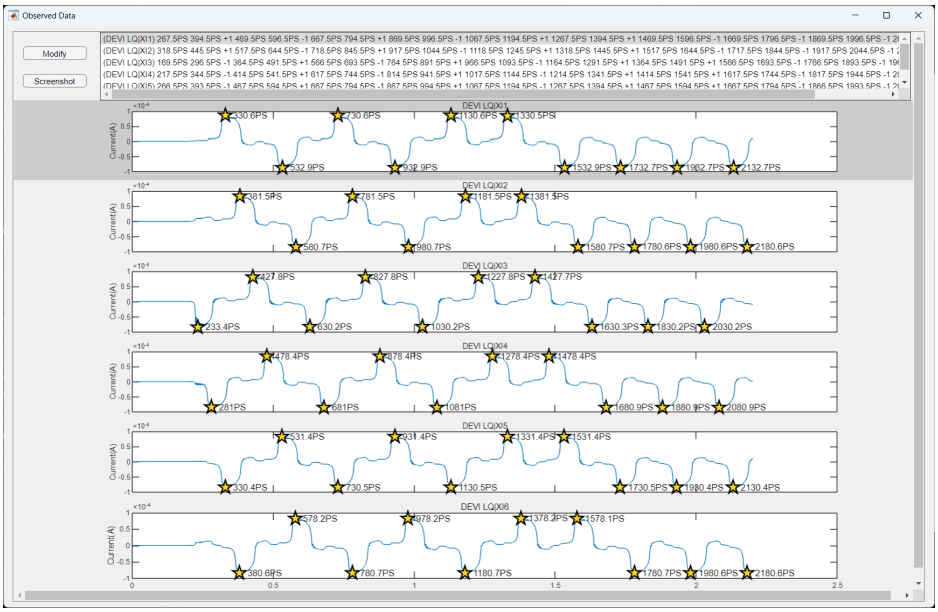


Figure 36: Waveform window after using the Check button on the Margin tab (AQFP).

Table screenshot (the layout is identical here, only the rows differ).

8.2.5 Yield Analysis

Pulse Settings (with *Negative Pulses* enabled) and the expected ±-pulse pattern are inherited automatically on the Yield tab. Set MC Sample # and click Calculate; the bar graph reports the fraction of MC samples that still satisfy the AQFP pattern. The on-screen layout

matches RSFQ §8.1.5 (see Fig. 27 for the representative screenshot); the only AQFP-specific point is that each sample is judged against both pulse polarities.

8.2.6 Critical Margin Optimization

On the Optimization tab, click Optimize. The ANPS-FW flow (PSO swarm, auto-generated optimized netlists, best-version selection by smallest Euclidean distance where margins tie) is identical to RSFQ §8.1.6; the only AQFP-specific point is that the inner cost loop runs the Pulse Settings detector with *Negative Pulses* on, so each candidate is scored against both polarities. The Settings, Log, Result, and Trend sub-tab layouts are unchanged from the RSFQ walkthrough — see Fig. 28–31 for the representative screenshots.

This concludes the qCS AQFP JoSIM example.

8.3 RQL

8.3.1 Loading Netlist

The RQL example exercises the Phase Settings analysis path. RQL output nodes carry continuous phase trajectories, so qCS auto-detects RQL and selects the Phase Settings sub-tab automatically on the Margin / Yield / Optimization tabs. Use `RQL_ANDOR_JOSIM.cir` from the `Netlists/` folder; the testbench wraps the `PCL_OA2` AND/OR cell behind four JTL stages and exposes ten PHASE outputs across the data path.

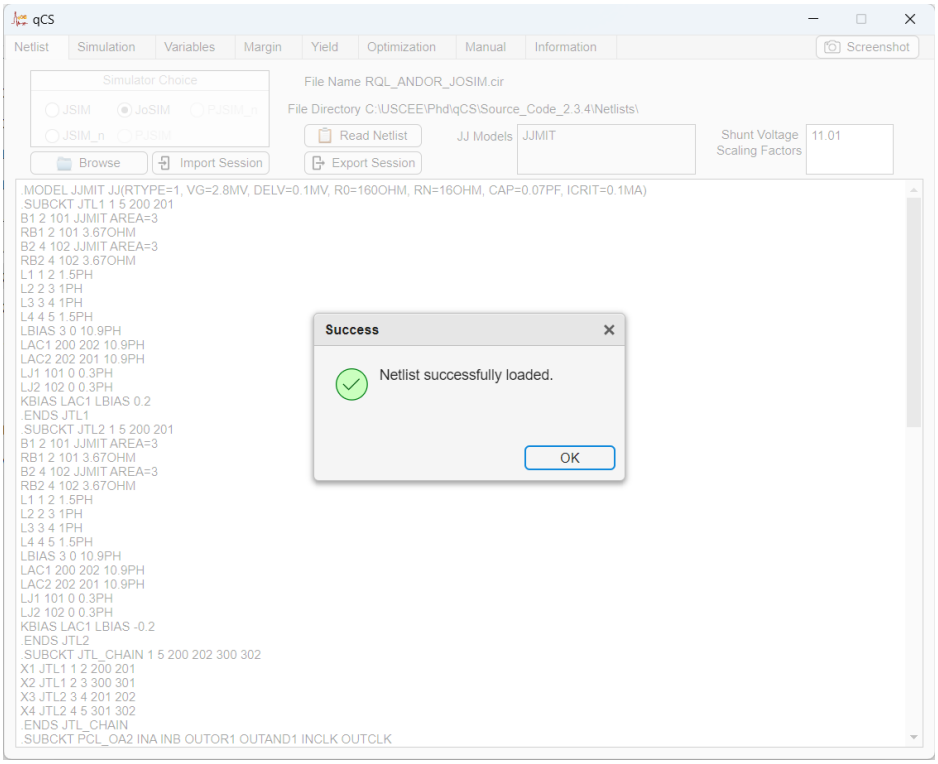


Figure 37: Netlist tab after reading the netlist (RQL): `RQL_ANDOR_JOSIM.cir` loaded with subcircuits `JTL1`, `JTL2`, `JTL_CHAIN`, and the `PCL_OA2` AND/OR cell.

8.3.2 Simulation

Proceed to Simulation. Tick the *Observe* checkbox on each of the ten PHASE output nodes (the four JTL stages `B1.X1.X-BA1`–`B1.X4.X-BA1`, the AND/OR cell internal nodes `OA2-INA`, `OA2-INB`, `OA2-OUTOR`, `OA2-OUTAND`, and the two termination nodes `TERM-OR`, `TERM-AND`); each trajectory rises by 2π at every logic-1 transition and falls by 2π at every logic-0 (Fig. 38). The *Total Pattern #* counter reflects the number of observed outputs (here 10).

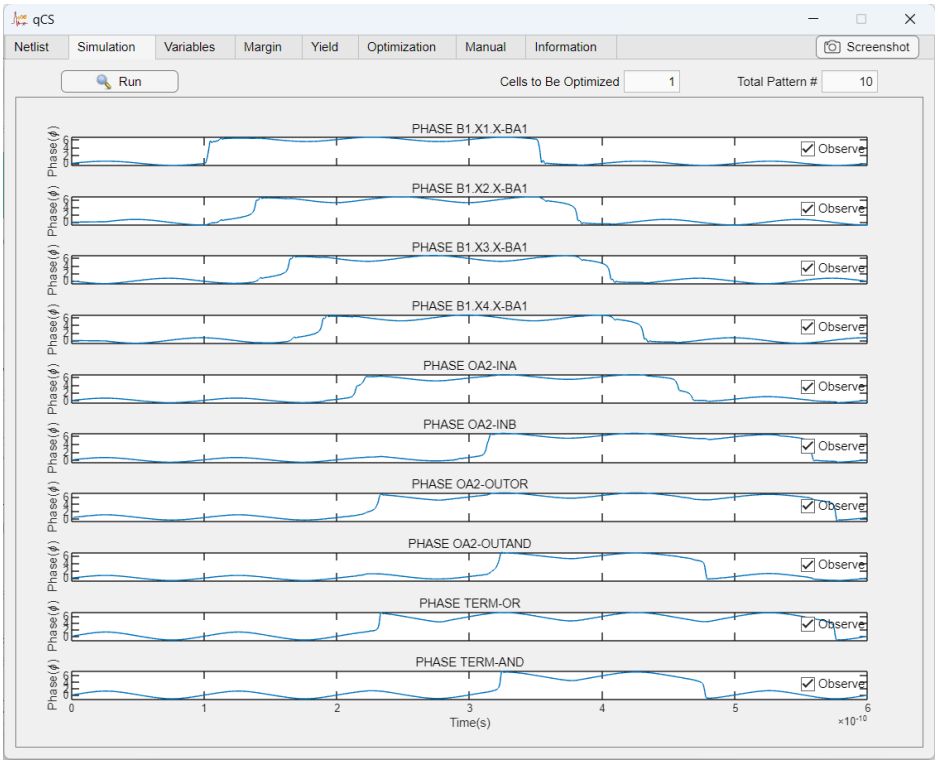


Figure 38: Simulate tab after completing the simulation (RQL): all ten observed PHASE outputs of the AND/OR testbench.

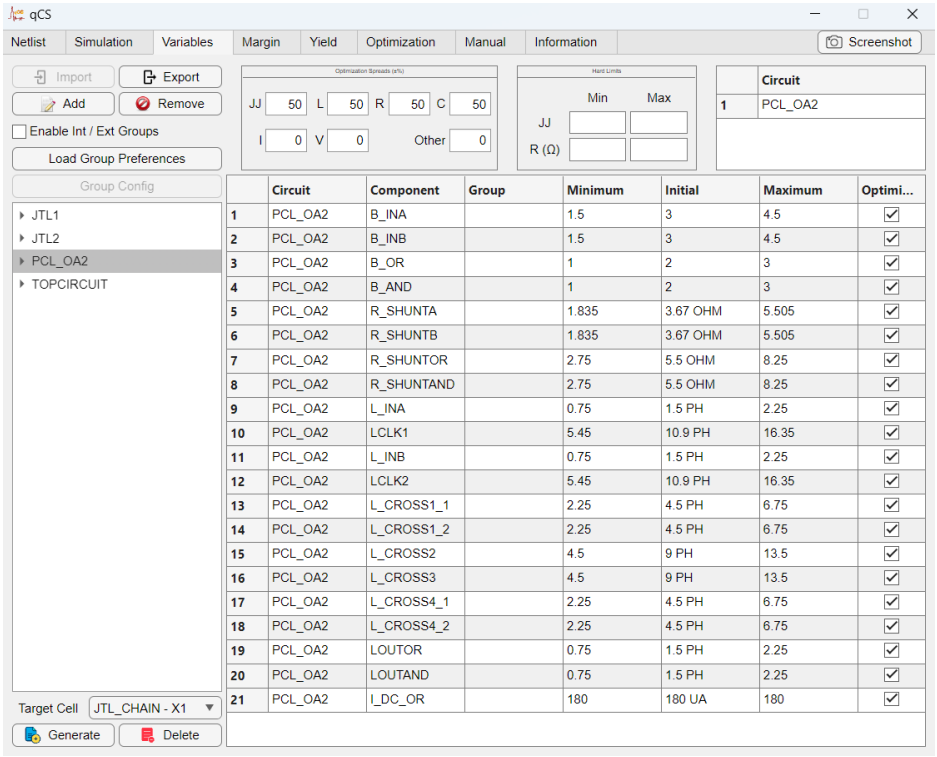
8.3.3 Selecting Components

On Variables, click Add and pick the `PCL_OA2` cell's components: four JJs (`B_INA`, `B_INB`, `B_OR`, `B_AND`), four shunt resistors (`R_SHUNTA`, `R_SHUNTB`, `R_SHUNTOR`, `R_SHUNTAND`), the input / cross-coupled / clock inductors (`L_INA`, `LCLK1`, `L_INB`, `LCLK2`, `L_CROSS1_1`–`L_CROSS4_2`, `LOUTOR`, `LOUTAND`), and the OR-side DC bias `I_DC_OR` (21 rows total, Fig. 39). The 7-column legacy layout is sufficient here (no JJ chains in the AND/OR cell), so *Enable Int / Ext Groups* stays off and same-Cell rows fold automatically.

8.3.4 Critical Margin Calculation

The Phase Settings sub-tab is selected automatically. *Phase Jump Threshold* defaults to 5 rad. Click Fill; the auto-generated pattern is a sequence of `+1` (positive phase jump, logic-1) and `-1` (negative phase jump, logic-0) triples — one window per detected jump, with sign chosen by direction. Check overlays the detected jumps (Fig. 41).

Click Calculate. The Margin Table populates with lower / upper margin bounds per parameter row, in the same format as RSFQ §8.1.4; see Fig. 26 for a representative Margin Table screenshot (the layout is identical here, only the rows differ).



	Circuit	Component	Group	Minimum	Initial	Maximum	Optimi...
1	PCL_OA2	B_INA		1.5	3	4.5	☑
2	PCL_OA2	B_INB		1.5	3	4.5	☑
3	PCL_OA2	B_OR		1	2	3	☑
4	PCL_OA2	B_AND		1	2	3	☑
5	PCL_OA2	R_SHUNTA		1.835	3.67 OHM	5.505	☑
6	PCL_OA2	R_SHUNTB		1.835	3.67 OHM	5.505	☑
7	PCL_OA2	R_SHUNTOR		2.75	5.5 OHM	8.25	☑
8	PCL_OA2	R_SHUNTAND		2.75	5.5 OHM	8.25	☑
9	PCL_OA2	L_INA		0.75	1.5 PH	2.25	☑
10	PCL_OA2	LCLK1		5.45	10.9 PH	16.35	☑
11	PCL_OA2	L_INB		0.75	1.5 PH	2.25	☑
12	PCL_OA2	LCLK2		5.45	10.9 PH	16.35	☑
13	PCL_OA2	L_CROSS1_1		2.25	4.5 PH	6.75	☑
14	PCL_OA2	L_CROSS1_2		2.25	4.5 PH	6.75	☑
15	PCL_OA2	L_CROSS2		4.5	9 PH	13.5	☑
16	PCL_OA2	L_CROSS3		4.5	9 PH	13.5	☑
17	PCL_OA2	L_CROSS4_1		2.25	4.5 PH	6.75	☑
18	PCL_OA2	L_CROSS4_2		2.25	4.5 PH	6.75	☑
19	PCL_OA2	LOUTOR		0.75	1.5 PH	2.25	☑
20	PCL_OA2	LOUTAND		0.75	1.5 PH	2.25	☑
21	PCL_OA2	I_DC_OR		180	180 UA	180	☑

Figure 39: Variables tab after adding the PCL_OA2 cell's components (RQL).

8.3.5 Yield Analysis

The auto-selected Phase Settings sub-tab and the +1 / −1 phase-jump pattern are inherited on the Yield tab. Set MC Sample # and click Calculate; the bar graph reports the fraction of MC samples whose detected phase jumps still match the expected sequence. The on-screen layout matches RSFQ §8.1.5 (see Fig. 27 for the representative screenshot); the only RQL-specific point is that scoring uses the Phase Settings detector instead of the Pulse Settings detector.

8.3.6 Critical Margin Optimization

Click Optimize on the Optimization tab. The ANPS-FW flow (PSO swarm, auto-generated optimized netlists, best-version selection by smallest Euclidean distance where margins tie) is identical to RSFQ §8.1.6; the RQL-specific point is that the inner cost loop uses the Phase Settings detector, so each candidate netlist is scored against detected phase jumps rather than voltage pulses. The Settings, Log, Result, and Trend sub-tab layouts are unchanged from the RSFQ walkthrough — see Fig. 28–31 for the representative screenshots.

This concludes the qCS RQL JoSIM example.

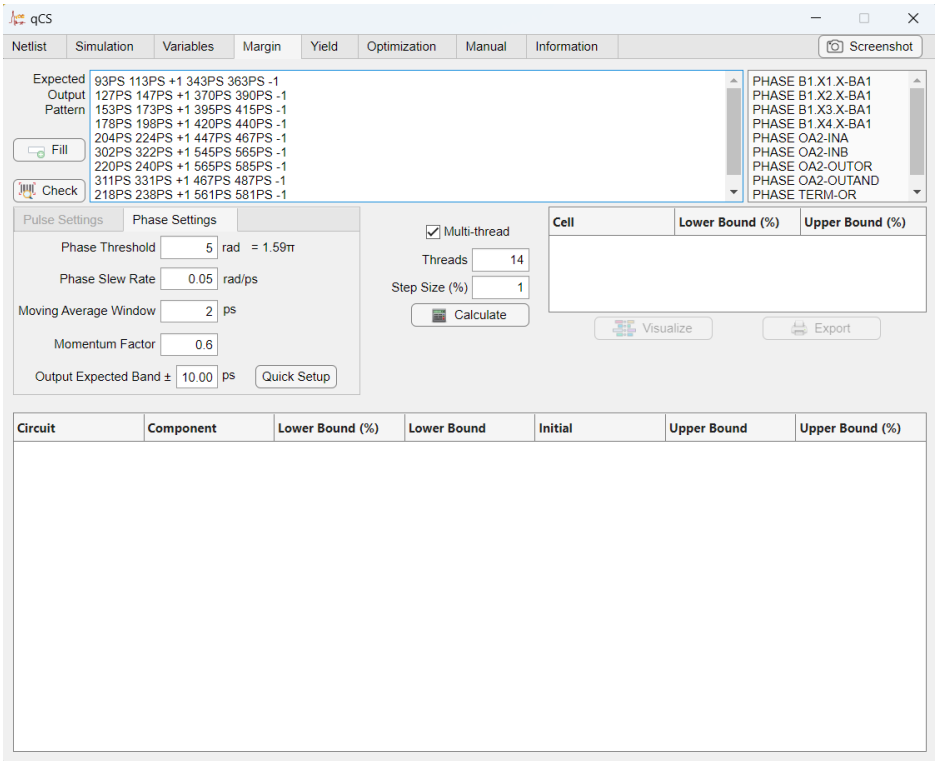


Figure 40: Margin tab after providing the specifications (RQL): Phase Settings filled with *Phase Threshold* = 5 rad $\approx 1.59\pi$, *Phase Slew Rate* = 0.05 rad/ps, *Moving Average Window* = 2 ps, *Momentum Factor* = 0.6, *Output Expected Band* = $\pm$10 ps; ten expected output patterns are populated, one per observed PHASE signal.

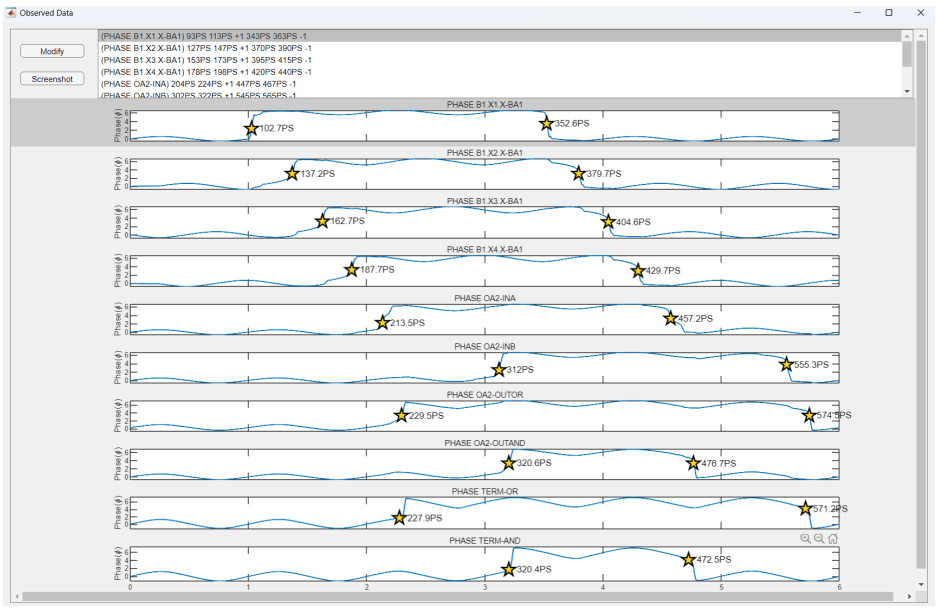


Figure 41: Waveform window opened by the Check button (RQL): one panel per observed PHASE signal; yellow star markers tag each detected phase-jump location and timestamp.

8.4 Fast Phase Logic (FPL)

The FPL walkthrough exercises the **Serial JJ chain** feature, the Phase Settings analysis path, and **Ext Group** mirroring all at once. The example netlist is `FPL_AND_JOSIM.cir` from the `Netlists/` folder; the `AND` subcircuit holds several JJs that should be rewritten into three serial chains, two of which are tied together via a shared Ext Group, plus a pair of standalone ext-only JJ mirrors.

8.4.1 Loading Netlist

Click Browse on the Netlist tab and select `FPL_AND_JOSIM.cir` from the `Netlists/` folder. The netlist text appears in the editor panel (compare with Fig. 19 for the empty state and Fig. 20 for the post-open state shown in the RSFQ walkthrough). Before proceeding, the user is free to edit component values or add commands directly in the panel.

Click Read Netlist to convert the netlist into the internal object model. As with every netlist, the shunt voltage scaling factor is automatically calculated from the first Josephson junction's AREA and shunt resistor and propagated to all JJs sharing the same model. A confirmation message appears (Fig. 42); the auto-detected pulse mode is PHASE, since every `.PRINT` on this netlist reports a phase quantity, which is what later auto-selects the Phase Settings sub-tab on the Margin / Yield / Optimization tabs.

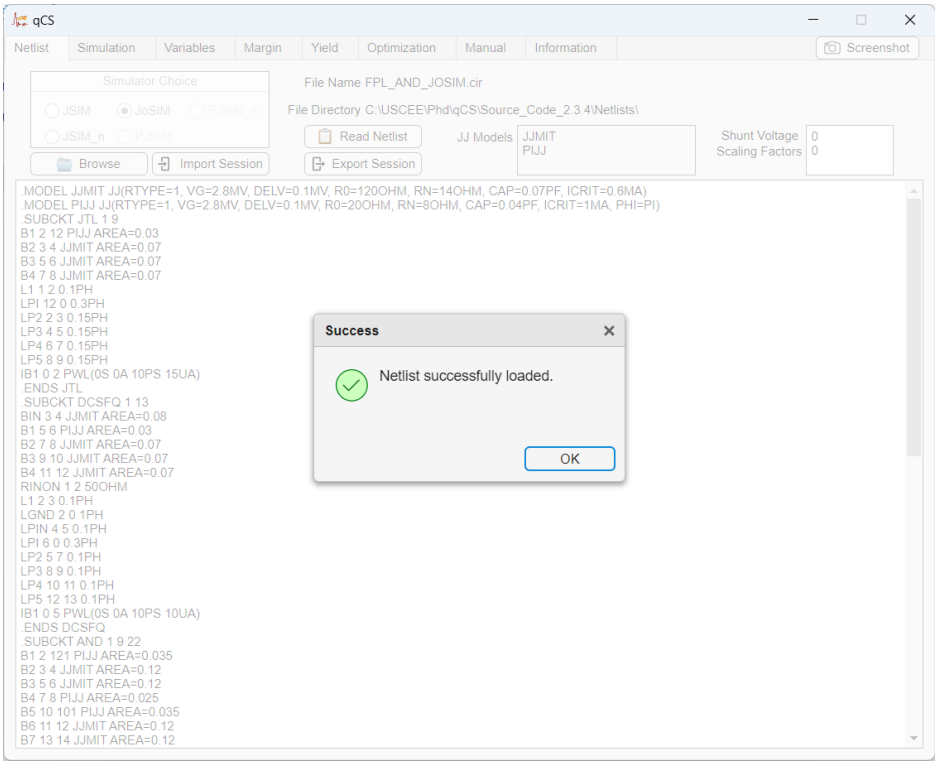


Figure 42: Netlist tab after reading the netlist (FPL). The `AND` subcircuit (visible at the bottom of the editor panel) holds the JJ chain that this walkthrough will configure.

8.4.2 Simulation

Run the simulation. The AND cell exposes three phase output nodes along its data path; tick the *Observe* checkbox on each so all three trajectories are plotted at once. Every trace

shows the staircase of 2π jumps that mark each clocked logic transition (Fig. 43). The *Total Pattern #* counter at the top right reflects the number of observed output patterns (here 3).

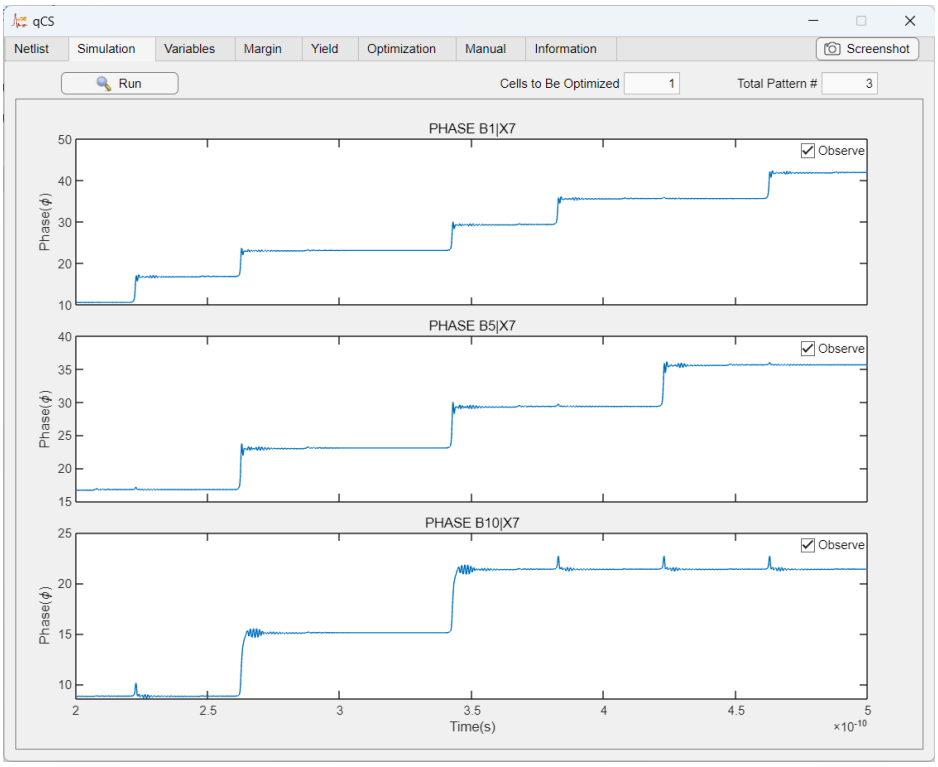


Figure 43: Simulate tab after completing the simulation (FPL): three phase output nodes (`PHASE B1|X7`, `PHASE B5|X7`, `PHASE B10|X7`) observed simultaneously.

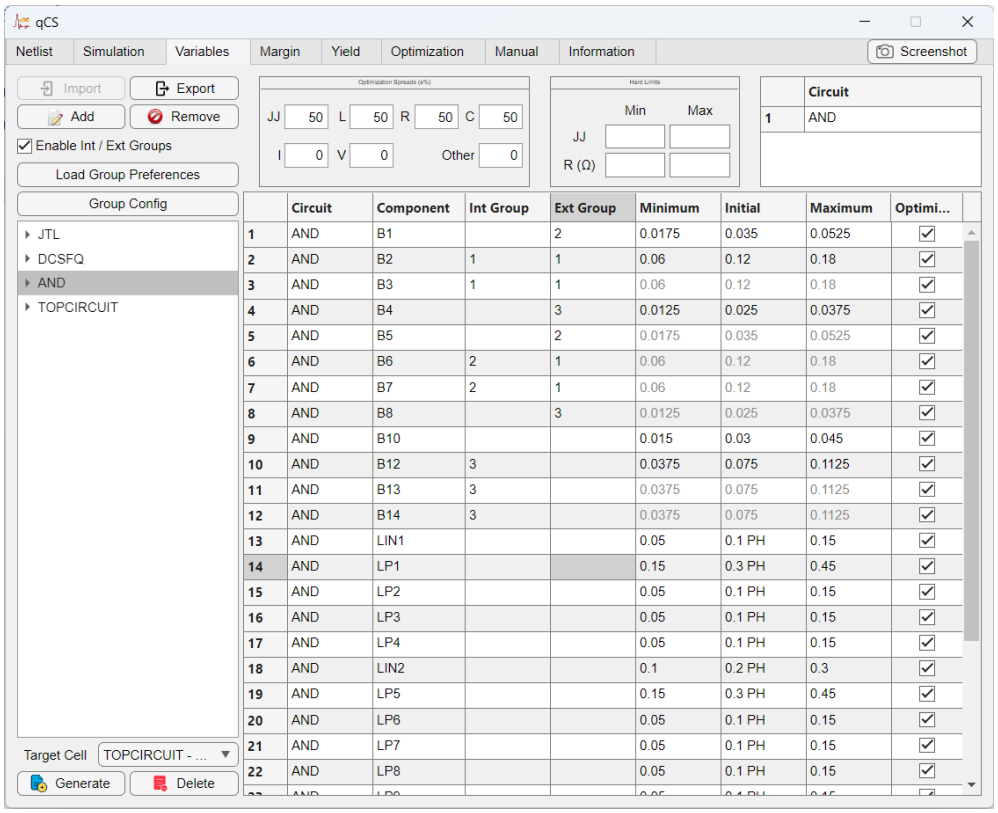
8.4.3 Selecting Components and Configuring the Chain

On Variables, tick *Enable Int / Ext Groups* in the top-left so the parameter table switches into the 8-column layout. Add the AND cell's components; the table populates with the JJs `B1–B8`, `B10`, `B12–B14`, the inductors `LIN1`, `LP1–LP4`, `LIN2`, `LP5–LP11`, and the bias resistors `IB1–IB3`. Assign the group memberships shown in Fig. 44:

- B2, B3 → Int Group 1, Ext Group 1
- B6, B7 → Int Group 2, Ext Group 1 (sharing Ext 1 with Group 1 ties the two chains together so they scale jointly during optimization)
- B12, B13, B14 → Int Group 3 (standalone chain, no Ext Group)
- B1, B5 → Ext Group 2 only (Int Group blank). This makes them an *ext-only* mirror pair: `B5` mirrors `B1` at scan time but no chain rewrite occurs.
- B4, B8 → Ext Group 3 only (Int Group blank). Same idea as above.
- `B10` and all inductors / bias resistors stay ungrouped.

Whenever Int Group is filled, the Min / Initial / Max columns of the non-first members in that group grey out to indicate they share the first member's parameter slot at scan time.

Click Group Config to open the Group Config dialog (Fig. 45). qCS analyses the assignments and lists four buckets:



	Circuit	Component	Int Group	Ext Group	Minimum	Initial	Maximum	Optimi...
1	AND	B1		2	0.0175	0.035	0.0525	☑
2	AND	B2	1	1	0.06	0.12	0.18	☑
3	AND	B3	1	1	0.06	0.12	0.18	☑
4	AND	B4		3	0.0125	0.025	0.0375	☑
5	AND	B5		2	0.0175	0.035	0.0525	☑
6	AND	B6	2	1	0.06	0.12	0.18	☑
7	AND	B7	2	1	0.06	0.12	0.18	☑
8	AND	B8		3	0.0125	0.025	0.0375	☑
9	AND	B10			0.015	0.03	0.045	☑
10	AND	B12	3		0.0375	0.075	0.1125	☑
11	AND	B13	3		0.0375	0.075	0.1125	☑
12	AND	B14	3		0.0375	0.075	0.1125	☑
13	AND	LIN1			0.05	0.1 PH	0.15	☑
14	AND	LP1			0.15	0.3 PH	0.45	☑
15	AND	LP2			0.05	0.1 PH	0.15	☑
16	AND	LP3			0.05	0.1 PH	0.15	☑
17	AND	LP4			0.05	0.1 PH	0.15	☑
18	AND	LIN2			0.1	0.2 PH	0.3	☑
19	AND	LP5			0.15	0.3 PH	0.45	☑
20	AND	LP6			0.05	0.1 PH	0.15	☑
21	AND	LP7			0.05	0.1 PH	0.15	☑
22	AND	LP8			0.05	0.1 PH	0.15	☑

Figure 44: Variables tab after adding the AND components and assigning Int Groups 1–3 and Ext Groups 1–3 as described above (FPL).

- **Ext Group 1**: *Group 1 (AND): B2, B3* + *Group 2 (AND): B6, B7* — the two Int Group chains share Ext Group 1, so they appear behind one gear icon and one chain detail dialog (because their chain-length parameter is co-optimised).
- **Group 3 (AND)**: B12, B13, B14 — the standalone chain, behind its own gear icon.
- **Ext Group 2 (Int empty)**: B1, B5 — shown greyed and gear-less (no chain to configure; mirroring is automatic).
- **Ext Group 3 (Int empty)**: B4, B8 — same as above.

Click the gear next to *Ext Group 1 (Groups 1, 2)* to open its chain detail dialog (Fig. 46). Because both Group 1 and Group 2 have $N_{initial}$ = 2 JJs each and share Ext Group 1, the dialog opens at $N_{initial}$ = 2 and the user sets N_{min} = 2, N_{max} = 4 with both *Insert inter-JJ inductor* and *Calculate margin for internal L* left unchecked. Click OK, then click the gear next to *Group 3* to open the second chain detail dialog (Fig. 47): $N_{initial}$ = 3, set N_{min} = 3, N_{max} = 5, tick *Insert inter-JJ inductor* to generate bridging inductors between consecutive chain JJs. Click OK on the chain dialog and then OK on the Group Config dialog; qCS rewrites the netlist with the canonical chain primaries and runs a lightweight re-simulation.

After the commit, the Variables table is repopulated with the canonical chain-expansion rows: `B1_int1_ext1`, `B2_int1_ext1` for Group 1; `B1_int2_ext1`, `B2_int2_ext1` for Group 2; `B1_int3`, `B2_int3`, `B3_int3` for Group 3 (Fig. 48). The ext-only mirror pairs (B1/B5, B4/B8) keep their user-entered names; their mirror linkage is enforced at scan time without changing the netlist topology.

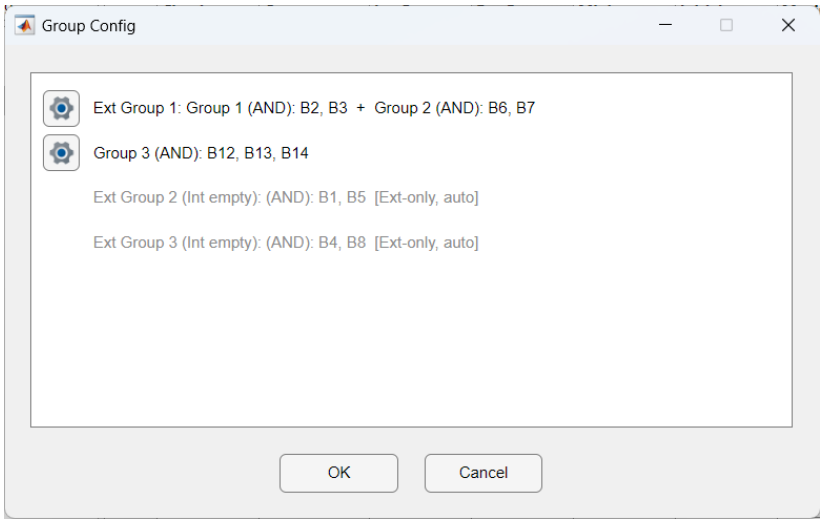


Figure 45: Group Config dialog: one row per Int-Group bucket (gear-enabled for the chain configs) plus greyed gear-less rows for the ext-only mirror pairs.

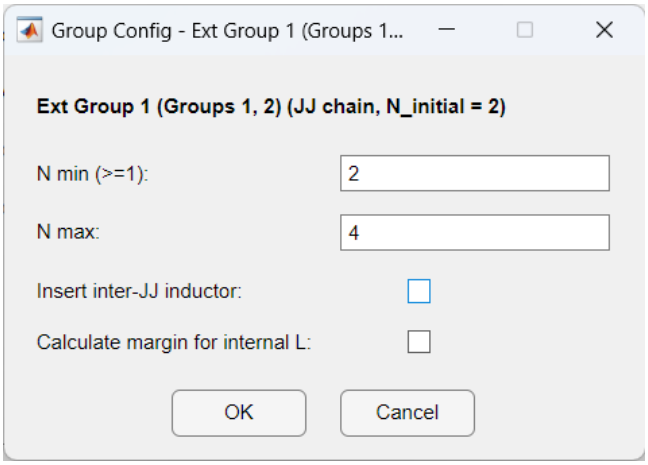


Figure 46: Chain detail dialog for *Ext Group 1 (Groups 1, 2)*: shared chain length $N \in [2, 4]$ with $N_{\text{initial}} = 2$; bridging inductor and internal-L margin both off.

8.4.4 Critical Margin Calculation

The Phase Settings sub-tab on the Margin tab is selected automatically (every `.PRINT` on this netlist reports a `PHASE` quantity). Click Fill, then Check, then Calculate. The margin scan reports per-physical-junction margins at each chain's current *N* value: Group 1 / Group 2 contribute rows `B1_int1_ext1`, `B2_int1_ext1` and `B1_int2_ext1`, `B2_int2_ext1` (two JJs each at *N* = 2); Group 3 contributes `B1_int3`, `B2_int3`, `B3_int3` (three JJs at *N* = 3); the standalone JJs and inductors fill the rest. See Fig. 18 in §5.7 for the full bar graph.

After Calculate completes, the top-right *Cell* panel summarises the AND cell's overall margin window (−22% / +11%), the Margin Table fills in with per-component bounds, and the Visualize / Export buttons become active (Fig. 51). Click Visualize to open the Margin Results bar graph (Fig. 52), which renders the same data as horizontal bars and includes the chain-expansion rows (`AND-B1-int1-ext1`, `AND-B2-int1-ext1`, `AND-B1-int2-ext1`, `AND-B2-int2-ext1` for Groups 1 / 2 sharing Ext Group 1, plus `AND-B1-int3` through `AND-B3-int3` for the standalone Group 3) alongside the non-chain JJs, inductors, and bias resistors.

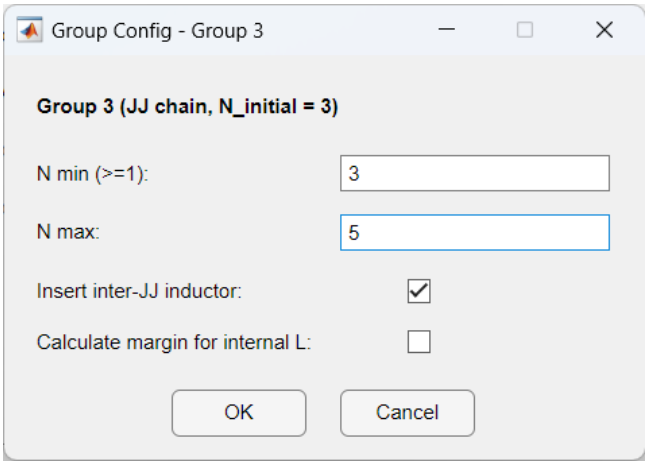


Figure 47: Chain detail dialog for *Group 3*: chain length $N \in [3, 5]$ with $N_{\text{initial}} = 3$ and bridging inductor enabled.

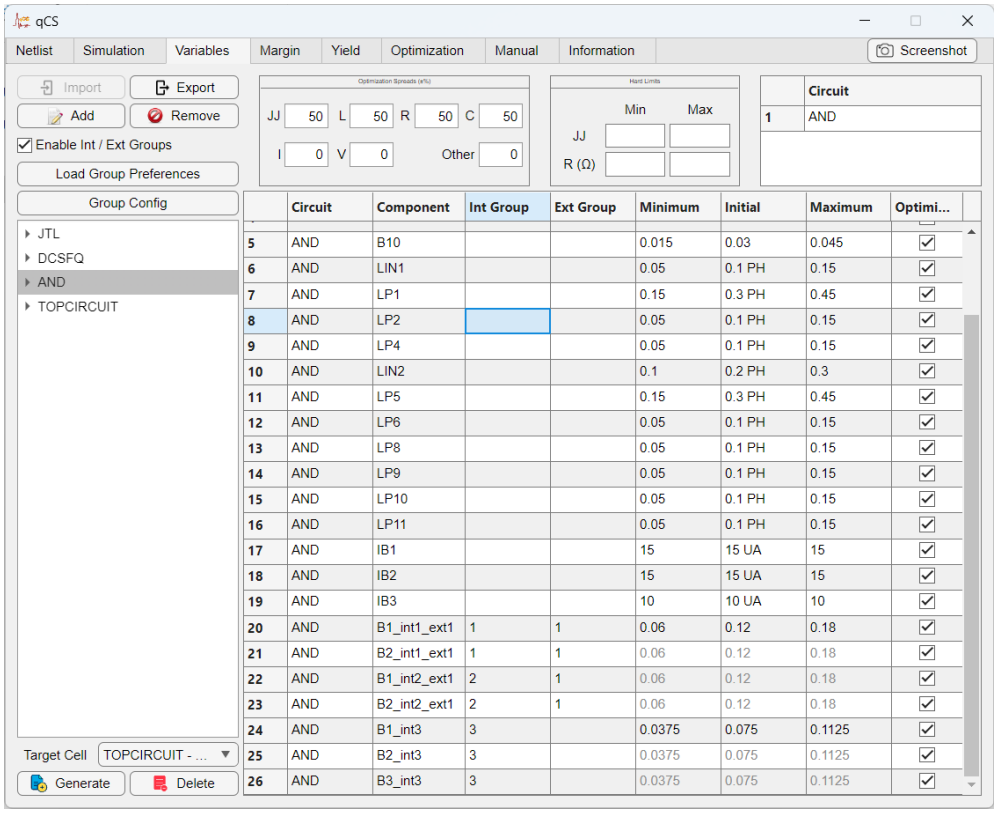


Figure 48: Variables tab after the Group Config commit: chain JJs now appear under the canonical names `B<n>_int<I>_ext<E>` (for ext-bound chains) or `B<n>_int<I>` (for the standalone chain).

8.4.5 Yield Analysis

The auto-selected Phase Settings sub-tab and the +1 / −1 phase-jump pattern are inherited on the Yield tab. Set MC Sample # and click Calculate; the bar graph reports the fraction of MC samples whose detected phase jumps still match the expected sequence at each chain's current *N* value. The on-screen layout matches RSFQ §8.1.5 (see Fig. 27 for the representative screenshot); the only FPL-specific point is that the inner scoring uses the

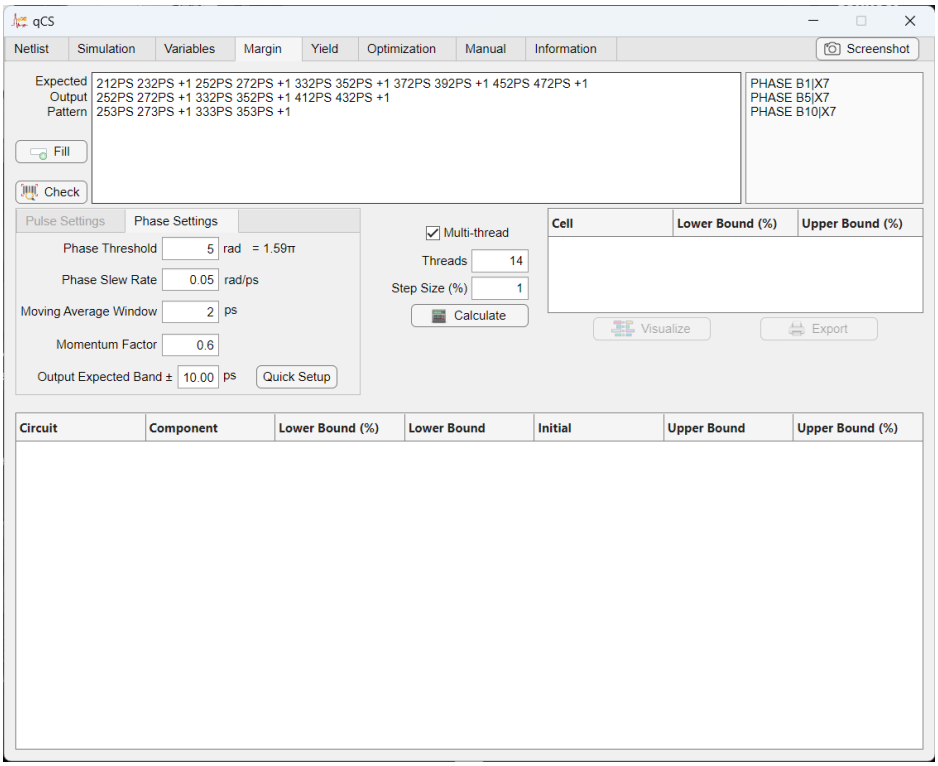


Figure 49: Margin tab after providing the specifications (FPL): Phase Settings filled with *Phase Threshold* = 5 rad $\approx 1.59\pi$, *Phase Slew Rate* = 0.05 rad/ps, *Moving Average Window* = 2 ps, *Momentum Factor* = 0.6, *Output Expected Band* = $\pm$10 ps; the expected output pattern is populated for the three observed PHASE signals (B1 | X7, B5 | X7, B10 | X7).

Phase Settings detector and the chain-expansion rows participate per-physical-junction.

8.4.6 Critical Margin Optimization

Click Optimize. The ANPS-FW flow (PSO swarm, auto-generated optimized netlists, best-version selection by smallest Euclidean distance where margins tie) is identical to RSFQ §8.1.6. The FPL-specific points are: (i) the swarm explores the chain-area continuous parameter *and* the chain-length integer parameter *N* (a discrete Gaussian sampler is used internally for *N*); (ii) the inner cost loop uses the Phase Settings detector; (iii) with *Enable Mirrors Affect Optimize* on (auto-enabled when an Ext Group is set), Ext-Group secondaries also contribute to the cost. The Settings, Log, Result, and Trend sub-tab layouts are unchanged from the RSFQ walkthrough — see Fig. 28–31 for the representative screenshots.

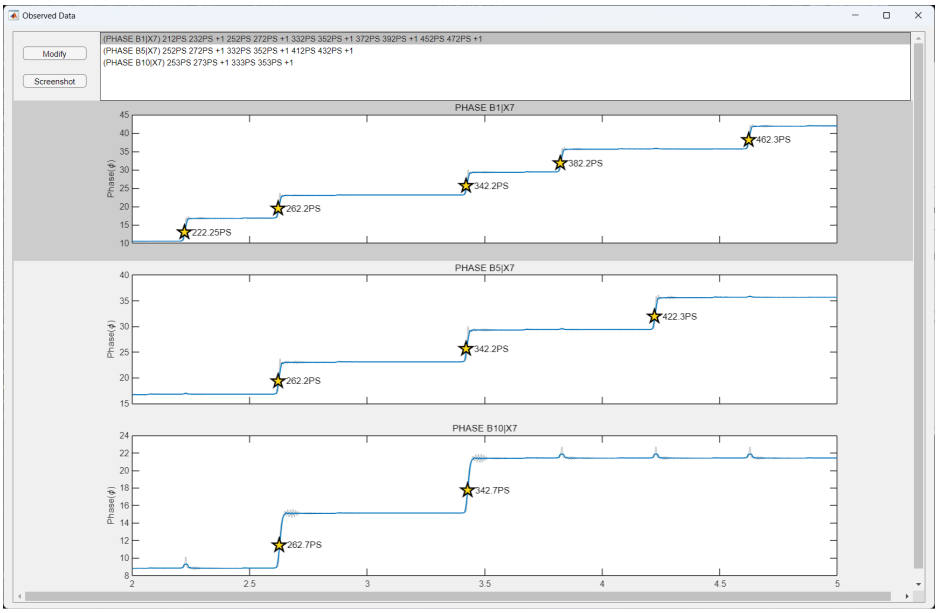


Figure 50: Waveform window opened by the Check button (FPL): one panel per observed PHASE signal; yellow star markers tag each detected phase-jump location (where the cumulative phase crosses the midpoint of the 5 rad threshold) and its timestamp.

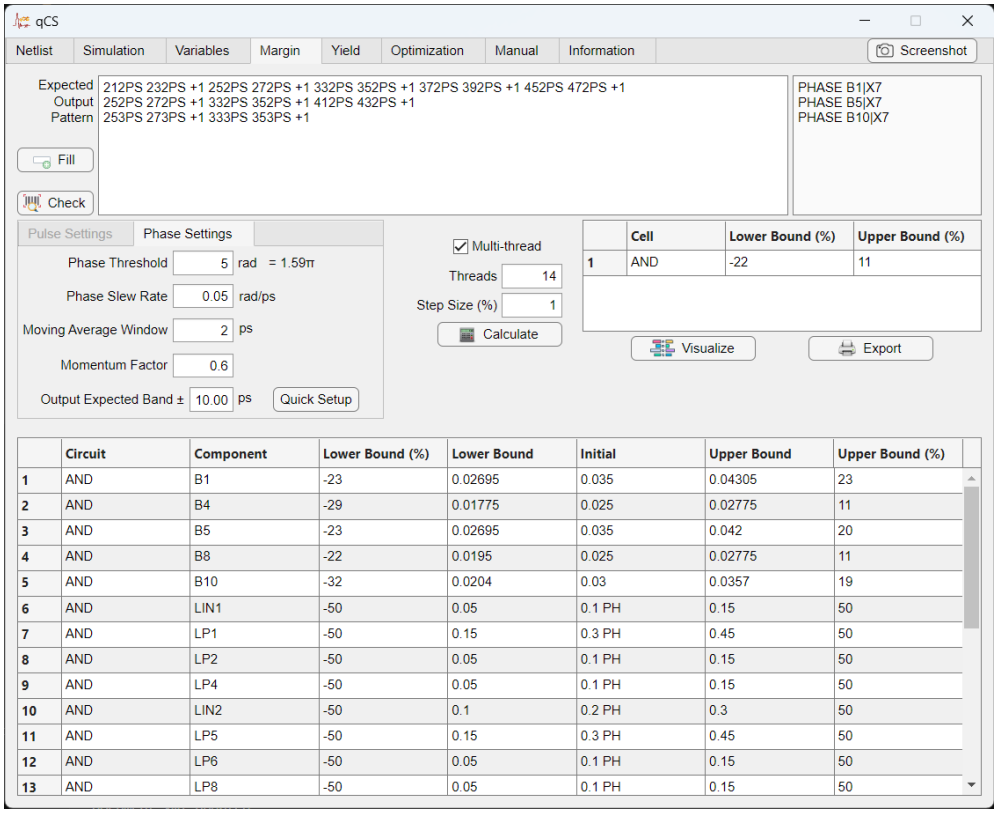


Figure 51: Margin tab after Calculate (FPL): the Cell panel reports the AND cell's overall margin window (−22% / +11%); the Margin Table shows per-component Lower Bound %, Lower Bound, Initial, Upper Bound, Upper Bound % for the non-chain JJs (B1, B4, B5, B8, B10), the inductors (LIN1, LP1–LP11, LIN2), and bias resistors. Rows whose bounds saturate at ±50% are not the margin-limiting elements.

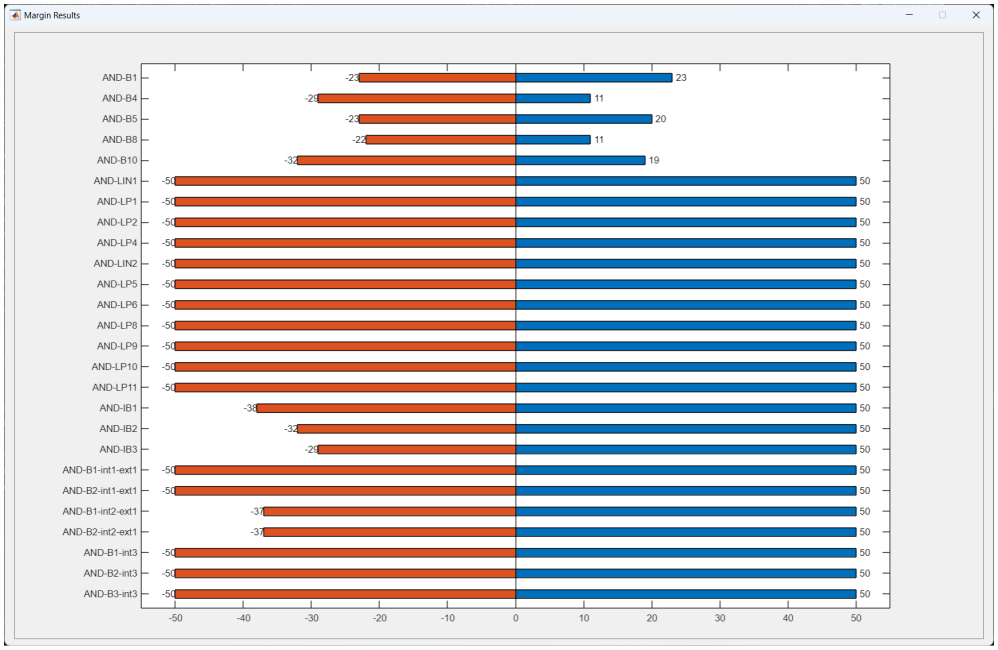


Figure 52: Margin Results bar graph from Visualize (FPL): orange = lower margin %, blue = upper margin %. Chain-expansion JJs appear at the bottom with the canonical `<Cell>-B<n>-int<I>[-ext<E>]` label format alongside the cell's non-chain components.

This concludes the qCS Fast Phase Logic JoSIM example.

9 Notes

Known issues

Future plan

Referencing qCS

There are two papers and a book chapter for qCS. One paper and a book chapter are dedicated to the algorithm, including performance metrics, while the second paper focuses on the tool itself. The related links for them will be provided here once they are published.

- Book chapter: Karamuftuoglu, M.A., Nazar Shahsavani, S., Pedram, M. (2023). Margin Optimization of Single Flux Quantum Logic Cells. In: Topaloglu, R.O. (eds) Design Automation of Quantum Computers. Springer, Cham. https://doi.org/10.1007/978-3-031-15699-1_6
- Algorithm paper: M. A. Karamuftuoglu, S. N. Shahsavani and M. Pedram, "Margin and Yield Optimization of Single Flux Quantum Logic Cells Using Swarm Optimization Techniques," in IEEE Transactions on Applied Superconductivity, vol. 33, no. 1, pp. 1-10, Jan. 2023, Art. no. 1300110, doi: 10.1109/TASC.2022.3223883.
- Tool paper: Mustafa Altay Karamuftuoglu, Haolin Cong, and Massoud Pedram, "qCS: Design Analysis and Optimization Tool for Superconductor Circuits", in preparation.

Useful links

- *qCS Code*: https://github.com/Karamuft/qCS
- *SPORT Lab*: https://sportlab.usc.edu/
- *JSIM and JSIM_n (1)*: https://github.com/coldlogix/jsim
- *JSIM and JSIM_n (2) (under "Free Tools")*: https://www.sun-magnetics.com/resources/
- *JoSIM*: https://github.com/JoeyDelp/JoSIM
- *PJSIM*: http://www.nashilab.ynu.ac.jp/eng/pjsim.html

Acknowledgments

The authors would like to thank Dr. Scott Holmes (IARPA) for feedback on tool features and Christopher Ayala (Yokohama National University) for discussions about Adiabatic Quantum-Flux-Parametron (AQFP) circuits during the development.